PulseOps por dentro — Volume 2: código explicado linha a linha

Sumário

Como ler este volume

- Ordem sugerida
- O que não é explicado linha a linha (exclusões)

D1 — Base do pacote: versão, configuração, relógio, logs e linha de comando

- `src/pulseops/__init__.py`
- `src/pulseops/config.py`
- `src/pulseops/clock.py`
- `src/pulseops/logs.py`
- `src/pulseops/cli.py`

D2 — Domínio: as regras sem banco e sem rede

- `src/pulseops/domain/outcomes.py`
- `src/pulseops/domain/status_codes.py`
- `src/pulseops/domain/monitor_rules.py`
- `src/pulseops/domain/health.py`
- `src/pulseops/domain/metrics.py`

D3 — Persistência: modelos, sessão e migrações

- `src/pulseops/db/models.py`
- `src/pulseops/db/session.py`
- `alembic.ini`
- `migrations/env.py`
- `migrations/script.py.mako`
- `migrations/versions/0001_esquema_inicial.py` (explicado por padrão)

D4 — Segurança: política de destino, transporte protegido e senhas

- `src/pulseops/security/url_policy.py`
- `src/pulseops/security/transport.py`
- `src/pulseops/security/passwords.py`

D5 — Cliente HTTP: o verificador e o envio de webhooks

- `src/pulseops/checks/checker.py`
- `src/pulseops/notifications/webhook.py`

D6 — Serviços: os casos de uso

- `src/pulseops/services/errors.py`
- `src/pulseops/services/auth.py`
- `src/pulseops/services/monitors.py`
- `src/pulseops/services/incidents.py`
- `src/pulseops/services/alerts.py`
- `src/pulseops/services/processing.py`
- `src/pulseops/services/queries.py`

D7 — Worker: o processo que verifica e avisa

- `src/pulseops/worker/runner.py`
- `src/pulseops/worker/__main__.py`

D8 — Web: API JSON, painel e login

- `src/pulseops/web/__init__.py`
- `src/pulseops/web/app.py`
- `src/pulseops/web/deps.py`
- `src/pulseops/web/health.py`
- `src/pulseops/web/schemas.py`
- `src/pulseops/web/api.py`
- `src/pulseops/web/pages.py`
- `src/pulseops/web/templating.py`
- `src/pulseops/web/charts.py`
- Templates (`src/pulseops/web/templates/`)
- `src/pulseops/web/static/app.js`
- `src/pulseops/web/static/style.css`

D9 — Laboratório de falhas e scripts

- `lab/__init__.py`
- `lab/target.py`
- `lab/receiver.py`
- `scripts/demo.sh`
- `scripts/demo_flow.py`
- `scripts/ensaio_carga.py`
- `scripts/ensaio_carga.sh`
- `scripts/conferir_referencias.py`
- `scripts/capturar_telas.mjs`

D10 — Docker, Compose, CI e configuração do projeto

- `Dockerfile`
- `.dockerignore`
- `compose.yaml`
- `compose.demo.yaml`
- `.env.example`
- `.gitignore`
- `pyproject.toml`
- `.github/workflows/ci.yml`
- Arquivos de dados do repositório

D11 — Os testes

- `tests/conftest.py`
- `tests/support.py`
- `tests/support_db.py`
- Padrão 1: regra pura, sem banco
- Padrão 2: um cenário de ponta a ponta com o worker real
- Padrão 3: provar que o banco recusa o impossível
- Padrão 4: segurança com rede simulada no nível do socket
- Padrão 5: migrações num banco vazio
- Os demais arquivos

Como ler este volume

Item	Valor
Material	*PulseOps por dentro — Volume 2: código explicado linha a linha*
Versão do material	1.0
Data de geração	2026-10-22
Commit do código explicado	`a46ee8835aacde9f5b5bb40a5a516e23d98a0e76`
Volume 1	partes A, B, C, E, F, G, H, I e J

Este volume é a **parte D** do guia. Ele percorre os arquivos de código autoral do PulseOps, um por um. O código aparece em trechos pequenos, copiados por script do commit acima, com a numeração de linhas do arquivo. Logo depois de cada trecho vem a explicação.

Para cada arquivo há uma ficha com as oito informações pedidas:

1. **Caminho e finalidade** (o título da seção e a primeira linha da ficha);
2. **Código com numeração** (os trechos);
3. **Explicação das linhas significativas** (depois de cada trecho);
4. **Dados**: de onde vêm e de que tipo são;
5. **Efeito** de cada função, classe e bloco (nas explicações);
6. **Relações** com outros arquivos;
7. **Falhas** que podem ocorrer;
8. **Testes** que verificam o comportamento.

Linhas estruturais repetitivas (imports, colunas parecidas, testes no mesmo formato) são agrupadas. Quando um padrão se repete, o primeiro é explicado por completo e os seguintes são indicados pelos intervalos de linhas, com o que muda.

Ordem sugerida

Não é preciso ler na ordem do sumário. O caminho do volume 1 (caminho de estudo) sugere começar por `domain/health.py`, depois seguir um check pelo worker. Use o sumário como índice.

O que não é explicado linha a linha (exclusões)

Item	Motivo
Dependências externas (FastAPI, SQLAlchemy, HTTPX, Pydantic, argon2-cffi, pytest…)	não são código do projeto; o uso delas é explicado onde aparecem
`uv.lock`	arquivo de lock gerado pelo `uv`; lista versões exatas, sem lógica
`.venv/`, `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`	artefatos gerados localmente, fora do Git

Item	Motivo
`migrations/versions/0001_esquema_inicial.py`	gerado pelo `alembic revision --autogenerate`; é explicado **por padrão** (um exemplo de cada tipo de comando), e a correspondência com os modelos é garantida por teste
`docs/imagens/*.png`, `docs/evidencias/*`	imagens e saídas de execução, não código
`docs/estudo/*.pdf`, `study/build/`	o próprio material de estudo gerado
`docker/no-extra-ca.pem`, `scripts/.gitkeep`, os `__init__.py` vazios	arquivos vazios; a existência deles é explicada onde importa
`docs/*.md`, `README.md`, `CLAUDE.md`, `LICENSE`	documentação em texto, não código
`study/*` (o gerador deste guia)	ferramenta do material de estudo, não do PulseOps; o funcionamento está descrito no topo de `study/build.py` e `study/captura_sql.py`

D1 — Base do pacote: versão, configuração, relógio, logs e linha de comando

`src/pulseops/__init__.py`

Finalidade. Marca a pasta `src/pulseops` como pacote Python e define a versão.

src/pulseops/**init**.py

```
1 """PulseOps: monitoramento explicável de APIs HTTP."""
2
3 __version__ = "0.1.0"
```

- Linha 1: *docstring* do módulo, a descrição curta que aparece em `help(pulseops)`.
- Linha 3: `__version__` é uma convenção da comunidade Python. É lida pelo worker para montar o `User-Agent` (`PulseOps/0.1.0`) e pela parte B do volume 1 para conferir a instalação.

Dados. Uma constante de texto. **Relações.** `worker/__main__.py`, `web/app.py`. **Falhas.** Nenhuma. **Testes.** Indiretamente, por todos (o pacote precisa importar).

Os outros `__init__.py` (`checks/`, `db/`, `domain/`...) são **vazios**: só existem para que cada pasta seja um subpacote importável (`from pulseops.domain import health`).

`src/pulseops/config.py`

Finalidade. Lê toda a configuração das variáveis de ambiente `PULSEOPS_*` e valida os valores na inicialização.

Dados. Texto vindo do ambiente do processo ou do arquivo `.env`, convertido pelo Pydantic para `str`, `int`, `float` e `bool`. **Relações.** Instanciado em `web/app.py`, `worker/__main__.py`, `cli.py` e `migrations/env.py`; os valores viram parâmetros dos serviços. **Testes.** `tests/unit/test_config.py` (`test_minimum_interval_depends_on_demo_mode`, `test_allowed_ports_parsing`, `test_unknown_timezone_is_rejected`).

src/pulseops/config.py, linhas 1–14

```
1 """Configuração lida de variáveis de ambiente com prefixo PULSEOPS_."""
2
3 from functools import cached_property
4 from zoneinfo import ZoneInfo, ZoneInfoNotFoundError
5
6 from pydantic import Field, field_validator
7 from pydantic_settings import BaseSettings, SettingsConfigDict
8
9
10 class Settings(BaseSettings):
11     model_config = SettingsConfigDict(env_prefix="PULSEOPS_", env_file=".env", extra="ignore")
12
13     database_url: str = "postgresql+psycopg://pulseops:pulseops@localhost:5432/pulseops"
14     log_level: str = "INFO"
```

- Linha 3: `cached_property` calcula um atributo na primeira leitura e guarda o resultado (usado nas linhas 48–70).

- Linha 4: `ZoneInfo` representa um fuso horário da base IANA (`America/Sao_Paulo`).
- Linha 7: `BaseSettings` é a classe do pacote `pydantic-settings` que preenche os campos a partir do ambiente.
- Linha 10: `Settings` herda de `BaseSettings`: cada atributo anotado é uma variável de configuração.
- Linha 11: `env_prefix="PULSEOPS_"` faz o campo `database_url` ser lido de `PULSEOPS_DATABASE_URL`. `env_file=".env"` também lê o arquivo `.env`, se existir. `extra="ignore"` ignora variáveis desconhecidas (por exemplo `POSTGRES_PASSWORD`, que é do Compose) em vez de falhar.
- Linha 13: o valor padrão serve para desenvolvimento local; no Compose ele é sempre substituído. `postgresql+psycopg` diz ao SQLAlchemy para usar o driver psycopg 3.

`src/pulseops/config.py, linhas 16–35`

```
16    demo_mode: bool = False
17    # Pares host:porta separados por vírgula. Só têm efeito com demo_mode ligado.
18    demo_allowed_hosts: str = ""
19    allowed_ports: str = "80,443,8080,8443"
20
21    cookie_secure: bool = False
22    session_ttl_minutes: int = Field(default=480, ge=5, le=7 * 24 * 60)
23    display_timezone: str = "America/Sao_Paulo"
24
25    worker_concurrency: int = Field(default=20, ge=1, le=200)
26    worker_tick_seconds: float = Field(default=1.0, gt=0, le=10)
27    worker_stale_after_seconds: int = Field(default=30, ge=5)
28    heartbeat_interval_seconds: float = Field(default=5.0, gt=0)
29
30    webhook_timeout_seconds: float = Field(default=5.0, gt=0, le=30)
31    webhook_max_attempts: int = Field(default=6, ge=1, le=20)
32    webhook_backoff_base_seconds: float = Field(default=5.0, gt=0)
33    webhook_backoff_max_seconds: float = Field(default=300.0, gt=0)
34
35    check_retention_days: int = Field(default=30, ge=1)
```

Cada linha é um campo com tipo e padrão. `Field(default=..., ge=..., le=...)` acrescenta limites: `ge` é “maior ou igual”, `le` “menor ou igual”, `gt` “maior que”. Um valor fora do limite impede o processo de iniciar, com uma mensagem que diz qual variável está errada. Isso é melhor do que descobrir o erro horas depois.

Linhas	Grupo	O que controlam
16–19	rede	modo demonstração, exceções do laboratório e portas permitidas (política SSRF)
21–23	painel	cookie só por HTTPS, duração da sessão (5 min a 7 dias, padrão 8 h) e fuso de exibição
25–28	worker	checks simultâneos, tick, quando o worker é considerado sem sinal, intervalo do heartbeat
30–33	webhooks	timeout, tentativas e espera exponencial
35	retenção	dias que os checks ficam guardados

src/pulseops/config.py, linhas 37–46

```
37    @field_validator("display_timezone")
38    @classmethod
39    def _known_timezone(cls, value: str) -> str:
40        # ZoneInfoNotFoundError herda de KeyError, que o Pydantic não converte em erro de
41        # validação; por isso a conversão explícita para ValueError.
42        try:
43            ZoneInfo(value)
44        except (ZoneInfoNotFoundError, ValueError) as exc:
45            raise ValueError(f"fuso horário desconhecido: {value}") from exc
46        return value
```

- Linhas 37–38: `@field_validator` registra uma função que valida um campo; `@classmethod` porque roda antes de a instância existir.
- Linhas 42–45: tenta criar o `ZoneInfo`. Se o fuso não existe, converte o erro para `ValueError`. O comentário explica por quê: `ZoneInfoNotFoundError` herda de `KeyError`, e o Pydantic só transforma `ValueError` e `AssertionError` em erro de validação. Era o bug **B5**.
- Linha 46: um validador precisa devolver o valor (que poderia ser transformado).

src/pulseops/config.py, linhas 48–74

```
48    @cached_property
49    def timezone(self) -> ZoneInfo:
50        return ZoneInfo(self.display_timezone)
51
52    @cached_property
53    def allowed_port_set(self) -> frozenset[int]:
54        return frozenset(int(p) for p in self.allowed_ports.split(",") if p.strip())
55
56    @cached_property
57    def demo_allowlist(self) -> frozenset[tuple[str, int]]:
58        """Hosts nominais liberados para o laboratório. Vazio fora do modo demo."""
59        if not self.demo_mode:
60            return frozenset()
61        pairs = set()
62        for item in self.demo_allowed_hosts.split(","):
63            item = item.strip().lower()
64            if not item:
65                continue
66            host, _, port = item.rpartition(":")
67            if not host or not port.isdigit():
68                raise ValueError(f"PULSEOPS_DEMO_ALLOWED_HOSTS: use host:porta, recebido {item!r}")
69            pairs.add((host, int(port)))
70        return frozenset(pairs)
71
72    @property
73    def min_interval_seconds(self) -> int:
74        return 2 if self.demo_mode else 30
```

- Linhas 48–50: `timezone` devolve o objeto de fuso pronto, criado uma vez só.
- Linhas 52–54: transforma `"80,443,8080,8443"` em `frozenset({80, 443, 8080, 8443})`. `frozenset` é um conjunto imutável; ler `porta in conjunto` é rápido. `if p.strip()` ignora vírgulas sobrando.
- Linhas 56–70: monta a lista de exceções do laboratório. Fora do modo demo, devolve vazio **mesmo que a variável esteja preenchida** (linha 59): a exceção nunca vaza para uma instalação normal. `rpartition(":")` divide pelo **último** `:`; `lab-target:8081` vira `("lab-target", 8081)`. Um item malformado levanta erro com a variável e o valor recebido (linha 68).
- Linhas 72–74: `@property` (sem cache, porque é barato) com o intervalo mínimo: 2 s na demonstração, 30 s no modo normal.

Falhas possíveis. Variável com tipo errado (`PULSEOPS_WORKER_CONCURRENCY=abc`) ou fora do limite: o Pydantic levanta `ValidationError` e o processo não sobe. `PULSEOPS_ALLOWED_PORTS` com texto não numérico levanta `ValueError` na primeira leitura de `allowed_port_set`.

`src/pulseops/clock.py`

Finalidade. Define o relógio como uma dependência substituível. Nenhum código do PulseOps chama `datetime.now()` diretamente.

src/pulseops/clock.py

```python
"""Relógio substituível: o código de domínio nunca chama datetime.now() direto."""

import time
from datetime import UTC, datetime
from typing import Protocol


class Clock(Protocol):
    def now(self) -> datetime:
        """Instante atual em UTC, com fuso (timezone-aware)."""
        ...

    def monotonic(self) -> float:
        """Segundos de um relógio que nunca volta para trás; só serve para medir durações."""
        ...


class SystemClock:
    def now(self) -> datetime:
        return datetime.now(UTC)

    def monotonic(self) -> float:
        return time.monotonic()
```

- Linha 5: `Protocol` permite definir uma interface por **forma** (tipagem estrutural): qualquer classe com `now()` e `monotonic()` serve.
- Linhas 8–15: o protocolo. Os `...` (reticências) indicam corpo vazio; as docstrings dizem o contrato: `now()` em UTC e com fuso; `monotonic()` só para medir durações, porque nunca volta para trás (o relógio de parede pode ser ajustado pelo sistema e "voltar").
- Linhas 18–23: a implementação real. `datetime.now(UTC)` devolve um `datetime` com fuso. Um `datetime` **sem** fuso ("ingênuo") seria ambíguo; o PostgreSQL grava em `timestamptz`.

Dados. Saem `datetime` (UTC, com fuso) e `float` (segundos). **Relações.** `SystemClock` é criado em `web/app.py` e `worker/__main__.py`; os testes usam `FakeClock` (`tests/support.py`). **Falhas.** Nenhuma própria. **Testes.** Todos os testes que usam `FakeClock` dependem deste contrato; o uso de `monotonic` para medir a duração é testado em `tests/unit/test_checker.py`.

`src/pulseops/logs.py`

Finalidade. Formata os logs como JSON, acrescenta contexto (monitor, execução) e mascara segredos.

Dados. Recebe `logging.LogRecord` (mensagem, nível, campos extras). Produz uma linha de texto JSON por evento, na saída padrão. **Relações.** `configure_logging` é chamado por `web/app.py` e `worker/__main__.py`; `log_context` é usado em `services/processing.py` e `worker/runner.py`; `redact_text`

também é usado em `services/alerts.py` (URL no payload do webhook). **Testes.** `tests/unit/test_logging.py` (quatro testes: query e credenciais mascaradas, campos sensíveis, contexto em todas as linhas, formato JSON).

`src/pulseops/logs.py, linhas 12–19`

```
12 _context: ContextVar[dict[str, Any] | None] = ContextVar("log_context", default=None)
13
14 _SENSITIVE_KEYS = ("password", "senha", "token", "secret", "authorization", "cookie", "csrf")
15 _URL_QUERY = re.compile(r"(https?://[^\s?#\"']+)\?[^\s\"'#]*")
16 _URL_USERINFO = re.compile(r"(https?://)[^/\s@\"']+@")
17
18 # Atributos que todo LogRecord tem; o resto veio de `extra=` e vai para o JSON.
19 _RESERVED = set(vars(logging.LogRecord("", 0, "", 0, "", (), None))) | {"message", "asctime"}
```

- Linha 12: `ContextVar` é uma variável cujo valor é **por tarefa assíncrona**. Dois checks rodando ao mesmo tempo têm cada um o seu contexto; um não vê o `monitor_id` do outro. Com uma variável global comum, eles se misturariam.
- Linha 14: palavras que, se aparecerem no **nome** de um campo, fazem o valor ser trocado por `[redacted]`.
- Linha 15: expressão regular que encontra uma URL seguida de `?query`. O grupo `(...)` captura a parte antes do `?`.
- Linha 16: encontra `http://usuario:senha@` no começo de uma URL.
- Linha 19: cria um `LogRecord` vazio só para descobrir quais atributos todo registro tem por padrão. O que não estiver nesse conjunto veio de `extra=` e deve ir para o JSON.

`src/pulseops/logs.py, linhas 22–43`

```
22 def redact_text(text: str) -> str:
23     text = _URL_USERINFO.sub(r"\1[redacted]@", text)
24     return _URL_QUERY.sub(r"\1?[redacted]", text)
25
26
27 def redact_value(key: str, value: Any) -> Any:
28     if any(word in key.lower() for word in _SENSITIVE_KEYS):
29         return "[redacted]"
30     if isinstance(value, str):
31         return redact_text(value)
32     return value
33
34
35 @contextmanager
36 def log_context(**fields: Any) -> Iterator[None]:
37     """Acrescenta campos a todos os logs emitidos dentro do bloco (inclusive em corrotinas)."""
38     current = _context.get() or {}
39     token = _context.set({**current, **fields})
40     try:
41         yield
42     finally:
43         _context.reset(token)
```

- Linhas 22–24: troca credenciais por `[redacted]@` e a query por `?[redacted]`. `\1` repete o grupo capturado (o começo da URL).
- Linhas 27–32: decide pelo nome do campo primeiro; se não for sensível e for texto, ainda passa pela limpeza de URLs.
- Linhas 35–43: `log_context` é um *context manager* (`with log_context(...)`). Linha 38 lê o contexto atual; linha 39 grava um novo, juntando o antigo com os campos novos, e guarda o `token`; o `finally` (linha 42–43) restaura o anterior mesmo se houver exceção. Isso permite aninhar contextos.

src/pulseops/logs.py, linhas 46–74

```
46 class JsonFormatter(logging.Formatter):
47     def format(self, record: logging.LogRecord) -> str:
48         payload: dict[str, Any] = {
49             "ts": datetime.fromtimestamp(record.created, UTC).isoformat(timespec="milliseconds"),
50             "level": record.levelname,
51             "logger": record.name,
52             "msg": redact_text(record.getMessage()),
53         }
54         for key, value in (_context.get() or {}).items():
55             payload[key] = redact_value(key, value)
56         for key, value in vars(record).items():
57             if key not in _RESERVED and not key.startswith("_"):
58                 payload[key] = redact_value(key, value)
59         if record.exc_info:
60             payload["error"] = redact_text(self.formatException(record.exc_info))
61         return json.dumps(payload, ensure_ascii=False, default=str)
62
63
64 def configure_logging(level: str = "INFO") -> None:
65     handler = logging.StreamHandler()
66     handler.setFormatter(JsonFormatter())
67     root = logging.getLogger()
68     root.handlers[:] = [handler]
69     root.setLevel(level.upper())
70     # O log de acesso do uvicorn imprime a URL inteira; o nosso formatador mascara a query.
71     for name in ("uvicorn", "uvicorn.access", "uvicorn.error"):
72         logging.getLogger(name).handlers[:] = []
73         logging.getLogger(name).propagate = True
74     logging.getLogger("httpx").setLevel(logging.WARNING)
```

- Linhas 46–61: o formatador. Monta um dicionário com horário (em UTC, com milissegundos), nível, nome do logger e mensagem mascarada (linhas 48–53); acrescenta o contexto (54–55) e os campos extras (56–58); se houver exceção, acrescenta o *traceback* mascarado (59–60). `ensure_ascii=False` mantém os acentos legíveis; `default=str` converte o que o JSON não conhece (por exemplo `datetime`).
- Linhas 64–69: troca os *handlers* do logger raiz por um só, com o formatador JSON.
- Linhas 70–73: o uvicorn tem *handlers* próprios que imprimiriam a URL inteira. Removê-los e ligar `propagate` faz as linhas dele passarem pelo nosso formatador, que mascara a query.
- Linha 74: o HTTPX registra cada requisição em INFO; subir para WARNING evita uma linha por check.

Falhas possíveis. Um segredo em formato não previsto (no **caminho** da URL, ou num campo com nome inocente) não é mascarado. Está listado como limitação (volume 1, H32).

`src/pulseops/cli.py`

Finalidade. Comando `pulseops create-admin`, que cria a operadora. É o único jeito de criar usuário: não há cadastro pela web.

Dados. Nome de usuário pelo argumento `--username`; senha pelo terminal (sem eco) ou pela entrada padrão. Grava uma linha em `operators`. **Relações.** Registrado como comando em `pyproject.toml` (`[project.scripts]`); usa `services/auth.py`. Chamado pelo README e por `scripts/demo.sh`. **Testes.** `tests/integration/test_cli.py` (`test_create_admin_from_stdin`, `test_short_password_is_rejected`).

src/pulseops/cli.py, linhas 21–43

```python
async def _create_admin(username: str, password: str, *, exists_ok: bool) -> bool:
    """Devolve False se o usuário já existia e `exists_ok` foi pedido."""
    engine = create_engine(Settings().database_url, pool_size=1)
    try:
        sessions = create_session_factory(engine)
        async with sessions.begin() as session:
            if exists_ok and await session.scalar(
                select(Operator.id).where(Operator.username == username.strip())
            ):
                return False
            await create_operator(session, username, password, now=SystemClock().now())
            return True
    finally:
        await engine.dispose()


def _read_password(from_stdin: bool) -> str:
    if from_stdin:
        return sys.stdin.readline().rstrip("\n")
    first = getpass.getpass("Senha (mínimo 12 caracteres): ")
    if getpass.getpass("Repita a senha: ") != first:
        raise ValueError("as senhas não conferem")
    return first
```

- Linha 21: função assíncrona porque o acesso a banco do projeto é assíncrono.
- Linha 23: um engine próprio com uma conexão só (`pool_size=1`); o comando é curto.
- Linhas 26–32: uma transação. Com `--exists-ok`, se o usuário já existe, devolve `False` sem erro. A demonstração usa isso para poder rodar várias vezes.
- Linhas 33–34: `finally` fecha o engine em qualquer caso.
- Linhas 37–43: com `--password-stdin`, lê uma linha da entrada padrão (útil em scripts: `echo senha | pulseops ...`); senão, `getpass` pede a senha duas vezes sem mostrar o que se digita. Senha nunca vai como argumento, porque argumentos aparecem no histórico do shell e na lista de processos (docstring, linhas 3–4).

src/pulseops/cli.py, linhas 46–68

```python
def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="pulseops")
    commands = parser.add_subparsers(dest="command", required=True)
    admin = commands.add_parser("create-admin", help="cria o operador do PulseOps")
    admin.add_argument("--username", required=True)
    admin.add_argument("--password-stdin", action="store_true", help="lê a senha da entrada padrão")
    admin.add_argument(
        "--exists-ok", action="store_true", help="não falha se o usuário já existir (usado na demo)"
    )
    args = parser.parse_args(argv)

    try:
        password = _read_password(args.password_stdin)
        created = asyncio.run(_create_admin(args.username, password, exists_ok=args.exists_ok))
    except ValueError as exc:
        print(f"erro: {exc}", file=sys.stderr)
        return 1
    print(f"operador {args.username!r} {'criado' if created else 'já existia'}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

- Linhas 47–55: `argparse` monta a interface: subcomando obrigatório (`required=True`), `--username` obrigatório e duas opções booleanas (`store_true`).

- Linha 59: `asyncio.run` cria um event loop, executa a corrotina e fecha o loop.
- Linhas 60–62: erros esperados (senha curta, senhas diferentes, usuário repetido) são `ValueError`: imprime em `stderr` e devolve código 1.
- Linha 63: `!r` imprime o nome entre aspas (`'amanda'`).
- Linhas 67–68: permite rodar também como `python -m pulseops.cli`.

Falhas possíveis. Banco inacessível: a exceção do driver não é `ValueError`, então sai com *traceback* e código diferente de zero. Senha com menos de 12 caracteres: “erro: a senha precisa ter pelo menos 12 caracteres”, código 1.

D2 — Domínio: as regras sem banco e sem rede

Os arquivos de `src/pulseops/domain/` só usam a biblioteca padrão do Python. Não importam SQLAlchemy, HTTPX nem FastAPI. Por isso as regras podem ser testadas com objetos simples, em milissegundos (volume 1, parte G).

`src/pulseops/domain/outcomes.py`

Finalidade. Lista as oito categorias de resultado de um check e diz quais contam como falha do alvo.

Dados. Os valores são textos curtos (`"healthy"`, `"timeout"`…), os mesmos gravados na coluna `checks.outcome` e aceitos pela constraint `ck_checks_outcome`. **Relações.** Produzido por `checks/checker.py`; lido por `domain/health.py`, `services/processing.py`, `services/queries.py` e pelos templates (rótulos). **Testes.** `test_every_target_failure_category_counts` e `test_internal_error_changes_nothing` (`tests/unit/test_health.py`).

`src/pulseops/domain/outcomes.py, linhas 1–28`

```python
"""Categorias de resultado de uma verificação e o que conta como falha do alvo."""

from enum import StrEnum


class Outcome(StrEnum):
    HEALTHY = "healthy"
    SLOW = "slow"
    UNEXPECTED_STATUS = "unexpected_status"
    TIMEOUT = "timeout"
    CONNECTION_ERROR = "connection_error"
    TLS_ERROR = "tls_error"
    BLOCKED = "blocked"
    INTERNAL_ERROR = "internal_error"

    @property
    def is_failure(self) -> bool:
        """Falha do serviço monitorado (RF03.1). Erro interno do PulseOps não é falha do alvo."""
        return self in _FAILURES

    @property
    def is_success(self) -> bool:
        """Respondeu com status aceito, rápido ou lento (RF03.2)."""
        return self in (Outcome.HEALTHY, Outcome.SLOW)

    @property
    def label(self) -> str:
        return _LABELS[self]
```

- Linha 3: `StrEnum` (Python 3.11+) é uma enumeração cujos membros **também são textos**: `Outcome.TIMEOUT == "timeout"` é verdadeiro. Assim, o valor pode ir direto para o banco e para o JSON.
- Linhas 7–14: os oito membros. `BLOCKED` é o destino recusado pela política SSRF no momento da conexão; `INTERNAL_ERROR` é um erro do próprio PulseOps.
- Linhas 16–19: `is_failure` é uma *property*: lê-se como atributo (`outcome.is_failure`), sem parênteses. Consulta o conjunto `_FAILURES`.

- Linhas 21–24: `is_success`. **Observação honesta:** no commit explicado, essa propriedade não é usada em lugar nenhum; a regra de saúde trata "não é falha e não é erro interno" como sucesso. É código morto que poderia ser removido.
- Linhas 26–28: `label`, o texto em português para a tela.

src/pulseops/domain/outcomes.py, linhas 31–50

```
31 _FAILURES = frozenset(
32     {
33         Outcome.UNEXPECTED_STATUS,
34         Outcome.TIMEOUT,
35         Outcome.CONNECTION_ERROR,
36         Outcome.TLS_ERROR,
37         Outcome.BLOCKED,
38     }
39 )
40
41 _LABELS = {
42     Outcome.HEALTHY: "Saudável",
43     Outcome.SLOW: "Lento",
44     Outcome.UNEXPECTED_STATUS: "Status inesperado",
45     Outcome.TIMEOUT: "Timeout",
46     Outcome.CONNECTION_ERROR: "Falha de conexão/DNS",
47     Outcome.TLS_ERROR: "Falha TLS",
48     Outcome.BLOCKED: "Destino bloqueado",
49     Outcome.INTERNAL_ERROR: "Erro interno do PulseOps",
50 }
```

- Linhas 31–39: as falhas do alvo, num `frozenset`. Erro interno fica de fora de propósito (RF03.1): se o PulseOps quebrar, isso não pode virar "o serviço caiu".
- Linhas 41–50: rótulos. Um dicionário indexado pelo próprio membro.

Falhas possíveis. Um valor de texto desconhecido em `Outcome("xyz")` levanta `ValueError`; o banco já impede que um valor assim seja gravado.

`src/pulseops/domain/status_codes.py`

Finalidade. Interpreta o texto da lista de status aceitos (`"200-299,304"`).

Dados. Entrada: texto digitado pela operadora. Saída: `AcceptedStatusCodes`, uma tupla de faixas `(início, fim)`. **Relações.** Usado em `domain/monitor_rules.py` (validação) e `services/processing.py` (monta o alvo do check). **Testes.** `tests/unit/test_status_codes.py` (`test_ranges_and_single_codes`, `test_invalid_specs_are_rejected`).

src/pulseops/domain/status_codes.py

```
1  """Interpretação da lista de status HTTP aceitos, por exemplo "200-299,301"."""
2
3  from dataclasses import dataclass
4
5  MAX_SPEC_LENGTH = 100
6
7
8  @dataclass(frozen=True)
9  class AcceptedStatusCodes:
10     ranges: tuple[tuple[int, int], ...]
11
12     def __contains__(self, code: object) -> bool:
13         return isinstance(code, int) and any(low <= code <= high for low, high in self.ranges)
14
15     def __str__(self) -> str:
```

```
16         return ",".join(str(lo) if lo == hi else f"{lo}-{hi}" for lo, hi in self.ranges)
17
18
19 def parse_status_codes(spec: str) -> AcceptedStatusCodes:
20     """Converte o texto em faixas. Levanta ValueError com mensagem para o operador."""
21     spec = spec.replace(" ", "")
22     if not spec:
23         raise ValueError("informe pelo menos um status, por exemplo 200-299")
24     if len(spec) > MAX_SPEC_LENGTH:
25         raise ValueError(f"no máximo {MAX_SPEC_LENGTH} caracteres")
26
27     ranges: list[tuple[int, int]] = []
28     for part in spec.split(","):
29         low_text, sep, high_text = part.partition("-")
30         if not low_text.isdigit() or (sep and not high_text.isdigit()):
31             raise ValueError(f"trecho inválido: {part!r}")
32         low = int(low_text)
33         high = int(high_text) if sep else low
34         if not (100 <= low <= 599 and 100 <= high <= 599):
35             raise ValueError(f"status fora de 100–599: {part!r}")
36         if low > high:
37             raise ValueError(f"faixa invertida: {part!r}")
38         ranges.append((low, high))
39     return AcceptedStatusCodes(tuple(sorted(ranges)))
```

- Linhas 8–10: `@dataclass(frozen=True)` gera `__init__`, `__eq__` e `__repr__` e impede alterar os campos depois de criado. `tuple[tuple[int, int], ...]` é "uma tupla de qualquer tamanho cujos itens são pares de inteiros".
- Linhas 12–13: `__contains__` é o método que o Python chama em `x in objeto`. Confere que é inteiro e que cai em alguma faixa. `any(...)` para no primeiro verdadeiro.
- Linhas 15–16: `__str__` refaz o texto (`200-299,304`), para exibir.
- Linha 21: remove espaços, então `"200 - 299"` é aceito.
- Linhas 22–25: vazio ou longo demais é recusado.
- Linhas 28–38: para cada trecho separado por vírgula, `partition("-")` divide em três partes: antes do hífen, o hífen (ou vazio) e depois. Linha 30: os dois lados precisam ser só dígitos (isso recusa números negativos e letras). Linhas 32–33: sem hífen, a faixa é de um código só. Linhas 34–37: códigos HTTP vão de 100 a 599, e a faixa não pode estar invertida.
- Linha 39: ordena as faixas, para o texto exibido ficar previsível.

Falhas possíveis. Entrada inválida levanta `ValueError` com mensagem própria para a operadora (é mostrada no formulário).

`src/pulseops/domain/monitor_rules.py`

Finalidade. Limites de cada campo do monitor e as validações que dependem de mais de um campo.

Dados. Recebe um `MonitorConfig` (vindo do JSON da API ou do formulário) e devolve uma lista de `FieldError` (campo + mensagem). **Relações.** Chamado por `services/monitors.py`; o `MonitorConfig` é montado por `web/schemas.py` e `web/pages.py`. Os mesmos limites estão repetidos como constraints no banco (`db/models.py`), como segunda linha de defesa. **Testes.** `tests/unit/test_monitor_rules.py` (seis testes: padrões válidos, fora de faixa, timeout menor que intervalo, limite de latência, modo demo, todos os erros juntos).

`src/pulseops/domain/monitor_rules.py, linhas 7–37`

```
7 MAX_INTERVAL_SECONDS = 86_400
8 MIN_TIMEOUT_SECONDS, MAX_TIMEOUT_SECONDS = 1, 30
```

```python
MIN_LATENCY_MS = 50
MIN_THRESHOLD, MAX_THRESHOLD = 1, 10
MAX_NAME, MAX_DESCRIPTION, MAX_URL = 100, 500, 2048
METHODS = ("GET", "HEAD")

# Campos cuja mudança altera o significado dos contadores (RF03.5).
CHECK_FIELDS = (
    "url",
    "method",
    "accepted_status_codes",
    "timeout_seconds",
    "latency_threshold_ms",
    "failure_threshold",
    "recovery_threshold",
)


@dataclass(frozen=True)
class MonitorConfig:
    name: str
    url: str
    method: str = "GET"
    interval_seconds: int = 60
    timeout_seconds: int = 5
    accepted_status_codes: str = "200-299"
    latency_threshold_ms: int = 1000
    failure_threshold: int = 3
    recovery_threshold: int = 2
    description: str = ""
```

- Linhas 7–12: limites como constantes nomeadas. `86_400` é o mesmo que `86400` (o sublinhado só ajuda a ler). A linha 8 atribui duas constantes de uma vez.
- Linhas 15–23: campos que, se mudarem, mudam o **significado** dos contadores. Mudar o nome ou a descrição não zera nada; mudar a URL ou o timeout sim. Usado por `update_monitor` em `services/monitors.py`.
- Linhas 26–37: a configuração com os valores padrão do projeto (intervalo 60 s, timeout 5 s, 3 falhas para abrir, 2 sucessos para fechar).

`src/pulseops/domain/monitor_rules.py, linhas 46–88`

```python
def validate_config(config: MonitorConfig, *, min_interval_seconds: int) -> list[FieldError]:
    """Devolve todos os erros de uma vez, para o formulário mostrar tudo junto."""
    errors: list[FieldError] = []

    def check_range(field: str, value: int, low: int, high: int) -> None:
        if not low <= value <= high:
            errors.append(FieldError(field, f"deve estar entre {low} e {high}"))

    name = config.name.strip()
    if not name or len(name) > MAX_NAME:
        errors.append(FieldError("name", f"obrigatório, até {MAX_NAME} caracteres"))
    if len(config.description) > MAX_DESCRIPTION:
        errors.append(FieldError("description", f"até {MAX_DESCRIPTION} caracteres"))
    if len(config.url) > MAX_URL:
        errors.append(FieldError("url", f"até {MAX_URL} caracteres"))
    if config.method not in METHODS:
        errors.append(FieldError("method", "use GET ou HEAD"))

    check_range(
        "interval_seconds", config.interval_seconds, min_interval_seconds, MAX_INTERVAL_SECONDS
    )
    check_range("timeout_seconds", config.timeout_seconds, MIN_TIMEOUT_SECONDS, MAX_TIMEOUT_SECONDS)
    check_range("failure_threshold", config.failure_threshold, MIN_THRESHOLD, MAX_THRESHOLD)
    check_range("recovery_threshold", config.recovery_threshold, MIN_THRESHOLD, MAX_THRESHOLD)

    if config.timeout_seconds >= config.interval_seconds:
        errors.append(FieldError("timeout_seconds", "deve ser menor que o intervalo"))
```

```
73
74      max_latency = config.timeout_seconds * 1000
75      if not MIN_LATENCY_MS <= config.latency_threshold_ms <= max_latency:
76          errors.append(
77              FieldError(
78                  "latency_threshold_ms",
79                  f"deve estar entre {MIN_LATENCY_MS} e {max_latency} (timeout em ms)",
80              )
81          )
82
83      try:
84          parse_status_codes(config.accepted_status_codes)
85      except ValueError as exc:
86          errors.append(FieldError("accepted_status_codes", str(exc)))
87
88      return errors
```

- Linha 46: o `*` obriga `min_interval_seconds` a ser passado pelo nome. Ele vem da configuração (30 s, ou 2 s no modo demo).
- Linha 47: a docstring diz a intenção: devolver **todos** os erros, não parar no primeiro. O formulário mostra tudo de uma vez.
- Linhas 50–52: uma função dentro da função (*closure*). Ela enxerga a lista `errors` de fora e acrescenta nela.
- Linhas 54–62: validações de um campo só.
- Linhas 64–69: faixas numéricas, todas com a mesma função.
- Linhas 71–72: validação entre dois campos: o timeout precisa ser menor que o intervalo.
- Linhas 74–81: o limite de lentidão depende do timeout: entre 50 ms e o timeout em milissegundos.
- Linhas 83–86: reaproveita `parse_status_codes` e transforma a exceção em erro de campo.

Falhas possíveis. Nenhuma exceção sai daqui: tudo vira item da lista.

`src/pulseops/domain/health.py`

Finalidade. A regra central: dado o estado do monitor e o resultado de um check, calcula o novo estado e o que fazer com o incidente (abrir, atualizar, resolver ou nada), com o motivo em texto.

Dados. Entrada: `TrackerState` (lido das colunas do monitor e da existência de incidente aberto) e `Observation` (resultado do check). Saída: `Decision`. Nenhum efeito colateral. **Relações.** Chamado só por `_apply_domain` em `services/processing.py`; `Health` também é usado em `services/monitors.py` e nos templates. **Testes.** `tests/unit/test_health.py` (17 testes, incluindo os quatro do exemplo obrigatório) e, com banco, `tests/integration/test_incident_lifecycle.py`.

`src/pulseops/domain/health.py, linhas 15–38`

```
15 class Health(StrEnum):
16      NO_DATA = "no_data"
17      HEALTHY = "healthy"
18      DEGRADED = "degraded"
19      DOWN = "down"
20      PAUSED = "paused"
21
22      @property
23      def label(self) -> str:
24          return {
25              Health.NO_DATA: "Sem dados",
26              Health.HEALTHY: "Saudável",
27              Health.DEGRADED: "Degradado",
```

```
            Health.DOWN: "Indisponível",
            Health.PAUSED: "Pausado",
        }[self]


class IncidentAction(StrEnum):
    NONE = "none"
    OPEN = "open"
    UPDATE = "update"
    RESOLVE = "resolve"
```

- Linhas 15–30: os cinco estados de saúde de um monitor. `label` monta um dicionário e indexa pelo próprio membro.
- Linhas 33–37: as quatro ações possíveis sobre o incidente. A regra **não** abre incidente: ela devolve `OPEN`, e quem chama (`processing.py`) faz a gravação. Separar decisão de efeito é o que deixa a regra testável sem banco.

`src/pulseops/domain/health.py, linhas 40–73`

```
@dataclass(frozen=True)
class Thresholds:
    failure_threshold: int
    recovery_threshold: int


@dataclass(frozen=True)
class TrackerState:
    """Parte do monitor que a regra lê e altera. Fica persistida no banco."""

    health: Health
    consecutive_failures: int
    consecutive_successes: int
    first_failure_at: datetime | None
    incident_open: bool


@dataclass(frozen=True)
class Observation:
    outcome: Outcome
    observed_at: datetime
    explanation: str


@dataclass(frozen=True)
class Decision:
    state: TrackerState
    incident_action: IncidentAction
    previous_health: Health
    reason: str

    @property
    def health_changed(self) -> bool:
        return self.state.health != self.previous_health
```

- Linhas 40–43: os dois limiares, agrupados num objeto.
- Linhas 46–54: o estado que a regra lê e altera. `first_failure_at` guarda o horário da primeira falha da sequência (o "início" do incidente). `incident_open` diz se já há incidente aberto.
- Linhas 57–61: o que se observou num check.
- Linhas 64–73: o resultado da regra. `previous_health` permite saber se a saúde mudou (`health_changed`, linhas 71–73), o que decide se uma linha nova vai para `health_changes`.

Todos são `frozen=True`: imutáveis. A regra nunca altera o estado recebido; ela cria um novo com `replace(...)`, que copia o objeto trocando só os campos indicados.

src/pulseops/domain/health.py, linhas 76–99

```python
class HealthTracker:
    """Aplica um resultado de check ao estado do monitor.

    Regras (docs/requisitos.md, RF03):
    - falha soma em `consecutive_failures` e zera os sucessos; sucesso faz o contrário;
    - erro interno não muda nada;
    - o incidente abre quando as falhas atingem `failure_threshold` e fecha quando os
      sucessos atingem `recovery_threshold`.
    """

    def __init__(self, thresholds: Thresholds) -> None:
        self._thresholds = thresholds

    def apply(self, state: TrackerState, obs: Observation) -> Decision:
        if obs.outcome is Outcome.INTERNAL_ERROR:
            return Decision(
                state,
                IncidentAction.NONE,
                state.health,
                f"Erro interno do PulseOps ignorado na contagem: {obs.explanation}",
            )
        if obs.outcome.is_failure:
            return self._on_failure(state, obs)
        return self._on_success(state, obs)
```

- Linhas 86–87: o rastreador guarda só os limiares.
- Linhas 89–96: erro interno sai cedo. Devolve o **mesmo** estado, ação `NONE` e um motivo. Nada soma, nada zera (RF03.8).
- Linhas 97–99: falha vai para `_on_failure`; o resto (saudável ou lento) vai para `_on_success`.

src/pulseops/domain/health.py, linhas 101–128

```python
    def _on_failure(self, state: TrackerState, obs: Observation) -> Decision:
        failures = state.consecutive_failures + 1
        first_failure_at = state.first_failure_at if state.consecutive_failures else obs.observed_at
        new = replace(
            state,
            consecutive_failures=failures,
            consecutive_successes=0,
            first_failure_at=first_failure_at,
        )
        limit = self._thresholds.failure_threshold

        if state.incident_open:
            reason = f"Incidente continua aberto; último resultado: {obs.explanation}."
            new = replace(new, health=Health.DOWN)
            return Decision(new, IncidentAction.UPDATE, state.health, reason)

        if failures >= limit:
            reason = (
                f"Incidente confirmado após {failures} "
                f"{_plural(failures, 'falha consecutiva', 'falhas consecutivas')}; "
                f"último resultado: {obs.explanation}."
            )
            new = replace(new, health=Health.DOWN, incident_open=True)
            return Decision(new, IncidentAction.OPEN, state.health, reason)

        reason = f"Falha {failures} de {limit} necessárias para abrir incidente: {obs.explanation}."
        new = replace(new, health=Health.DEGRADED)
        return Decision(new, IncidentAction.NONE, state.health, reason)
```

- Linha 102: soma uma falha.
- Linha 103: se é a **primeira** falha da sequência (o contador anterior era 0), o início é o horário desta observação; senão, mantém o início anterior. Uma expressão `a if condição else b`.

- Linhas 104–109: novo estado: falhas somadas, sucessos zerados.
- Linhas 112–115: já existe incidente aberto: ação `UPDATE`, saúde “indisponível”. O mesmo incidente recebe a nova falha; nunca um segundo.
- Linhas 117–124: atingiu o limiar: ação `OPEN`, saúde “indisponível”. O motivo monta “Incidente confirmado após 3 falhas consecutivas; último resultado: …”. `_plural` (linhas 169–170) escolhe singular ou plural.
- Linhas 126–128: abaixo do limiar: “degradado”, com o progresso (“Falha 2 de 3 necessárias para abrir incidente”).

`src/pulseops/domain/health.py, linhas 130–166`

```
130     def _on_success(self, state: TrackerState, obs: Observation) -> Decision:
131         successes = state.consecutive_successes + 1
132         new = replace(
133             state,
134             consecutive_failures=0,
135             consecutive_successes=successes,
136             first_failure_at=None,
137         )
138         limit = self._thresholds.recovery_threshold
139 
140         if state.incident_open:
141             if successes >= limit:
142                 health = Health.DEGRADED if obs.outcome is Outcome.SLOW else Health.HEALTHY
143                 reason = (
144                     f"Recuperado após {successes} "
145                     f"{_plural(successes, 'sucesso consecutivo', 'sucessos consecutivos')}; "
146                     f"último resultado: {obs.explanation}."
147                 )
148                 new = replace(new, health=health, incident_open=False)
149                 return Decision(new, IncidentAction.RESOLVE, state.health, reason)
150             reason = (
151                 f"Sucesso {successes} de {limit} necessários para encerrar o incidente: "
152                 f"{obs.explanation}."
153             )
154             return Decision(
155                 replace(new, health=Health.DOWN), IncidentAction.NONE, state.health, reason
156             )
157 
158         if obs.outcome is Outcome.SLOW:
159             reason = f"Resposta lenta: {obs.explanation}."
160             return Decision(
161                 replace(new, health=Health.DEGRADED), IncidentAction.NONE, state.health, reason
162             )
163         reason = f"Respondendo normalmente: {obs.explanation}."
164         return Decision(
165             replace(new, health=Health.HEALTHY), IncidentAction.NONE, state.health, reason
166         )
```

- Linhas 131–137: soma um sucesso, zera as falhas e esquece o início da sequência.
- Linhas 140–156: há incidente aberto.
  - Linhas 141–149: se os sucessos atingiram o limiar de recuperação, ação `RESOLVE`. A saúde final é “degradado” se o último sucesso foi lento (linha 142), e “saudável” se foi rápido. Recuperar não apaga a lentidão.
  - Linhas 150–156: ainda não atingiu: continua “indisponível”, com o progresso (“Sucesso 1 de 2…”).
- Linhas 158–162: sem incidente, resposta lenta: “degradado”, sem ação.
- Linhas 163–166: sem incidente, resposta rápida: “saudável”.

Falhas possíveis. Nenhuma exceção é levantada aqui. O risco é de **regra**: como o lento conta como sucesso, um serviço que alterna lento e timeout nunca abre incidente com limiar 2 (volume 1, gabarito J10). É uma consequência documentada da ADR 0004.

`src/pulseops/domain/metrics.py`

Finalidade. Fórmulas das métricas: percentil, proporção e duração de incidente recortada à janela.

Dados. Listas de números (latências em ms), contagens inteiras e datas com fuso. Saídas `float`, `None` ou `timedelta`. **Relações.** `ratio` e `clipped_duration` são usados em `services/queries.py`; `nearest_rank` é a referência que o teste compara com o SQL. **Testes.** `tests/unit/test_metrics_formulas.py` (oito testes) e `test_sql_percentile_matches_nearest_rank`.

src/pulseops/domain/metrics.py

```python
"""Fórmulas das métricas (RF05). As consultas ficam em services/queries.py."""

import math
from collections.abc import Sequence
from datetime import datetime, timedelta


def nearest_rank(values: Sequence[float], percentile: float) -> float | None:
    """Percentil pelo método nearest-rank: valor na posição ceil(p/100 × n), 1-indexada.

    Devolve None quando não há amostras, para a interface mostrar "N/A".
    """
    if not 0 < percentile <= 100:
        raise ValueError("percentil deve estar em (0, 100]")
    if not values:
        return None
    ordered = sorted(values)
    rank = math.ceil(percentile / 100 * len(ordered))
    return ordered[rank - 1]


def ratio(part: int, total: int) -> float | None:
    """Proporção em porcentagem, ou None quando não há base (nunca inventa 0% ou 100%)."""
    if total <= 0:
        return None
    return 100.0 * part / total


def clipped_duration(
    window_start: datetime,
    window_end: datetime,
    started_at: datetime,
    ended_at: datetime | None,
    now: datetime,
) -> timedelta:
    """Parte de um incidente que cai dentro da janela [início, fim).

    Incidente ainda aberto conta até `now` (ou até o fim da janela, o que vier antes).
    """
    end = min(window_end, ended_at or now)
    start = max(window_start, started_at)
    return max(end - start, timedelta(0))
```

- Linhas 8–19: percentil *nearest-rank*. Linhas 13–14 recusam percentil fora de (0, 100]. Linhas 15–16: sem amostras, `None` (vira "N/A"). Linhas 17–19: ordena e pega a posição `ceil(p/100 × n)`; como listas do Python começam em 0, usa `rank - 1`.

- Linhas 22–26: proporção em porcentagem; sem base, `None`. A docstring é a regra do projeto: nunca inventar 0% ou 100%.
- Linhas 29–42: quanto de um incidente cai dentro da janela `[início, fim)`. O fim efetivo é o menor entre o fim da janela e o fim do incidente (ou "agora", se ainda aberto, linha 40). O começo efetivo é o maior entre os dois começos (linha 41). Se o incidente está todo fora, a diferença seria negativa, e `max(..., timedelta(0))` devolve zero (linha 42).

Falhas possíveis. `nearest_rank` com percentil inválido levanta `ValueError`; no código de produção ele só é chamado pelos testes (o painel usa o `percentile_disc` do banco).

D3 — Persistência: modelos, sessão e migrações

A parte F do volume 1 explica os conceitos (chaves, constraints, índices, transações) e mostra o SQL gerado. Aqui o foco é cada linha dos arquivos.

`src/pulseops/db/models.py`

Finalidade. Declara as 13 tabelas como classes do SQLAlchemy, com tipos, chaves, constraints e índices.

Dados. Cada classe corresponde a uma tabela; cada atributo `Mapped[...]` a uma coluna. Datas são sempre `timestamptz` (com fuso). **Relações.** Importado por todos os serviços, pelo worker, pelos testes e por `migrations/env.py` (que usa `Base.metadata` para comparar com o banco). **Testes.** `test_upgrade_on_empty_database_creates_all_tables` (modelos = migração), `test_database_rejects_second_running_check`, `test_database_rejects_second_open_incident`, `test_duplicate_name_is_rejected_case_insensitive`.

src/pulseops/db/models.py, linhas 1–42

```python
"""Tabelas do PulseOps (SQLAlchemy 2.x, estilo `Mapped[...]`).

Restrições que protegem regras de negócio ficam aqui e na migração, para que o
banco garanta o que a aplicação promete (por exemplo, um incidente aberto por
monitor), mesmo que exista um bug no código Python.
"""

from datetime import datetime
from typing import Any

from sqlalchemy import (
    BigInteger,
    Boolean,
    CheckConstraint,
    DateTime,
    ForeignKey,
    Index,
    Integer,
    MetaData,
    String,
    Text,
    UniqueConstraint,
    func,
    text,
)
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

# Nomes previsíveis para constraints: facilitam migrações e mensagens de erro.
NAMING = {
    "ix": "ix_%(column_0_label)s",
    "uq": "uq_%(table_name)s_%(column_0_name)s",
    "ck": "ck_%(table_name)s_%(constraint_name)s",
    "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",
    "pk": "pk_%(table_name)s",
}

Timestamp = DateTime(timezone=True)


class Base(DeclarativeBase):
    metadata = MetaData(naming_convention=NAMING)
```

- Linhas 1–6: a docstring diz o porquê do arquivo: o banco garante o que a aplicação promete, mesmo com bug no Python.
- Linhas 11–27: imports. `JSONB` (linha 26) é um tipo específico do PostgreSQL; `DeclarativeBase`, `Mapped` e `mapped_column` são o estilo tipado do SQLAlchemy 2.x.
- Linhas 30–36: **convenção de nomes**. Sem ela, o PostgreSQL inventaria nomes como `checks_monitor_id_fkey`. Com ela, toda constraint tem nome previsível (`ck_monitors_state`, `fk_checks_monitor_id_monitors`), o que facilita migrações futuras e mensagens de erro.
- Linha 38: um apelido para `DateTime(timezone=True)`, usado em todas as colunas de data.
- Linhas 41–42: a classe base de todos os modelos, com a convenção aplicada.

src/pulseops/db/models.py, linhas 45–62

```
45 class Operator(Base):
46     __tablename__ = "operators"
47
48     id: Mapped[int] = mapped_column(primary_key=True)
49     username: Mapped[str] = mapped_column(String(50), unique=True)
50     password_hash: Mapped[str] = mapped_column(Text)
51     created_at: Mapped[datetime] = mapped_column(Timestamp)
52
53
54 class OperatorSession(Base):
55     __tablename__ = "operator_sessions"
56
57     # SHA-256 do token do cookie, em hexadecimal. O token em si nunca é gravado.
58     token_hash: Mapped[str] = mapped_column(String(64), primary_key=True)
59     operator_id: Mapped[int] = mapped_column(ForeignKey("operators.id", ondelete="CASCADE"))
60     csrf_token: Mapped[str] = mapped_column(String(64))
61     created_at: Mapped[datetime] = mapped_column(Timestamp)
62     expires_at: Mapped[datetime] = mapped_column(Timestamp, index=True)
```

- `Operator` (linhas 45–51): `id` inteiro gerado pelo banco (`primary_key=True`); `username` único com até 50 caracteres; `password_hash` em `Text` (o hash Argon2 tem cerca de 100 caracteres).
- `OperatorSession` (linhas 54–62): a chave primária é o **hash** do token (linha 58, 64 caracteres hexadecimais do SHA-256). `ondelete="CASCADE"` (linha 59. apaga as sessões junto com a operadora. `expires_at` tem índice (linha 62) porque o login apaga as sessões vencidas.

Os padrões dessas duas classes se repetem em todas: `Mapped[tipo]` define o tipo Python e se aceita nulo (`| None`); `mapped_column(...)` define o tipo SQL e extras. Daqui em diante só o que é novo é comentado.

src/pulseops/db/models.py, linhas 65–100

```
65 class Monitor(Base):
66     __tablename__ = "monitors"
67     __table_args__ = (
68         CheckConstraint("method IN ('GET', 'HEAD')", name="method"),
69         CheckConstraint("state IN ('active', 'paused', 'archived')", name="state"),
70         CheckConstraint(
71             "health IN ('no_data', 'healthy', 'degraded', 'down', 'paused')", name="health"
72         ),
73         # Limites absolutos. O mínimo de 30 s do modo normal é validado na aplicação,
74         # porque o modo demonstração permite 2 s.
75         CheckConstraint("interval_seconds BETWEEN 2 AND 86400", name="interval"),
76         CheckConstraint("timeout_seconds BETWEEN 1 AND 30", name="timeout"),
77         CheckConstraint("timeout_seconds < interval_seconds", name="timeout_lt_interval"),
78         CheckConstraint(
79             "latency_threshold_ms BETWEEN 50 AND timeout_seconds * 1000", name="latency"
80         ),
81         CheckConstraint("failure_threshold BETWEEN 1 AND 10", name="failure_threshold"),
```

```
        CheckConstraint("recovery_threshold BETWEEN 1 AND 10", name="recovery_threshold"),
        CheckConstraint(
            "consecutive_failures >= 0 AND consecutive_successes >= 0", name="counters"
        ),
        CheckConstraint("(state = 'archived') = (archived_at IS NOT NULL)", name="archived_at"),
        # Nome único entre monitores não arquivados, sem diferenciar maiúsculas (RF01.7).
        Index(
            "uq_monitors_active_name",
            func.lower(text("name")),
            unique=True,
            postgresql_where=text("state <> 'archived'"),
        ),
        # Consulta do agendador: monitores ativos com next_run_at vencido.
        Index(
            "ix_monitors_due",
            "next_run_at",
            postgresql_where=text("state = 'active'"),
        ),
    )
```

As constraints de `monitors`. Volume 1, parte F, explica as linhas 68–86 uma a uma e os dois índices:

- Linhas 88–93: índice **único parcial** sobre `lower(name)`, só para monitores não arquivados. `func.lower(text("name"))` gera a expressão SQL `lower(name)`.
- Linhas 95–99: índice parcial para o agendador.

`src/pulseops/db/models.py, linhas 102–129`

```
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
    description: Mapped[str] = mapped_column(String(500), default="")
    url: Mapped[str] = mapped_column(String(2048))
    method: Mapped[str] = mapped_column(String(4), default="GET")
    interval_seconds: Mapped[int] = mapped_column(Integer)
    timeout_seconds: Mapped[int] = mapped_column(Integer)
    accepted_status_codes: Mapped[str] = mapped_column(String(100))
    latency_threshold_ms: Mapped[int] = mapped_column(Integer)
    failure_threshold: Mapped[int] = mapped_column(Integer)
    recovery_threshold: Mapped[int] = mapped_column(Integer)

    state: Mapped[str] = mapped_column(String(10), default="active")
    health: Mapped[str] = mapped_column(String(10), default="no_data")
    health_reason: Mapped[str] = mapped_column(Text, default="Ainda sem verificações.")
    health_changed_at: Mapped[datetime] = mapped_column(Timestamp)
    consecutive_failures: Mapped[int] = mapped_column(Integer, default=0)
    consecutive_successes: Mapped[int] = mapped_column(Integer, default=0)
    first_failure_at: Mapped[datetime | None] = mapped_column(Timestamp)
    config_version: Mapped[int] = mapped_column(Integer, default=1)

    next_run_at: Mapped[datetime | None] = mapped_column(Timestamp)
    # "Verificar agora" pedido pelo operador; o próximo check é marcado como manual (CB5).
    check_requested: Mapped[bool] = mapped_column(Boolean, default=False)
    last_check_at: Mapped[datetime | None] = mapped_column(Timestamp)
    created_at: Mapped[datetime] = mapped_column(Timestamp)
    updated_at: Mapped[datetime] = mapped_column(Timestamp)
    archived_at: Mapped[datetime | None] = mapped_column(Timestamp)
```

As colunas de `monitors`, em três grupos:

- Linhas 102–112, **configuração**: o que a operadora define. `accepted_status_codes` é texto (`"200-299"`), interpretado por `domain/status_codes.py`.
- Linhas 114–121, **estado calculado**: `state` (ciclo de vida: ativo, pausado, arquivado), `health` (saúde calculada), `health_reason` (o porquê, em texto), contadores, início da sequência de falhas e `config_version` (versão da contagem; bug B6).

- Linhas 123–129, **agendamento e auditoria**: `next_run_at` (quando verificar; nulo se pausado ou arquivado), `check_requested` ("verificar agora"), `last_check_at`, datas de criação, alteração e arquivamento.

`default=...` é o valor que o SQLAlchemy põe se o código não informar. O código de `services/monitors.py` informa tudo explicitamente, então os padrões são uma rede de segurança.

`src/pulseops/db/models.py, linhas 132–173`

```
132 class Check(Base):
133     __tablename__ = "checks"
134     __table_args__ = (
135         CheckConstraint("status IN ('running', 'completed')", name="status"),
136         CheckConstraint("trigger IN ('scheduled', 'manual')", name="trigger"),
137         CheckConstraint(
138             "outcome IN ('healthy', 'slow', 'unexpected_status', 'timeout', "
139             "'connection_error', 'tls_error', 'blocked', 'internal_error')",
140             name="outcome",
141         ),
142         CheckConstraint(
143             "(status = 'running' AND outcome IS NULL AND finished_at IS NULL) OR "
144             "(status = 'completed' AND outcome IS NOT NULL AND finished_at IS NOT NULL)",
145             name="completed_has_outcome",
146         ),
147         # No máximo um check em andamento por monitor: o banco impede sobreposição (CB3).
148         Index(
149             "uq_checks_one_running_per_monitor",
150             "monitor_id",
151             unique=True,
152             postgresql_where=text("status = 'running'"),
153         ),
154         # Histórico de um monitor, mais recente primeiro, com desempate por id (RF04.1).
155         Index("ix_checks_monitor_history", "monitor_id", "started_at", "id"),
156         # Retenção: apagar checks antigos de todos os monitores (CB6).
157         Index("ix_checks_started_at", "started_at"),
158     )
159
160     id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
161     monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
162     # Identidade da execução agendada: "{monitor_id}:{scheduled_at ISO UTC}".
163     run_key: Mapped[str] = mapped_column(String(80), unique=True)
164     trigger: Mapped[str] = mapped_column(String(10), default="scheduled")
165     status: Mapped[str] = mapped_column(String(10), default="running")
166     config_version: Mapped[int] = mapped_column(Integer)
167     scheduled_at: Mapped[datetime] = mapped_column(Timestamp)
168     started_at: Mapped[datetime] = mapped_column(Timestamp)
169     finished_at: Mapped[datetime | None] = mapped_column(Timestamp)
170     duration_ms: Mapped[int | None] = mapped_column(Integer)
171     status_code: Mapped[int | None] = mapped_column(Integer)
172     outcome: Mapped[str | None] = mapped_column(String(20))
173     explanation: Mapped[str | None] = mapped_column(String(300))
```

`checks`, um registro por verificação:

- Linhas 143–146: a constraint mais interessante: um check `running` **não tem** resultado nem fim; um `completed` **tem** os dois. O banco não aceita um estado intermediário.
- Linhas 148–157: os três índices (volume 1, parte F).
- Linha 160: `BigInteger`, porque esta tabela cresce sem parar.
- Linha 161: `RESTRICT`: um monitor com checks não pode ser apagado.
- Linhas 162–163: `run_key` único (identidade da execução).
- Linhas 166–167: versão da contagem na reserva e o horário **agendado** (diferente do `started_at`, o horário real).

- Linhas 169–173: preenchidos na conclusão. `explanation` tem no máximo 300 caracteres; `services/processing.py` corta o texto antes de gravar.

src/pulseops/db/models.py, linhas 176–215

```python
class Incident(Base):
    __tablename__ = "incidents"
    __table_args__ = (
        CheckConstraint("status IN ('open', 'resolved', 'closed')", name="status"),
        CheckConstraint("(status = 'open') = (ended_at IS NULL)", name="ended_when_not_open"),
        CheckConstraint("started_at <= confirmed_at", name="started_before_confirmed"),
        # Um incidente aberto por monitor, garantido pelo banco (RF03.11).
        Index(
            "uq_incidents_one_open_per_monitor",
            "monitor_id",
            unique=True,
            postgresql_where=text("status = 'open'"),
        ),
        # Lista de incidentes (geral e por monitor), mais recentes primeiro.
        Index("ix_incidents_confirmed", "confirmed_at", "id"),
        Index("ix_incidents_monitor", "monitor_id", "confirmed_at"),
        # A retenção apaga checks; o banco precisa achar quem aponta para eles (SET NULL).
        Index("ix_incidents_last_check", "last_check_id"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
    status: Mapped[str] = mapped_column(String(10), default="open")
    started_at: Mapped[datetime] = mapped_column(Timestamp)
    confirmed_at: Mapped[datetime] = mapped_column(Timestamp)
    ended_at: Mapped[datetime | None] = mapped_column(Timestamp)
    # Retrato da configuração no momento da abertura (RF03.12).
    failure_threshold: Mapped[int] = mapped_column(Integer)
    recovery_threshold: Mapped[int] = mapped_column(Integer)
    timeout_seconds: Mapped[int] = mapped_column(Integer)
    latency_threshold_ms: Mapped[int] = mapped_column(Integer)
    opening_reason: Mapped[str] = mapped_column(Text)
    resolution_reason: Mapped[str | None] = mapped_column(String(30))
    resolution_explanation: Mapped[str | None] = mapped_column(Text)
    failure_count: Mapped[int] = mapped_column(Integer)
    last_outcome: Mapped[str] = mapped_column(String(20))
    last_explanation: Mapped[str] = mapped_column(String(300))
    last_check_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("checks.id", ondelete="SET NULL")
    )
```

`incidents`:

- Linha 180: `(status = 'open') = (ended_at IS NULL)`: aberto se e somente se não tem fim.
- Linha 181: o início não pode ser depois da confirmação.
- Linhas 183–188: um incidente aberto por monitor (RF03.11).
- Linhas 202–206: **retrato** da configuração no momento da abertura (RF03.12). Se a operadora mudar o limiar depois, o incidente continua dizendo com qual limiar foi aberto.
- Linhas 207–212: motivos de abertura e de resolução, número de falhas, último resultado e última explicação.
- Linhas 213–215: o último check, com `SET NULL` (a retenção pode apagá-lo).

src/pulseops/db/models.py, linhas 218–272

```python
class IncidentEvent(Base):
    __tablename__ = "incident_events"
    __table_args__ = (
        CheckConstraint(
            "kind IN ('first_failure', 'opened', 'error_changed', 'config_changed', "
            "'resolved', 'closed')",
```

```
224             name="kind",
225         ),
226         Index("ix_incident_events_timeline", "incident_id", "occurred_at", "id"),
227         Index("ix_incident_events_check", "check_id"),
228     )
229
230     id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
231     incident_id: Mapped[int] = mapped_column(ForeignKey("incidents.id", ondelete="RESTRICT"))
232     kind: Mapped[str] = mapped_column(String(20))
233     occurred_at: Mapped[datetime] = mapped_column(Timestamp)
234     message: Mapped[str] = mapped_column(Text)
235     details: Mapped[dict[str, Any]] = mapped_column(JSONB, default=dict)
236     check_id: Mapped[int | None] = mapped_column(
237         BigInteger, ForeignKey("checks.id", ondelete="SET NULL")
238     )
239
240
241 class HealthChange(Base):
242     __tablename__ = "health_changes"
243     __table_args__ = (
244         Index("ix_health_changes_monitor", "monitor_id", "changed_at", "id"),
245         Index("ix_health_changes_check", "check_id"),
246     )
247
248     id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
249     monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
250     changed_at: Mapped[datetime] = mapped_column(Timestamp)
251     from_health: Mapped[str] = mapped_column(String(10))
252     to_health: Mapped[str] = mapped_column(String(10))
253     reason: Mapped[str] = mapped_column(Text)
254     check_id: Mapped[int | None] = mapped_column(
255         BigInteger, ForeignKey("checks.id", ondelete="SET NULL")
256     )
257
258
259 class CoverageGap(Base):
260     """Período em que checks previstos não foram executados (worker parado ou atrasado)."""
261
262     __tablename__ = "coverage_gaps"
263     __table_args__ = (
264         CheckConstraint("gap_start < gap_end", name="ordered"),
265         Index("ix_coverage_gaps_monitor", "monitor_id", "gap_start"),
266     )
267
268     id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
269     monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
270     gap_start: Mapped[datetime] = mapped_column(Timestamp)
271     gap_end: Mapped[datetime] = mapped_column(Timestamp)
272     reason: Mapped[str] = mapped_column(String(30))
```

Três tabelas de “fatos derivados”, no mesmo padrão (id `BigInteger`, chave para o pai, data, texto e, quando faz sentido, o check que causou):

- `incident_events` (218–238): a linha do tempo do incidente. `kind` tem seis valores possíveis (linha 222); `details` é `JSONB` com o retrato da configuração (linha 235). `default=dict` (sem parênteses) passa a **função** `dict`, que cria um dicionário novo a cada linha. Passar `{}` faria todas as linhas compartilharem o mesmo objeto.
- `health_changes` (241–256): de qual saúde para qual, quando e por quê.
- `coverage_gaps` (259–272): períodos sem checks, com início antes do fim (constraint da linha 264).

`src/pulseops/db/models.py, linhas 275–306`

```
275 class AlertDestination(Base):
276     __tablename__ = "alert_destinations"
277
```

```python
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100), unique=True)
    url: Mapped[str] = mapped_column(String(2048))
    enabled: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(Timestamp)
    updated_at: Mapped[datetime] = mapped_column(Timestamp)


class AlertEvent(Base):
    """Outbox: criado na mesma transação da mudança de incidente (RF06.1)."""

    __tablename__ = "alert_events"
    __table_args__ = (
        CheckConstraint(
            "kind IN ('incident.opened', 'incident.resolved', 'incident.closed')", name="kind"
        ),
        Index("ix_alert_events_created", "created_at", "id"),
        # Notificações de um incidente, na página do incidente.
        Index("ix_alert_events_incident", "incident_id"),
    )

    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    # Chave estável "incident:{id}:{tipo}". Também é o event_id enviado ao destino.
    event_key: Mapped[str] = mapped_column(String(80), unique=True)
    kind: Mapped[str] = mapped_column(String(30))
    incident_id: Mapped[int] = mapped_column(ForeignKey("incidents.id", ondelete="RESTRICT"))
    monitor_id: Mapped[int] = mapped_column(ForeignKey("monitors.id", ondelete="RESTRICT"))
    payload: Mapped[dict[str, Any]] = mapped_column(JSONB)
    created_at: Mapped[datetime] = mapped_column(Timestamp)
```

- `alert_destinations` (275–283): destinos de webhook, nome único, ativo ou não.
- `alert_events` (286–306): o evento do outbox. `event_key` único (linha 301) é o `event_id` enviado ao destino. `payload` é o JSON exato que será enviado, montado uma vez na criação.

`src/pulseops/db/models.py, linhas 309–364`

```python
class Delivery(Base):
    __tablename__ = "deliveries"
    __table_args__ = (
        UniqueConstraint("event_id", "destination_id", name="uq_deliveries_event_destination"),
        CheckConstraint("status IN ('pending', 'sending', 'delivered', 'failed')", name="status"),
        CheckConstraint("attempt_count >= 0", name="attempts"),
        # Consulta do entregador: entregas pendentes (ou com reserva vencida) por horário.
        Index(
            "ix_deliveries_due",
            "next_attempt_at",
            postgresql_where=text("status IN ('pending', 'sending')"),
        ),
    )

    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    event_id: Mapped[int] = mapped_column(ForeignKey("alert_events.id", ondelete="RESTRICT"))
    destination_id: Mapped[int] = mapped_column(
        ForeignKey("alert_destinations.id", ondelete="RESTRICT")
    )
    status: Mapped[str] = mapped_column(String(10), default="pending")
    attempt_count: Mapped[int] = mapped_column(Integer, default=0)
    # Para 'pending': quando tentar. Para 'sending': até quando vale a reserva.
    next_attempt_at: Mapped[datetime] = mapped_column(Timestamp)
    last_error: Mapped[str | None] = mapped_column(String(300))
    delivered_at: Mapped[datetime | None] = mapped_column(Timestamp)
    created_at: Mapped[datetime] = mapped_column(Timestamp)
    updated_at: Mapped[datetime] = mapped_column(Timestamp)


class DeliveryAttempt(Base):
    __tablename__ = "delivery_attempts"
    __table_args__ = (
        UniqueConstraint("delivery_id", "attempt_number", name="uq_delivery_attempts_number"),
```

```
        CheckConstraint(
            "result IN ('success', 'retryable_error', 'permanent_error')", name="result"
        ),
    )

    id: Mapped[int] = mapped_column(BigInteger, primary_key=True)
    delivery_id: Mapped[int] = mapped_column(ForeignKey("deliveries.id", ondelete="RESTRICT"))
    attempt_number: Mapped[int] = mapped_column(Integer)
    attempted_at: Mapped[datetime] = mapped_column(Timestamp)
    duration_ms: Mapped[int] = mapped_column(Integer)
    status_code: Mapped[int | None] = mapped_column(Integer)
    result: Mapped[str] = mapped_column(String(20))
    error: Mapped[str | None] = mapped_column(String(300))


class WorkerHeartbeat(Base):
    __tablename__ = "worker_heartbeats"

    worker_id: Mapped[str] = mapped_column(String(100), primary_key=True)
    hostname: Mapped[str] = mapped_column(String(255))
    pid: Mapped[int] = mapped_column(Integer)
    started_at: Mapped[datetime] = mapped_column(Timestamp)
    last_seen_at: Mapped[datetime] = mapped_column(Timestamp)
```

- `deliveries` (309–335): uma entrega por par (evento, destino), garantida pela `UniqueConstraint` da linha 312. A linha 330 explica o duplo uso de `next_attempt_at`: para `pending`, quando tentar; para `sending`, até quando vale a reserva.
- `delivery_attempts` (338–354): uma linha por tentativa, com número único por entrega (linha 341).
- `worker_heartbeats` (357–364): chave primária textual (`hostname:pid`) e o último sinal de vida.

Falhas possíveis. Uma gravação que viole qualquer constraint levanta `IntegrityError` e a transação inteira é desfeita. Na criação de monitor e de destino, esse erro vira uma mensagem para a operadora (nome repetido); nos outros casos, indica bug e aparece como erro 500 ou como log de erro do worker.

`src/pulseops/db/session.py`

Finalidade. Cria o *engine* (o gerenciador do pool de conexões) e a fábrica de sessões.

src/pulseops/db/session.py

```
"""Criação do engine assíncrono e da fábrica de sessões."""

from sqlalchemy.ext.asyncio import (
    AsyncEngine,
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

SessionFactory = async_sessionmaker[AsyncSession]


def create_engine(database_url: str, *, pool_size: int = 10) -> AsyncEngine:
    return create_async_engine(
        database_url,
        pool_size=pool_size,
        pool_pre_ping=True,  # descarta conexões mortas (por exemplo, após o banco reiniciar)
        connect_args={"connect_timeout": 5},
    )


def create_session_factory(engine: AsyncEngine) -> SessionFactory:
```

```
    # expire_on_commit=False: objetos continuam legíveis depois do commit, sem nova consulta.
    return async_sessionmaker(engine, expire_on_commit=False)
```

- Linhas 3–8: os tipos assíncronos do SQLAlchemy.
- Linha 10: `SessionFactory` é um apelido de tipo, usado nas assinaturas dos serviços.
- Linhas 13–19: `pool_pre_ping` testa a conexão antes de usar; `connect_timeout: 5` faz uma conexão com banco inacessível falhar em 5 s, em vez de esperar indefinidamente.
- Linhas 22–24: `expire_on_commit=False` (volume 1, parte F).

Dados. A URL do banco (texto da configuração). **Relações.** Usado por `web/app.py`, `worker/__main__.py`, `cli.py` e `tests/conftest.py`. **Falhas.** URL inválida levanta erro na criação; banco fora do ar só aparece na primeira consulta. **Testes.** Indiretamente, todos os de integração; `test_ready_returns_503_when_database_is_down` cobre o banco inacessível.

`alembic.ini`

Finalidade. Configuração do Alembic: onde estão as migrações e como logar.

alembic.ini, linhas 1–20

```
[alembic]
script_location = %(here)s/migrations
file_template = %%(rev)s_%%(slug)s
prepend_sys_path = .
path_separator = os
# A URL do banco vem de PULSEOPS_DATABASE_URL (ver migrations/env.py).

[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARNING
handlers = console
qualname =
```

- Linha 2: `%(here)s` é a pasta do próprio arquivo; as migrações ficam em `migrations/`.
- Linha 3: modelo do nome dos arquivos novos: `0001_esquema_inicial.py`.
- Linha 4: acrescenta a raiz ao caminho de import, para `migrations/env.py` conseguir importar `pulseops`.
- Linha 5: separa os caminhos da linha 4 conforme o sistema operacional (`:` no Linux, `;` no Windows); é a forma atual recomendada pelo Alembic.
- Linha 6: o comentário avisa que **não** há URL de banco aqui (ela seria um segredo num arquivo versionado); vem do ambiente.
- Linhas 8–20 e o resto do arquivo: configuração padrão de logging do Alembic (nível WARNING para o SQLAlchemy, INFO para o Alembic, saída em `stderr`).

`migrations/env.py`

Finalidade. Script que o Alembic executa a cada comando: descobre a URL, abre a conexão e roda as migrações.

migrations/env.py

```python
"""Ambiente do Alembic: usa a mesma URL e os mesmos modelos da aplicação.

As migrações rodam de forma síncrona com o driver psycopg 3, o mesmo usado pela
aplicação no modo assíncrono.
"""

from logging.config import fileConfig

from alembic import context
from sqlalchemy import create_engine, pool

from pulseops.config import Settings
from pulseops.db.models import Base

config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata


def database_url() -> str:
    # Permite que os testes passem uma URL própria via config.attributes.
    return str(config.attributes.get("database_url") or Settings().database_url)


def run_migrations_offline() -> None:
    context.configure(url=database_url(), target_metadata=target_metadata, literal_binds=True)
    with context.begin_transaction():
        context.run_migrations()


def run_migrations_online() -> None:
    engine = create_engine(database_url(), poolclass=pool.NullPool)
    with engine.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)
        with context.begin_transaction():
            context.run_migrations()
    engine.dispose()


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

- Linhas 13–15: usa as mesmas classes de configuração e de modelos da aplicação.
- Linhas 17–19: aplica o logging do `alembic.ini`.
- Linha 21: `target_metadata` é o que o `--autogenerate` compara com o banco.
- Linhas 24–26: a URL vem primeiro de `config.attributes` (os testes passam a URL do banco de teste assim; `tests/conftest.py`, `alembic_config`), senão de `PULSEOPS_DATABASE_URL`.
- Linhas 29–32: modo *offline*: gera o SQL como texto, sem conectar (`alembic upgrade head --sql`).
- Linhas 35–41: modo normal. Um engine **síncrono** com `NullPool` (sem pool, porque é um comando de uma vez), uma transação, e as migrações dentro dela. No PostgreSQL, até `CREATE TABLE` é transacional: se a migração falhar no meio, nada fica criado pela metade.

Testes. `tests/integration/test_migrations.py` e o passo de migrações do CI.

`migrations/script.py.mako`

Finalidade. Modelo (template Mako) usado pelo `alembic revision` para criar arquivos de migração novos.

migrations/script.py.mako

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}
"""

from collections.abc import Sequence

import sqlalchemy as sa
from alembic import op
${imports if imports else ""}

revision: str = ${repr(up_revision)}
down_revision: str | None = ${repr(down_revision)}
branch_labels: str | Sequence[str] | None = ${repr(branch_labels)}
depends_on: str | Sequence[str] | None = ${repr(depends_on)}


def upgrade() -> None:
    ${upgrades if upgrades else "pass"}


def downgrade() -> None:
    ${downgrades if downgrades else "pass"}
```

`${...}` são marcadores que o Alembic preenche: mensagem, identificador da revisão, revisão anterior, data e os comandos gerados. O modelo foi ajustado para usar anotações de tipo (linhas 14–17), porque o mypy está em modo estrito.

`migrations/versions/0001_esquema_inicial.py` (explicado por padrão)

Finalidade. Cria todas as tabelas, constraints e índices (`upgrade`) e remove tudo na ordem inversa (`downgrade`).

O arquivo foi **gerado** por `alembic revision --autogenerate` a partir dos modelos, e por isso não é explicado linha a linha (ver exclusões). Ele tem 468 linhas, mas usa só cinco padrões. Um exemplo de cada:

migrations/versions/0001_esquema_inicial.py, linhas 14–18

```
revision: str = "0001"
down_revision: str | None = None
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None
```

Identificação. `revision` é o id desta migração; `down_revision = None` diz que é a primeira. Uma próxima migração teria `down_revision = "0001"`.

migrations/versions/0001_esquema_inicial.py, linhas 125–158

```
125     op.create_table(
126         "checks",
127         sa.Column("id", sa.BigInteger(), nullable=False),
128         sa.Column("monitor_id", sa.Integer(), nullable=False),
129         sa.Column("run_key", sa.String(length=80), nullable=False),
130         sa.Column("trigger", sa.String(length=10), nullable=False),
131         sa.Column("status", sa.String(length=10), nullable=False),
132         sa.Column("config_version", sa.Integer(), nullable=False),
133         sa.Column("scheduled_at", sa.DateTime(timezone=True), nullable=False),
134         sa.Column("started_at", sa.DateTime(timezone=True), nullable=False),
135         sa.Column("finished_at", sa.DateTime(timezone=True), nullable=True),
136         sa.Column("duration_ms", sa.Integer(), nullable=True),
137         sa.Column("status_code", sa.Integer(), nullable=True),
138         sa.Column("outcome", sa.String(length=20), nullable=True),
139         sa.Column("explanation", sa.String(length=300), nullable=True),
140         sa.CheckConstraint(
141             "(status = 'running' AND outcome IS NULL AND finished_at IS NULL) OR (status = 'completed'
    AND outcome IS NOT NULL AND finished_at IS NOT NULL)",
142             name=op.f("ck_checks_completed_has_outcome"),
143         ),
144         sa.CheckConstraint(
145             "outcome IN ('healthy', 'slow', 'unexpected_status', 'timeout', 'connection_error',
    'tls_error', 'blocked', 'internal_error')",
146             name=op.f("ck_checks_outcome"),
147         ),
148         sa.CheckConstraint("status IN ('running', 'completed')", name=op.f("ck_checks_status")),
149         sa.CheckConstraint("trigger IN ('scheduled', 'manual')", name=op.f("ck_checks_trigger")),
150         sa.ForeignKeyConstraint(
151             ["monitor_id"],
152             ["monitors.id"],
153             name=op.f("fk_checks_monitor_id_monitors"),
154             ondelete="RESTRICT",
155         ),
156         sa.PrimaryKeyConstraint("id", name=op.f("pk_checks")),
157         sa.UniqueConstraint("run_key", name=op.f("uq_checks_run_key")),
158     )
```

Criar tabela (`op.create_table`). Cada `sa.Column` traz nome, tipo e se aceita nulo. Depois vêm as constraints com os nomes da convenção (`op.f(...)` marca o nome como "já final", para o Alembic não aplicar a convenção de novo). O mesmo padrão se repete nas 13 tabelas, nas linhas 21–31, 32–92, 107–115, 116–124, 125–158, 170–185, 189–203, 207–229, 237–278, 291–318, 321–347, 355–386 e 394–416.

migrations/versions/0001_esquema_inicial.py, linhas 159–169

```
159     op.create_index(
160         "ix_checks_monitor_history", "checks", ["monitor_id", "started_at", "id"], unique=False
161     )
162     op.create_index("ix_checks_started_at", "checks", ["started_at"], unique=False)
163     op.create_index(
164         "uq_checks_one_running_per_monitor",
165         "checks",
166         ["monitor_id"],
167         unique=True,
168         postgresql_where=sa.text("status = 'running'"),
169     )
```

Criar índice (`op.create_index`), comum ou **parcial** (`postgresql_where`). O índice sobre `lower(name)` (linhas 100–106) usa `sa.literal_column` porque é uma expressão, não uma coluna.

migrations/versions/0001_esquema_inicial.py, linhas 419–426

```
419 def downgrade() -> None:
420     op.drop_table("delivery_attempts")
421     op.drop_index(
422         "ix_deliveries_due",
```

```
        table_name="deliveries",
        postgresql_where=sa.text("status IN ('pending', 'sending')"),
    )
    op.drop_table("deliveries")
```

Desfazer (`downgrade`). Remove índices e tabelas na ordem inversa da criação: uma tabela só pode ser apagada depois das que apontam para ela.

Como se sabe que está certo. `test_upgrade_on_empty_database_creates_all_tables` aplica a migração e compara com os modelos: qualquer diferença (coluna faltando, índice diferente) faz o teste falhar. Foi assim que o bug B1 apareceu.

D4 — Segurança: política de destino, transporte protegido e senhas

`src/pulseops/security/url_policy.py`

Finalidade. Decide se uma URL e um IP podem ser usados como destino de check ou de webhook (proteção contra SSRF, ADR 0005).

Dados. Entrada: texto da URL (digitado pela operadora) e objetos `IPv4Address`/`IPv6Address` (vindos da resolução DNS). Saída: `Destination` ou `UnsafeDestinationError` com mensagem legível. **Relações.** A política é criada a partir de `Settings` em `web/app.py` e `worker/__main__.py`; usada por `security/transport.py`, `services/monitors.py`, `web/api.py`, `web/pages.py` e `notifications/webhook.py`. **Testes.** `tests/unit/test_url_policy.py` (seis funções e 38 casos, com listas parametrizadas de URLs aceitas e recusadas) e `tests/unit/test_transport.py`.

src/pulseops/security/url_policy.py, linhas 1–37

```
"""Política de destino contra SSRF (ver docs/modelo-de-ameacas.md e ADR 0005).

Duas camadas:
1. `validate_url` olha só o texto da URL (cadastro de monitor e de webhook).
2. `check_address` decide se um IP já resolvido pode ser usado. É chamada pelo
   transporte no momento da conexão, então vale também contra DNS rebinding.
"""

import ipaddress
import re
from dataclasses import dataclass, field
from urllib.parse import urlsplit

IPAddress = ipaddress.IPv4Address | ipaddress.IPv6Address

# Hosts formados só por números/hex com pontos: "2130706433", "0x7f.1", "0177.0.0.1".
# O resolvedor do sistema aceita essas formas; aqui só a forma canônica passa.
_NUMERIC_HOST = re.compile(r"^(0x[0-9a-f]*|[0-9]+)(\.(0x[0-9a-f]*|[0-9]+))*\.?$")
_NAT64_PREFIX = ipaddress.IPv6Network("64:ff9b::/96")


class UnsafeDestinationError(ValueError):
    """Destino recusado pela política. A mensagem é segura para mostrar ao operador."""


@dataclass(frozen=True)
class Destination:
    scheme: str
    host: str
    port: int


@dataclass(frozen=True)
class DestinationPolicy:
    allowed_ports: frozenset[int] = frozenset({80, 443, 8080, 8443})
    # Pares (host, porta) do laboratório. Só é preenchido em modo demonstração.
    demo_allowlist: frozenset[tuple[str, int]] = field(default_factory=frozenset)
```

- Linhas 1–7: a docstring resume as duas camadas.
- Linha 14: um apelido de tipo para “IPv4 ou IPv6”.

- Linhas 16–18: expressão regular que reconhece hosts escritos só com números ou hexadecimal separados por ponto. O resolvedor do sistema aceita formas como `2130706433` (127.0.0.1 em decimal) e `0x7f.1`; o PulseOps só aceita a forma canônica (`127.0.0.1`), que é verificada como IP.
- Linha 19: `64:ff9b::/96` é o prefixo NAT64, que embute um IPv4 dentro de um IPv6.
- Linhas 22–23: a exceção herda de `ValueError`. Quem já trata `ValueError` (validações) a captura sem código extra.
- Linhas 26–30: o destino validado.
- Linhas 33–37: a política, imutável. Portas permitidas por padrão e a lista de exceções do laboratório (vazia fora do modo demo). `field(default_factory=frozenset)` cria um conjunto vazio novo por instância.

`src/pulseops/security/url_policy.py, linhas 39–79`

```
39    def validate_url(self, url: str) -> Destination:
40        try:
41            parts = urlsplit(url.strip())
42            port = parts.port
43        except ValueError as exc:
44            raise UnsafeDestinationError("URL inválida") from exc
45
46        scheme = parts.scheme.lower()
47        if scheme not in ("http", "https"):
48            raise UnsafeDestinationError("use http:// ou https://")
49        if parts.username is not None or parts.password is not None:
50            raise UnsafeDestinationError("credenciais na URL não são permitidas")
51        host = (parts.hostname or "").lower()
52        if not host:
53            raise UnsafeDestinationError("URL sem host")
54        port = port or (443 if scheme == "https" else 80)
55
56        if self.is_demo_host(host, port):
57            return Destination(scheme, host, port)
58
59        if port not in self.allowed_ports:
60            allowed = ", ".join(str(p) for p in sorted(self.allowed_ports))
61            raise UnsafeDestinationError(f"porta {port} não permitida (permitidas: {allowed})")
62
63        literal = _parse_ip(host)
64        if literal is None and _NUMERIC_HOST.match(host):
65            raise UnsafeDestinationError("endereço IP em forma não canônica")
66        if literal is not None:
67            self.check_address(host, port, literal)
68        return Destination(scheme, host, port)
69
70    def is_demo_host(self, host: str, port: int) -> bool:
71        return (host.lower().rstrip("."), port) in self.demo_allowlist
72
73    def check_address(self, host: str, port: int, address: IPAddress) -> None:
74        """Levanta UnsafeDestinationError se o IP não puder ser usado para este host."""
75        if self.is_demo_host(host, port):
76            return
77        reason = blocked_reason(address)
78        if reason:
79            raise UnsafeDestinationError(f"{host} aponta para {reason} ({address})")
```

- Linhas 40–44: `urlsplit` divide a URL; ler `parts.port` pode levantar `ValueError` (porta não numérica ou acima de 65535), que vira “URL inválida”.
- Linhas 46–53: esquema, credenciais e host. `parts.hostname` já vem em minúsculas e sem colchetes de IPv6.
- Linha 54: porta padrão do esquema, se não informada.
- Linhas 56–57: a exceção do laboratório é **nominal**: vale para o par exato (`lab-target`, 8081) e nada mais.

- Linhas 59–61: porta fora da lista.
- Linhas 63–67: se o host é um IP escrito na forma canônica, confere o IP já no cadastro; se parece número mas não é IP canônico, recusa.
- Linhas 70–71: compara o host sem ponto final (`lab-target.` é o mesmo nome).
- Linhas 73–79: `check_address` é a função chamada **na hora da conexão** pelo transporte. A mensagem cita o host, o motivo e o IP.

`src/pulseops/security/url_policy.py, linhas 82–111`

```
 82 def blocked_reason(address: IPAddress) -> str | None:
 83     """Motivo do bloqueio, ou None se o endereço for público e unicast."""
 84     if isinstance(address, ipaddress.IPv6Address):
 85         embedded = address.ipv4_mapped or address.sixtofour
 86         if embedded is None and address in _NAT64_PREFIX:
 87             embedded = ipaddress.IPv4Address(int(address) & 0xFFFFFFFF)
 88         if embedded is not None:
 89             return blocked_reason(embedded)
 90         if address.teredo:
 91             return "endereço Teredo"
 92     if address.is_loopback:
 93         return "endereço de loopback"
 94     if address.is_link_local:
 95         return "endereço link-local (inclui metadados de nuvem)"
 96     if address.is_multicast:
 97         return "endereço multicast"
 98     if address.is_unspecified:
 99         return "endereço não especificado"
100     if address.is_private:
101         return "endereço privado"
102     if not address.is_global:
103         return "endereço reservado ou não roteável"
104     return None
105
106
107 def _parse_ip(host: str) -> IPAddress | None:
108     try:
109         return ipaddress.ip_address(host)
110     except ValueError:
111         return None
```

- Linhas 84–89: IPv6 que embute IPv4 (mapeado `::ffff:a.b.c.d`, 6to4 ou NAT64): extrai o IPv4 e confere **ele**, recursivamente. Sem isso, `::ffff:127.0.0.1` passaria por "IPv6 público".
- Linhas 90–91: Teredo é um túnel IPv6 sobre IPv4; recusado.
- Linhas 92–103: as categorias, na ordem do mais específico para o mais geral. A biblioteca `ipaddress` já sabe classificar (`is_loopback`, `is_private`…). A linha 95 lembra que `169.254.169.254`, o serviço de metadados de nuvem, é link-local. A linha 102 é a rede de segurança: tudo que não é "global" é recusado.
- Linhas 107–111: tenta interpretar o texto como IP; se não for, `None`.

Falhas possíveis. Uma URL que redireciona para a rede interna não é seguida (o cliente não segue redirecionamentos). Endereços que a biblioteca `ipaddress` classifique errado passariam; a lista de casos testados está em `tests/unit/test_url_policy.py`.

`src/pulseops/security/transport.py`

Finalidade. Faz o HTTPX conectar **só** em IPs aprovados, resolvendo e validando no momento da conexão (defesa contra DNS rebinding).

Dados. Recebe nome e porta do `httpcore`; devolve um *stream* de rede aberto no IP validado. **Relações.** `build_guarded_client` é chamado por `worker/__main__.py` (dois clientes: checks e webhooks); `validate_destination` é chamado por `services/monitors.py` e pelas rotas de destinos; `Resolver` é substituído por `FakeResolver` nos testes. **Testes.** `tests/unit/test_transport.py` (doze testes, entre eles `test_rebinding_is_checked_at_connect_time`, `test_mixed_public_and_private_answers_are_refused`, `test_clients_ignore_proxy_environment`, `test_redirects_are_not_followed_by_default`).

src/pulseops/security/transport.py, linhas 1–30

```
"""Transporte HTTPX que só conecta em IPs aprovados pela política (ADR 0005).

O `httpcore` chama `connect_tcp(host, porta)` do *network backend* para abrir cada
conexão. Este backend resolve o nome, valida todos os IPs e conecta no IP
validado. O `httpcore` continua usando o nome original no header Host, no SNI e
na verificação do certificado, porque esses valores vêm da URL e não do socket.
"""

import asyncio
import ipaddress
import socket
from collections.abc import Iterable
from typing import Any, Protocol

import httpcore
import httpx

from pulseops.security.url_policy import DestinationPolicy, IPAddress, UnsafeDestinationError


class Resolver(Protocol):
    async def resolve(self, host: str, port: int) -> list[IPAddress]: ...


class SystemResolver:
    async def resolve(self, host: str, port: int) -> list[IPAddress]:
        loop = asyncio.get_running_loop()
        # getaddrinfo do loop roda em thread separada: não bloqueia o event loop.
        infos = await loop.getaddrinfo(host, port, type=socket.SOCK_STREAM)
        return [ipaddress.ip_address(info[4][0]) for info in infos]
```

- Linhas 1–7: a docstring explica a técnica: o HTTPX (por meio do `httpcore`) pede ao *network backend* para abrir cada conexão. Substituindo o backend, o PulseOps decide em qual IP conectar. O `Host`, o SNI (nome enviado no TLS) e a verificação do certificado continuam usando o nome da URL.
- Linhas 21–22: protocolo do resolvedor, uma função assíncrona que devolve a lista de IPs.
- Linhas 25–30: o resolvedor real. `loop.getaddrinfo` executa a função bloqueante do sistema numa thread, sem travar o event loop. `type=socket.SOCK_STREAM` pede só endereços TCP. `info[4][0]` é o IP dentro da tupla que o `getaddrinfo` devolve.

src/pulseops/security/transport.py, linhas 33–80

```
class GuardedNetworkBackend(httpcore.AsyncNetworkBackend):
    def __init__(
        self,
        policy: DestinationPolicy,
        resolver: Resolver | None = None,
        inner: httpcore.AsyncNetworkBackend | None = None,
    ) -> None:
        self._policy = policy
        self._resolver = resolver or SystemResolver()
        self._inner = inner or httpcore.AnyIOBackend()

```

```
    async def resolve_allowed(self, host: str, port: int) -> IPAddress:
        try:
            addresses = await self._resolver.resolve(host, port)
        except OSError as exc:
            raise httpcore.ConnectError(f"não foi possível resolver o nome: {exc}") from exc
        if not addresses:
            raise httpcore.ConnectError("o nome não tem endereços")
        # Se qualquer endereço for proibido, recusa: evita escolher "o IP bom" de uma
        # resposta que mistura um IP público com um interno.
        for address in addresses:
            self._policy.check_address(host, port, address)
        return addresses[0]

    async def connect_tcp(
        self,
        host: str,
        port: int,
        timeout: float | None = None,  # noqa: ASYNC109 - assinatura do httpcore
        local_address: str | None = None,
        socket_options: Iterable[Any] | None = None,
    ) -> httpcore.AsyncNetworkStream:
        address = await self.resolve_allowed(host, port)
        return await self._inner.connect_tcp(
            str(address), port, timeout, local_address, socket_options
        )

    async def connect_unix_socket(
        self,
        path: str,
        timeout: float | None = None,  # noqa: ASYNC109 - assinatura do httpcore
        socket_options: Iterable[Any] | None = None,
    ) -> httpcore.AsyncNetworkStream:
        raise UnsafeDestinationError("sockets Unix não são permitidos")

    async def sleep(self, seconds: float) -> None:
        await self._inner.sleep(seconds)
```

- Linha 33: herda de `httpcore.AsyncNetworkBackend`, a interface que o `httpcore` espera.
- Linhas 34–42: recebe a política, o resolvedor (real por padrão) e o backend "de verdade" (`AnyIOBackend`), para o qual delega a conexão.
- Linhas 44–55: resolve e valida (volume 1, parte G). Erros de DNS viram `httpcore.ConnectError`, que o HTTPX traduz para `httpx.ConnectError` e o verificador classifica como falha de conexão.
- Linhas 57–68: `connect_tcp` é chamado pelo `httpcore`. Conecta no `str(address)`, o IP validado. O `# noqa: ASYNC109` silencia uma regra do Ruff que não gosta de parâmetros chamados `timeout` em funções assíncronas: a assinatura é imposta pelo `httpcore`.
- Linhas 70–76: socket Unix (arquivo local) é sempre recusado.
- Linhas 78–79: `sleep` faz parte da interface; delega.

`src/pulseops/security/transport.py, linhas 82–104`

```
def build_guarded_client(
    policy: DestinationPolicy,
    *,
    max_connections: int,
    user_agent: str,
    resolver: Resolver | None = None,
) -> httpx.AsyncClient:
    """Cliente HTTPX com o backend protegido, sem proxy do ambiente e sem redirecionamentos."""
    transport = httpx.AsyncHTTPTransport(trust_env=False, retries=0)
    # Detalhe interno do HTTPX 0.28: o transporte delega ao pool em `_pool`. Trocamos o
    # pool por um com o nosso backend. tests/unit/test_transport.py quebra se isso mudar.
    transport._pool = httpcore.AsyncConnectionPool(
        ssl_context=httpx.create_ssl_context(trust_env=False),
        max_connections=max_connections,
        max_keepalive_connections=0,
```

```
97          network_backend=GuardedNetworkBackend(policy, resolver),
98      )
99      return httpx.AsyncClient(
100         transport=transport,
101         trust_env=False,
102         follow_redirects=False,
103         headers={"User-Agent": user_agent},
104     )
```

- Linha 90: `trust_env=False` ignora `HTTP_PROXY` e afins do ambiente: um proxy faria a conexão sair de outro lugar, sem passar pela validação. `retries=0`: o HTTPX não repete conexões por conta própria.
- Linhas 91–98: troca o pool interno do transporte por um com o backend protegido. O comentário admite que é um **detalhe interno** do HTTPX 0.28 e aponta o teste que quebra se isso mudar. `max_keepalive_connections=0`: cada requisição abre uma conexão nova, então cada check passa pela validação de IP (e mede o custo de conexão de verdade).
- Linhas 99–104: o cliente final: sem proxy, sem seguir redirecionamentos, com o `User-Agent` do PulseOps.

src/pulseops/security/transport.py, linhas 107–123

```
107 async def validate_destination(
108     policy: DestinationPolicy, url: str, resolver: Resolver | None = None
109 ) -> None:
110     """Validação de cadastro: texto da URL e, se o nome resolver, os IPs atuais.
111
112     Um nome que não resolve agora é aceito (o serviço pode estar fora do ar); a
113     proteção definitiva acontece de novo em cada conexão.
114     """
115     destination = policy.validate_url(url)
116     if policy.is_demo_host(destination.host, destination.port):
117         return
118     try:
119         addresses = await (resolver or SystemResolver()).resolve(destination.host, destination.port)
120     except OSError:
121         return
122     for address in addresses:
123         policy.check_address(destination.host, destination.port, address)
```

Validação de **cadastro**: texto da URL e, se o nome resolver agora, os IPs atuais. Um nome que não resolve é aceito (linhas 120–121), porque o serviço pode estar fora do ar no momento do cadastro; a proteção definitiva acontece em cada conexão.

Falhas possíveis. Uma atualização do HTTPX que mude `_pool` quebra a injeção; o teste `test_connects_to_validated_ip_and_keeps_host_header` detecta. Um DNS que devolve listas diferentes a cada consulta não é problema, porque só uma consulta é feita por conexão e o IP usado é o validado.

`src/pulseops/security/passwords.py`

Finalidade. Gera e confere hashes de senha com Argon2id.

src/pulseops/security/passwords.py

```
1 """Hash de senha com Argon2id (biblioteca argon2-cffi, parâmetros padrão dela)."""
2
3 from argon2 import PasswordHasher
4 from argon2.exceptions import InvalidHashError, VerificationError
5
6 MIN_PASSWORD_LENGTH = 12
7
```

```
 8 _hasher = PasswordHasher()
 9 # Hash de uma senha qualquer, usado quando o usuário não existe: assim o login de um
10 # usuário inexistente leva o mesmo tempo que o de uma senha errada.
11 _DUMMY_HASH = _hasher.hash("pulseops-dummy-password")
12
13
14 def hash_password(password: str) -> str:
15     if len(password) < MIN_PASSWORD_LENGTH:
16         raise ValueError(f"a senha precisa ter pelo menos {MIN_PASSWORD_LENGTH} caracteres")
17     return _hasher.hash(password)
18
19
20 def verify_password(password_hash: str | None, password: str) -> bool:
21     try:
22         return _hasher.verify(password_hash or _DUMMY_HASH, password) and password_hash is not None
23     except (VerificationError, InvalidHashError, ValueError):
24         # Um hash corrompido no banco (por exemplo, texto com acentos) levanta
25         # UnicodeEncodeError, subclasse de ValueError: tratamos como senha inválida.
26         return False
```

- Linha 6: tamanho mínimo da senha.
- Linha 8: um `PasswordHasher` com os parâmetros padrão da biblioteca (Argon2id, 64 MiB de memória, 3 iterações, na versão travada).
- Linha 11: hash de uma senha fixa, calculado uma vez na importação. Serve para conferir "alguma coisa" quando o usuário não existe, gastando o mesmo tempo.
- Linhas 14–17: recusa senha curta e devolve o hash (o texto começa com `$argon2id$v=19$m=65536,t=3,p=4$...`, que guarda os parâmetros junto).
- Linhas 20–26: confere. Linha 22: se não há hash (usuário inexistente), confere contra o hash fictício e, mesmo que batesse, devolve `False` por causa do `and password_hash is not None`. Linhas 23–26: qualquer erro de formato vira "não confere" (bug B4).

Dados. Senha em texto (vinda do formulário ou da linha de comando) e hash (texto do banco). **Relações.** `services/auth.py`. **Falhas.** Senha curta levanta `ValueError` (tratado pelo `cli.py`). **Testes.** `tests/unit/test_passwords.py` (`test_hash_is_argon2id_and_verifies`, `test_short_password_is_rejected`, `test_missing_user_never_verifies`).

D5 — Cliente HTTP: o verificador e o envio de webhooks

`src/pulseops/checks/checker.py`

Finalidade. Faz **uma** requisição HTTP ao alvo e classifica o resultado numa das oito categorias, com uma explicação curta.

Dados. Entrada: `CheckTarget`, montado pelo worker a partir das colunas do monitor. Saída: `CheckObservation` (resultado, horários, duração em ms, status HTTP ou nulo, explicação). Nunca levanta exceção. **Relações.** Criado em `worker/__main__.py` com o cliente protegido; chamado por `worker/runner.py`; a observação é gravada por `services/processing.py`. **Testes.** `tests/unit/test_checker.py` (treze testes: saudável, lento, status inesperado, rede e TLS, DNS, handshake TLS, prazo total, redirecionamento, destino bloqueado, exceção inesperada, explicação sem URL, corpo não lido, método HEAD).

src/pulseops/checks/checker.py, linhas 18–46

```
18 @dataclass(frozen=True)
19 class CheckTarget:
20     url: str
21     method: str
22     timeout_seconds: float
23     latency_threshold_ms: int
24     accepted: AcceptedStatusCodes
25
26
27 @dataclass(frozen=True)
28 class CheckObservation:
29     outcome: Outcome
30     started_at: datetime
31     finished_at: datetime
32     duration_ms: int
33     status_code: int | None
34     explanation: str
35
36
37 class Checker(Protocol):
38     """Qualquer objeto com este método serve ao worker (útil para testes e novos tipos)."""
39
40     async def check(self, target: CheckTarget) -> CheckObservation: ...
41
42
43 class HttpChecker:
44     def __init__(self, client: httpx.AsyncClient, clock: Clock) -> None:
45         self._client = client
46         self._clock = clock
```

- Linhas 18–24: o que verificar. `accepted` já é o objeto de faixas, não o texto.
- Linhas 27–34: o que foi observado. `status_code` é nulo quando não houve resposta (timeout, conexão recusada).
- Linhas 37–40: o protocolo `Checker`. O worker depende dele, não de `HttpChecker`; um verificador de outro tipo (TCP, por exemplo) só precisaria ter esse método.
- Linhas 43–46: recebe o cliente HTTP e o relógio prontos (composição).

src/pulseops/checks/checker.py, linhas 48–89

```python
    async def check(self, target: CheckTarget) -> CheckObservation:
        started_at = self._clock.now()
        start = self._clock.monotonic()
        status_code: int | None = None
        try:
            # Prazo total da verificação. Os timeouts do HTTPX valem por fase (conectar,
            # ler...), então sozinhos poderiam somar mais que o configurado.
            async with asyncio.timeout(target.timeout_seconds):
                async with self._client.stream(
                    target.method,
                    target.url,
                    timeout=target.timeout_seconds,
                    follow_redirects=False,
                ) as response:
                    # O corpo não é lido (RF02.4): a duração vai até os cabeçalhos.
                    status_code = response.status_code
            outcome, explanation = self._classify_response(
                target, status_code, self._elapsed_ms(start)
            )
        except (TimeoutError, httpx.TimeoutException):
            outcome = Outcome.TIMEOUT
            explanation = f"sem resposta em {target.timeout_seconds:g} s"
        except UnsafeDestinationError as exc:
            outcome = Outcome.BLOCKED
            explanation = f"destino bloqueado pela política de rede: {exc}"
        except httpx.TransportError as exc:
            outcome, explanation = _classify_transport_error(exc)
        except Exception as exc:  # erro do próprio PulseOps: não pode virar "alvo caiu"
            outcome = Outcome.INTERNAL_ERROR
            explanation = f"erro interno ao verificar ({type(exc).__name__})"

        return CheckObservation(
            outcome=outcome,
            started_at=started_at,
            finished_at=self._clock.now(),
            duration_ms=self._elapsed_ms(start),
            status_code=status_code,
            explanation=explanation,
        )

    def _elapsed_ms(self, start: float) -> int:
        return round((self._clock.monotonic() - start) * 1000)
```

- Linhas 49–50: dois relógios. `now()` dá o horário para gravar; `monotonic()` mede a duração sem ser afetado por ajustes do relógio do sistema.
- Linhas 53–55: `asyncio.timeout` impõe um prazo **total**. O comentário explica: os timeouts do HTTPX valem por fase (conectar, escrever, ler); um alvo que responde devagar em cada fase poderia somar mais que o configurado. Ao estourar, a tarefa é cancelada e levanta `TimeoutError`.
- Linhas 56–61: `client.stream(...)` faz a requisição e devolve a resposta **sem ler o corpo**. `follow_redirects=False` é explícito (o cliente já não segue, mas aqui fica documentado).
- Linhas 62–63: guarda só o status. Ao sair do `async with`, a conexão é fechada sem baixar o corpo (RF02.4): um alvo que devolve 1 GB não consome banda nem memória do PulseOps.
- Linhas 64–66: classifica pelo status e pela duração.
- Linhas 67–77: cada tipo de erro vira uma categoria. A ordem importa: o `UnsafeDestinationError` (bloqueio SSRF na conexão) precisa ser tratado antes do genérico. A linha 75 é a última barreira: qualquer outra exceção é **erro interno**, nunca "o alvo caiu". `type(exc).__name__` registra só o nome da exceção, não a mensagem, que poderia conter a URL.
- Linhas 79–86: monta a observação. A duração é medida de novo no fim, então inclui também o tempo até a exceção.
- `:g` (linha 69) formata `5.0` como `5`.

src/pulseops/checks/checker.py, linhas 91–132

```python
    @staticmethod
    def _classify_response(
        target: CheckTarget, status_code: int, duration_ms: int
    ) -> tuple[Outcome, str]:
        if status_code not in target.accepted:
            return (
                Outcome.UNEXPECTED_STATUS,
                f"HTTP {status_code} fora dos status aceitos ({target.accepted})",
            )
        if duration_ms > target.latency_threshold_ms:
            return (
                Outcome.SLOW,
                f"HTTP {status_code} em {duration_ms} ms (limite {target.latency_threshold_ms} ms)",
            )
        return Outcome.HEALTHY, f"HTTP {status_code} em {duration_ms} ms"


def _classify_transport_error(exc: httpx.TransportError) -> tuple[Outcome, str]:
    """Separa TLS de conexão/DNS olhando a cadeia de causas da exceção."""
    causes = _cause_chain(exc)
    if any(isinstance(c, ssl.SSLCertVerificationError) for c in causes):
        return Outcome.TLS_ERROR, "falha TLS: certificado não confiável ou de outro nome"
    if any(isinstance(c, ssl.SSLError) for c in causes) or "SSL" in str(exc):
        return Outcome.TLS_ERROR, "falha TLS durante o handshake"
    if isinstance(exc, httpx.ConnectError):
        if any(isinstance(c, socket.gaierror) for c in causes):
            return Outcome.CONNECTION_ERROR, "falha de DNS: nome não resolvido"
        if any(isinstance(c, ConnectionRefusedError) for c in causes) or "refused" in str(exc):
            return Outcome.CONNECTION_ERROR, "conexão recusada"
        return Outcome.CONNECTION_ERROR, "falha de conexão"
    if isinstance(exc, httpx.RemoteProtocolError):
        return Outcome.CONNECTION_ERROR, "resposta HTTP inválida ou conexão encerrada pelo alvo"
    return Outcome.CONNECTION_ERROR, f"erro de rede ({type(exc).__name__})"


def _cause_chain(exc: BaseException) -> list[BaseException]:
    chain: list[BaseException] = []
    current: BaseException | None = exc
    while current is not None and current not in chain:
        chain.append(current)
        current = current.__cause__ or current.__context__
    return chain
```

- Linhas 91–105: `@staticmethod` porque não usa `self`. Primeiro o status (fora da lista é falha, mesmo se rápido), depois a duração (acima do limite é lento). A explicação traz o status e o tempo, nunca a URL.
- Linhas 108–123: o HTTPX embrulha exceções de baixo nível. Para saber se foi TLS ou DNS, é preciso olhar a **cadeia de causas**: certificado não confiável (linha 111), outro erro TLS (113), DNS (116, `socket.gaierror`), conexão recusada (118), outras falhas de conexão (120), resposta HTTP inválida (121–122).
- Linhas 126–132: percorre `__cause__` (o `raise ... from ...`) ou `__context__` (exceção levantada durante o tratamento de outra). A condição `current not in chain` evita laço infinito se houver ciclo.

Falhas possíveis. A classificação por texto (`"SSL" in str(exc)`, `"refused"`) depende das mensagens das bibliotecas; uma mudança de texto faria um erro TLS aparecer como “falha de conexão”. Continua sendo falha do alvo, só com a categoria menos precisa.

`src/pulseops/notifications/webhook.py`

Finalidade. Faz o POST de um evento de alerta para o destino e classifica o resultado (sucesso, erro temporário, erro permanente).

Dados. Entrada: URL do destino (do banco), `event_id` (texto estável) e o payload (JSON montado na criação do evento). Saída: `SendResult`. **Relações.** Criado em `worker/__main__.py`; chamado por `Worker._deliver` em `worker/runner.py`; o resultado é gravado por `services/alerts.py`. **Testes.** `test_webhook_notifier_sends_event_id_and_classifies_errors`, `test_webhook_timeout_is_retryable`, `test_blocked_destination_fails_permanently` (`tests/integration/test_delivery.py`).

src/pulseops/notifications/webhook.py

```python
"""Envio de eventos para um webhook genérico (RF06)."""

import asyncio
from typing import Any, Protocol

import httpx

from pulseops.clock import Clock
from pulseops.security.url_policy import DestinationPolicy, UnsafeDestinationError
from pulseops.services.alerts import SendResult, is_permanent_status


class Notifier(Protocol):
    async def send(self, url: str, event_id: str, payload: dict[str, Any]) -> SendResult: ...


class WebhookNotifier:
    def __init__(
        self,
        client: httpx.AsyncClient,
        clock: Clock,
        policy: DestinationPolicy,
        timeout_seconds: float,
    ) -> None:
        self._client = client
        self._clock = clock
        self._policy = policy
        self._timeout = timeout_seconds

    async def send(self, url: str, event_id: str, payload: dict[str, Any]) -> SendResult:
        start = self._clock.monotonic()

        def result(ok: bool, permanent: bool, status: int | None, error: str | None) -> SendResult:
            elapsed = round((self._clock.monotonic() - start) * 1000)
            return SendResult(ok, permanent, status, elapsed, error)

        try:
            self._policy.validate_url(url)  # a URL pode ter sido cadastrada com outra política
            async with asyncio.timeout(self._timeout):
                # stream(): só os cabeçalhos são lidos; o corpo da resposta é descartado.
                async with self._client.stream(
                    "POST",
                    url,
                    json=payload,
                    headers={"X-PulseOps-Event-Id": event_id},
                    timeout=self._timeout,
                ) as response:
                    status_code = response.status_code
        except UnsafeDestinationError as exc:
            return result(False, True, None, f"destino bloqueado: {exc}")
        except (TimeoutError, httpx.TimeoutException):
            return result(False, False, None, f"sem resposta em {self._timeout:g} s")
        except httpx.TransportError as exc:
            return result(False, False, None, f"erro de rede ({type(exc).__name__})")
```

```
55
56          if 200 <= status_code < 300:
57              return result(True, False, status_code, None)
58          return result(False, is_permanent_status(status_code), status_code, f"HTTP {status_code}")
```

- Linhas 13–14: o protocolo; os testes usam um notificador falso que só anota as chamadas.
- Linhas 33–35: uma função interna que calcula a duração e monta o resultado. Evita repetir o cálculo em cada `return`.
- Linha 38: valida a URL de novo **no envio**: a política pode ter mudado desde o cadastro (por exemplo, a demonstração foi desligada).
- Linhas 39–48: mesmo padrão do verificador: prazo total, `stream` sem ler o corpo, o JSON no corpo do pedido e o `event_id` também num header, para o destino poder deduplicar sem abrir o corpo.
- Linha 50: destino bloqueado é **permanente** (`True` no segundo argumento): repetir não muda nada.
- Linhas 51–54: timeout e erro de rede são temporários.
- Linhas 56–58: 2xx é sucesso; outro status é erro, permanente ou não conforme `is_permanent_status` (4xx, exceto 408 e 429).

Falhas possíveis. Uma exceção fora das três tratadas (um bug no próprio notificador, por exemplo) sobe até `Worker._deliver`. Nesse caso nada é gravado, a entrega fica `sending` e volta à fila quando a reserva vence; o laço de entregas registra o erro e continua.

D6 — Serviços: os casos de uso

`src/pulseops/services/` é a camada que junta domínio e banco. Cada função recebe uma sessão (ou a fábrica de sessões) e executa um caso de uso: criar monitor, gravar resultado, abrir incidente, reservar entregas. As rotas da API e o worker só chamam essas funções.

Duas formas aparecem e é importante distingui-las:

- funções que recebem `session: AsyncSession` trabalham **dentro** de uma transação aberta por quem chama (as rotas e o próprio `processing.py`);
- funções que recebem `sessions: SessionFactory` abrem e fecham a própria transação curta (as do worker: `claim_due_runs`, `record_result`…).

`src/pulseops/services/errors.py`

Finalidade. Três exceções de caso de uso, independentes de HTTP.

src/pulseops/services/errors.py

```
 1 """Erros de caso de uso, traduzidos para HTTP pelas rotas."""
 2
 3 from pulseops.domain.monitor_rules import FieldError
 4
 5
 6 class NotFoundError(Exception):
 7     pass
 8
 9
10 class ConflictError(Exception):
11     """Operação incompatível com o estado atual (por exemplo, editar monitor arquivado)."""
12
13
14 class ValidationFailedError(Exception):
15     def __init__(self, errors: list[FieldError]) -> None:
16         super().__init__("; ".join(f"{e.field}: {e.message}" for e in errors))
17         self.errors = errors
```

- `NotFoundError` (6–7): o recurso não existe; as rotas devolvem 404.
- `ConflictError` (10–11): a operação não combina com o estado (pausar um monitor já pausado); as rotas devolvem 409.
- `ValidationFailedError` (14–17): carrega a lista de `FieldError`. A linha 16 monta uma mensagem legível para logs; a 17 guarda a lista para a API devolver 422 com os campos e o formulário mostrar cada erro ao lado do campo.

Relações. Levantadas em `services/monitors.py`; traduzidas em `web/api.py` (`_http_error`) e `web/pages.py`. **Testes.** `test_invalid_configuration_is_rejected`, `test_pause_resume_and_conflicts`.

`src/pulseops/services/auth.py`

Finalidade. Operadora, login e sessões (RF08, ADR 0008).

Dados. Nome e senha (texto, do formulário); token de sessão (texto aleatório de 43 caracteres, vai para o cookie); hash SHA-256 do token (vai para o banco). **Relações.** Usa `security/passwords.py` e os modelos `Operator`, `OperatorSession`; chamado por `cli.py`, `web/pages.py` (login e logout) e `web/deps.py` (cada requisição). **Testes.** `tests/integration/test_auth.py` (16 funções, 26 casos) e `tests/integration/test_cli.py`.

src/pulseops/services/auth.py, linhas 15–38

```
15 def hash_token(token: str) -> str:
16     return hashlib.sha256(token.encode()).hexdigest()
17
18
19 async def create_operator(
20     session: AsyncSession, username: str, password: str, *, now: datetime
21 ) -> Operator:
22     username = username.strip()
23     if not 3 <= len(username) <= 50:
24         raise ValueError("o usuário precisa ter de 3 a 50 caracteres")
25     if await session.scalar(select(Operator.id).where(Operator.username == username)):
26         raise ValueError(f"o usuário {username!r} já existe")
27     operator = Operator(username=username, password_hash=hash_password(password), created_at=now)
28     session.add(operator)
29     await session.flush()
30     return operator
31
32
33 async def authenticate(session: AsyncSession, username: str, password: str) -> Operator | None:
34     operator = await session.scalar(select(Operator).where(Operator.username == username.strip()))
35     # verify_password roda mesmo sem usuário, para não revelar quais usuários existem pelo tempo.
36     if verify_password(operator.password_hash if operator else None, password):
37         return operator
38     return None
```

- Linhas 15–16: SHA-256 em hexadecimal. Para tokens aleatórios longos, um hash rápido basta (não há o que adivinhar); Argon2 é para senhas, que são curtas e previsíveis.
- Linhas 19–30: cria a operadora. `strip()` remove espaços nas pontas; o tamanho é conferido (3 a 50); um nome repetido levanta `ValueError` (linha 26). `flush()` manda o `INSERT` sem confirmar a transação, para gerar o `id`.
- Linhas 33–38: login (volume 1, parte G). Devolve a operadora ou `None`, nunca diz qual dos dois (usuário ou senha) estava errado.

src/pulseops/services/auth.py, linhas 41–86

```
41 @dataclass(frozen=True)
42 class NewSession:
43     token: str  # vai para o cookie; o banco guarda só o hash
44     csrf_token: str
45     expires_at: datetime
46
47
48 async def start_session(
49     session: AsyncSession, operator: Operator, *, now: datetime, ttl: timedelta
50 ) -> NewSession:
51     token = secrets.token_urlsafe(32)
52     csrf_token = secrets.token_urlsafe(32)
53     expires_at = now + ttl
54     session.add(
55         OperatorSession(
56             token_hash=hash_token(token),
57             operator_id=operator.id,
58             csrf_token=csrf_token,
59             created_at=now,
60             expires_at=expires_at,
61         )
```

```
    )
    # Aproveita o login para limpar sessões vencidas.
    await session.execute(delete(OperatorSession).where(OperatorSession.expires_at <= now))
    return NewSession(token, csrf_token, expires_at)


async def load_session(
    session: AsyncSession, token: str, *, now: datetime
) -> tuple[OperatorSession, Operator] | None:
    row = (
        await session.execute(
            select(OperatorSession, Operator)
            .join(Operator, Operator.id == OperatorSession.operator_id)
            .where(
                OperatorSession.token_hash == hash_token(token), OperatorSession.expires_at > now
            )
        )
    ).first()
    return (row[0], row[1]) if row else None


async def end_session(session: AsyncSession, token: str) -> None:
    await session.execute(
        delete(OperatorSession).where(OperatorSession.token_hash == hash_token(token))
    )
```

- Linhas 41–45: o que o login devolve para a rota: o token (para o cookie), o token CSRF (para os formulários) e a validade.
- Linhas 48–65: `secrets.token_urlsafe(32)` gera 32 bytes aleatórios do gerador criptográfico do sistema, em base64 seguro para URL. O banco guarda o hash (linha 56). A linha 64 aproveita para apagar sessões vencidas de todo mundo.
- Linhas 68–80: carrega a sessão pelo hash do cookie, só se não estiver vencida (linha 76), junto com a operadora. Devolve uma tupla ou `None`.
- Linhas 83–86: logout apaga a linha da sessão. O cookie antigo deixa de valer imediatamente (`test_logout_invalidates_session`).

Falhas possíveis. Nenhuma exceção no caminho normal; erro de banco sobe para a rota (erro 500).

`src/pulseops/services/monitors.py`

Finalidade. Cadastro de monitores e o ciclo de vida: criar, editar, pausar, retomar, arquivar e "verificar agora" (RF01).

Dados. `MonitorConfig` vindo da API ou do formulário; o horário atual; a política de destinos. Grava em `monitors`, `health_changes` e, via `incidents`, em `incident_events`. **Relações.** Chamado por `web/api.py` e `web/pages.py`; usa `domain/monitor_rules.py`, `security/transport.py` e `services/incidents.py`. **Testes.** `tests/integration/test_api_monitors.py`, `test_edit_and_lifecycle_actions_through_forms`, `test_pause_closes_open_incident_and_resume_starts_clean`, `test_config_change_resets_counters_and_keeps_incident`.

`src/pulseops/services/monitors.py, linhas 19–44`

```
async def _validate(
    session: AsyncSession,
    config: MonitorConfig,
    *,
    policy: DestinationPolicy,
    min_interval_seconds: int,
    resolver: Resolver | None,
```

```
    exclude_id: int | None = None,
) -> None:
    errors = validate_config(config, min_interval_seconds=min_interval_seconds)
    if not any(e.field == "url" for e in errors):
        try:
            await validate_destination(policy, config.url, resolver)
        except UnsafeDestinationError as exc:
            errors.append(FieldError("url", str(exc)))

    duplicate = select(Monitor.id).where(
        func.lower(Monitor.name) == config.name.strip().lower(), Monitor.state != "archived"
    )
    if exclude_id is not None:
        duplicate = duplicate.where(Monitor.id != exclude_id)
    if await session.scalar(duplicate):
        errors.append(FieldError("name", "já existe um monitor com este nome"))

    if errors:
        raise ValidationFailedError(errors)
```

`_validate` junta três validações e levanta **uma** exceção com todos os erros:

- Linha 28: regras de campo (`domain/monitor_rules.py`).
- Linhas 29–33: só valida o destino (SSRF) se a URL passou nas regras de tamanho; o erro vira um `FieldError` da URL.
- Linhas 35–41: nome repetido, sem diferenciar maiúsculas, entre não arquivados. Na edição, o próprio monitor é excluído da busca (`exclude_id`).

`src/pulseops/services/monitors.py, linhas 47–83`

```
async def create_monitor(
    session: AsyncSession,
    config: MonitorConfig,
    *,
    now: datetime,
    policy: DestinationPolicy,
    min_interval_seconds: int,
    resolver: Resolver | None = None,
) -> Monitor:
    await _validate(
        session, config, policy=policy, min_interval_seconds=min_interval_seconds, resolver=resolver
    )
    values = asdict(config)
    values["name"] = config.name.strip()
    values["url"] = config.url.strip()
    monitor = Monitor(
        **values,
        state="active",
        health=Health.NO_DATA,
        health_reason="Ainda sem verificações.",
        health_changed_at=now,
        consecutive_failures=0,
        consecutive_successes=0,
        config_version=1,
        check_requested=False,
        next_run_at=now,  # primeira verificação logo após o cadastro
        created_at=now,
        updated_at=now,
    )
    session.add(monitor)
    try:
        await session.flush()
    except IntegrityError as exc:  # corrida entre dois cadastros com o mesmo nome
        raise ValidationFailedError(
            [FieldError("name", "já existe um monitor com este nome")]
        ) from exc
    return monitor
```

- Linhas 56–58: valida antes de qualquer gravação.
- Linhas 59–61: `asdict` transforma o `MonitorConfig` em dicionário; nome e URL são limpos de espaços.
- Linhas 62–75: `Monitor(**values, ...)` passa o dicionário como argumentos nomeados e completa o estado inicial: ativo, sem dados, contadores zerados, versão 1 e primeira verificação imediata (linha 72).
- Linhas 77–82: o `flush` dispara o índice único de nome. Se outra requisição cadastrou o mesmo nome **entre** a conferência da linha 41 e agora, o banco recusa, e o erro vira a mesma mensagem de validação. É a "corrida" que o índice resolve.

src/pulseops/services/monitors.py, linhas 86–105

```
 86 async def get_monitor(session: AsyncSession, monitor_id: int, *, lock: bool = False) -> Monitor:
 87     monitor = await session.get(Monitor, monitor_id, with_for_update=lock)
 88     if monitor is None:
 89         raise NotFoundError(f"monitor {monitor_id} não encontrado")
 90     return monitor
 91
 92
 93 def config_of(monitor: Monitor) -> MonitorConfig:
 94     return MonitorConfig(
 95         name=monitor.name,
 96         url=monitor.url,
 97         method=monitor.method,
 98         interval_seconds=monitor.interval_seconds,
 99         timeout_seconds=monitor.timeout_seconds,
100         accepted_status_codes=monitor.accepted_status_codes,
101         latency_threshold_ms=monitor.latency_threshold_ms,
102         failure_threshold=monitor.failure_threshold,
103         recovery_threshold=monitor.recovery_threshold,
104         description=monitor.description,
105     )
```

- Linhas 86–90: busca por id, opcionalmente com trava (`FOR UPDATE`); se não existe, `NotFoundError`.
- Linhas 93–105: o caminho inverso de `create_monitor`: monta a configuração a partir do monitor gravado (usado para comparar na edição e para preencher o formulário).

src/pulseops/services/monitors.py, linhas 108–150

```
108 async def update_monitor(
109     session: AsyncSession,
110     monitor_id: int,
111     config: MonitorConfig,
112     *,
113     now: datetime,
114     policy: DestinationPolicy,
115     min_interval_seconds: int,
116     resolver: Resolver | None = None,
117 ) -> Monitor:
118     monitor = await get_monitor(session, monitor_id, lock=True)
119     if monitor.state == "archived":
120         raise ConflictError("monitor arquivado não pode ser editado")
121     await _validate(
122         session,
123         config,
124         policy=policy,
125         min_interval_seconds=min_interval_seconds,
126         resolver=resolver,
127         exclude_id=monitor.id,
128     )
129
130     before = asdict(config_of(monitor))
131     after = asdict(config)
```

```
    after["name"] = config.name.strip()
    after["url"] = config.url.strip()
    changed_check_fields = [f for f in CHECK_FIELDS if before[f] != after[f]]
    for field, value in after.items():
        setattr(monitor, field, value)
    monitor.updated_at = now

    if changed_check_fields:
        # RF03.5: a configuração mudou o significado dos contadores; recomeça a contagem.
        monitor.config_version += 1
        monitor.consecutive_failures = 0
        monitor.consecutive_successes = 0
        monitor.first_failure_at = None
        await incidents.record_config_change(
            session, monitor=monitor, changed=changed_check_fields, now=now
        )
    if before["interval_seconds"] != after["interval_seconds"] and monitor.state == "active":
        monitor.next_run_at = now
    return monitor
```

- Linha 118: trava o monitor durante a edição (serializa com o worker).
- Linhas 119–120: arquivado não se edita.
- Linhas 130–134: compara antes e depois e lista os campos de `CHECK_FIELDS` que mudaram.
- Linhas 135–137: `setattr` aplica cada campo. É genérico de propósito: um campo novo em `MonitorConfig` passa a ser gravado sem mudar esta função.
- Linhas 139–147: se mudou algo que afeta a contagem, incrementa a versão, zera os contadores e registra o evento no incidente aberto, se houver (RF03.5). Essas quatro linhas repetem o que `_reset_counters` (linhas 171–177) faz; poderiam chamá-la.
- Linhas 148–149: mudou o intervalo de um monitor ativo: verifica já, em vez de esperar o agendamento antigo.

`src/pulseops/services/monitors.py, linhas 153–177`

```
def _set_health(
    session: AsyncSession, monitor: Monitor, health: Health, reason: str, now: datetime
) -> None:
    if monitor.health != health:
        session.add(
            HealthChange(
                monitor_id=monitor.id,
                changed_at=now,
                from_health=monitor.health,
                to_health=health,
                reason=reason,
            )
        )
        monitor.health = health
        monitor.health_changed_at = now
    monitor.health_reason = reason


def _reset_counters(monitor: Monitor) -> None:
    # Nova versão também invalida checks que já estavam em andamento: se o monitor for
    # retomado antes de eles terminarem, o resultado fica no histórico sem efeito (B6).
    monitor.config_version += 1
    monitor.consecutive_failures = 0
    monitor.consecutive_successes = 0
    monitor.first_failure_at = None
```

- Linhas 153–168: muda a saúde **e registra** a mudança em `health_changes`, sem check associado (foi a operadora, não um check). Se a saúde não muda, só atualiza o motivo (linha 168).
- Linhas 171–177: zera a contagem e incrementa a versão. O comentário explica o bug B6.

src/pulseops/services/monitors.py, linhas 180–236

```python
async def pause_monitor(session: AsyncSession, monitor_id: int, *, now: datetime) -> Monitor:
    monitor = await get_monitor(session, monitor_id, lock=True)
    if monitor.state != "active":
        raise ConflictError("apenas monitores ativos podem ser pausados")
    await incidents.close_incident_by_operator(session, monitor=monitor, action="paused", now=now)
    monitor.state = "paused"
    monitor.next_run_at = None
    monitor.check_requested = False
    monitor.updated_at = now
    _reset_counters(monitor)
    _set_health(session, monitor, Health.PAUSED, "Monitor pausado pelo operador.", now)
    return monitor


async def resume_monitor(session: AsyncSession, monitor_id: int, *, now: datetime) -> Monitor:
    monitor = await get_monitor(session, monitor_id, lock=True)
    if monitor.state != "paused":
        raise ConflictError("apenas monitores pausados podem ser retomados")
    monitor.state = "active"
    monitor.next_run_at = now
    monitor.updated_at = now
    _reset_counters(monitor)
    _set_health(
        session,
        monitor,
        Health.NO_DATA,
        "Monitor retomado; aguardando a primeira verificação.",
        now,
    )
    return monitor


async def archive_monitor(session: AsyncSession, monitor_id: int, *, now: datetime) -> Monitor:
    """Arquiva sem apagar nada: checks, incidentes e alertas continuam consultáveis (RF01.8)."""
    monitor = await get_monitor(session, monitor_id, lock=True)
    if monitor.state == "archived":
        raise ConflictError("monitor já arquivado")
    await incidents.close_incident_by_operator(session, monitor=monitor, action="archived", now=now)
    monitor.state = "archived"
    monitor.archived_at = now
    monitor.next_run_at = None
    monitor.check_requested = False
    monitor.updated_at = now
    _reset_counters(monitor)
    _set_health(session, monitor, Health.PAUSED, "Monitor arquivado pelo operador.", now)
    return monitor


async def request_check_now(session: AsyncSession, monitor_id: int, *, now: datetime) -> Monitor:
    """Antecipa a próxima execução. A reserva continua passando pelo mesmo controle de
    sobreposição; se já houver um check em andamento, este espera ele terminar (CB5)."""
    monitor = await get_monitor(session, monitor_id, lock=True)
    if monitor.state != "active":
        raise ConflictError("apenas monitores ativos podem ser verificados")
    monitor.next_run_at = now
    monitor.check_requested = True
    return monitor
```

As quatro ações seguem o mesmo padrão: travar o monitor, conferir o estado (`ConflictError` se não combina), alterar e registrar.

- Pausar (180–191): encerra o incidente aberto como `closed` (ADR 0006), tira o agendamento (`next_run_at = None`), zera a contagem e marca “pausado”.
- Retomar (194–209): agenda para agora e volta para “sem dados”. O monitor recomeça limpo: um incidente anterior não é reaberto.

- Arquivar (212–225): igual a pausar, mais a data de arquivamento. Nada é apagado (RF01.8).
- Verificar agora (228–236): antecipa o agendamento e marca o próximo check como manual. Não executa nada na rota (volume 1, H7).

Falhas possíveis. `NotFoundError`, `ConflictError` e `ValidationFailedError`, traduzidas pelas rotas. Erro de banco sobe como erro 500.

`src/pulseops/services/incidents.py`

Finalidade. Todas as mudanças em incidentes: abrir, registrar nova falha, resolver, encerrar pela operadora e registrar mudança de configuração. Cada uma grava eventos na linha do tempo e, quando é uma transição, o alerta.

Dados. Monitor e check já carregados na transação de quem chama. **Relações.** Chamado por `services/processing.py` (regras do worker) e `services/monitors.py` (ações da operadora); chama `services/alerts.py`. **Testes.** `tests/integration/test_incident_lifecycle.py` (nove testes), `test_incident_list_detail_and_timeline`, `test_outbox_event_is_atomic_with_incident`.

src/pulseops/services/incidents.py, linhas 1–35

```
1  """Mudanças em incidentes. Sempre chamadas dentro de uma transação aberta por quem chama."""
2
3  from datetime import datetime
4  from typing import Any
5
6  from sqlalchemy import select
7  from sqlalchemy.ext.asyncio import AsyncSession
8
9  from pulseops.db.models import Check, Incident, IncidentEvent, Monitor
10 from pulseops.services.alerts import enqueue_incident_alert
11
12
13 def _completed(check: Check) -> tuple[str, datetime]:
14     if check.outcome is None or check.finished_at is None:
15         raise ValueError(f"check {check.id} ainda não foi concluído")
16     return check.outcome, check.finished_at
17
18
19 def _snapshot(monitor: Monitor) -> dict[str, Any]:
20     """Valores vigentes guardados junto do evento (RF03.12)."""
21     return {
22         "failure_threshold": monitor.failure_threshold,
23         "recovery_threshold": monitor.recovery_threshold,
24         "timeout_seconds": monitor.timeout_seconds,
25         "latency_threshold_ms": monitor.latency_threshold_ms,
26         "config_version": monitor.config_version,
27     }
28
29
30 async def get_open_incident(session: AsyncSession, monitor_id: int) -> Incident | None:
31     return await session.scalar(
32         select(Incident)
33         .where(Incident.monitor_id == monitor_id, Incident.status == "open")
34         .with_for_update()
35     )
```

- Linha 1: a docstring é a regra do arquivo: **nunca** abre transação; sempre roda dentro da de quem chama.
- Linhas 13–16: garante que o check está concluído e devolve resultado e fim. A checagem existe também para o mypy: as colunas são `| None`.

- Linhas 19–27: o “retrato” da configuração vigente, guardado em cada evento.
- Linhas 30–35: busca o incidente aberto **travando-o** (`FOR UPDATE`).

`src/pulseops/services/incidents.py, linhas 38–94`

```python
async def open_incident(
    session: AsyncSession,
    *,
    monitor: Monitor,
    check: Check,
    first_failure_at: datetime,
    failure_count: int,
    reason: str,
) -> Incident:
    outcome, finished_at = _completed(check)
    incident = Incident(
        monitor_id=monitor.id,
        status="open",
        started_at=first_failure_at,
        confirmed_at=finished_at,
        failure_threshold=monitor.failure_threshold,
        recovery_threshold=monitor.recovery_threshold,
        timeout_seconds=monitor.timeout_seconds,
        latency_threshold_ms=monitor.latency_threshold_ms,
        opening_reason=reason,
        failure_count=failure_count,
        last_outcome=outcome,
        last_explanation=check.explanation or "",
        last_check_id=check.id,
    )
    session.add(incident)
    await session.flush()  # gera o id; o índice único parcial é verificado aqui

    details = _snapshot(monitor)
    session.add_all(
        [
            IncidentEvent(
                incident_id=incident.id,
                kind="first_failure",
                occurred_at=first_failure_at,
                message="Primeira falha da sequência observada.",
                details=details,
            ),
            IncidentEvent(
                incident_id=incident.id,
                kind="opened",
                occurred_at=finished_at,
                message=reason,
                details={**details, "outcome": outcome, "status_code": check.status_code},
                check_id=check.id,
            ),
        ]
    )
    await enqueue_incident_alert(
        session,
        kind="incident.opened",
        incident=incident,
        monitor=monitor,
        explanation=reason,
        now=finished_at,
    )
    return incident
```

- Linhas 48–62: cria o incidente com início (primeira falha), confirmação (este check), retrato dos limiares e o último resultado.
- Linha 64: `flush` gera o `id`; o comentário lembra que é aqui que o índice único de “um aberto por monitor” é verificado.

- Linhas 66–85: dois eventos de uma vez (`add_all`): “primeira falha” com o horário do início e “aberto” com o motivo e o status HTTP.
- Linhas 86–93: o alerta `incident.opened`, na mesma transação.

`src/pulseops/services/incidents.py, linhas 97–145`

```
 97 async def record_failure_on_open_incident(
 98     session: AsyncSession, *, incident: Incident, monitor: Monitor, check: Check
 99 ) -> None:
100     """Novas falhas atualizam o mesmo incidente; só registra evento se o tipo de erro mudar."""
101     outcome, finished_at = _completed(check)
102     incident.failure_count += 1
103     incident.last_check_id = check.id
104     incident.last_explanation = check.explanation or ""
105     if outcome != incident.last_outcome:
106         session.add(
107             IncidentEvent(
108                 incident_id=incident.id,
109                 kind="error_changed",
110                 occurred_at=finished_at,
111                 message=f"Tipo de falha mudou de {incident.last_outcome} para {outcome}: "
112                 f"{check.explanation}.",
113                 details={"from": incident.last_outcome, "to": outcome},
114                 check_id=check.id,
115             )
116         )
117         incident.last_outcome = outcome
118
119
120 async def resolve_incident(
121     session: AsyncSession, *, incident: Incident, monitor: Monitor, check: Check, reason: str
122 ) -> None:
123     _, finished_at = _completed(check)
124     incident.status = "resolved"
125     incident.ended_at = finished_at
126     incident.resolution_reason = "recovered"
127     incident.resolution_explanation = reason
128     session.add(
129         IncidentEvent(
130             incident_id=incident.id,
131             kind="resolved",
132             occurred_at=finished_at,
133             message=reason,
134             details=_snapshot(monitor),
135             check_id=check.id,
136         )
137     )
138     await enqueue_incident_alert(
139         session,
140         kind="incident.resolved",
141         incident=incident,
142         monitor=monitor,
143         explanation=reason,
144         now=finished_at,
145     )
```

- Linhas 97–117: nova falha com incidente aberto. Atualiza contagem e último check. Só cria um evento se o **tipo** de falha mudou (de timeout para 500, por exemplo): a linha do tempo não ganha uma linha por check. Teste: `test_error_type_change_is_recorded_on_timeline`. O parâmetro `monitor` não é usado dentro da função; está na assinatura por simetria com as outras.
- Linhas 120–145: resolução por recuperação: status `resolved`, fim, motivo, evento e alerta `incident.resolved`.

src/pulseops/services/incidents.py, linhas 148–201

```python
async def close_incident_by_operator(
    session: AsyncSession, *, monitor: Monitor, action: str, now: datetime
) -> Incident | None:
    """Pausa/arquivamento encerram o incidente como 'closed' (ADR 0006)."""
    incident = await get_open_incident(session, monitor.id)
    if incident is None:
        return None
    verb = "pausado" if action == "paused" else "arquivado"
    reason = (
        f"Monitor {verb} pelo operador com o incidente aberto; o incidente foi encerrado "
        "sem recuperação observada."
    )
    incident.status = "closed"
    incident.ended_at = now
    incident.resolution_reason = f"monitor_{action}"
    incident.resolution_explanation = reason
    session.add(
        IncidentEvent(
            incident_id=incident.id,
            kind="closed",
            occurred_at=now,
            message=reason,
            details=_snapshot(monitor),
        )
    )
    await enqueue_incident_alert(
        session,
        kind="incident.closed",
        incident=incident,
        monitor=monitor,
        explanation=reason,
        now=now,
    )
    return incident


async def record_config_change(
    session: AsyncSession, *, monitor: Monitor, changed: list[str], now: datetime
) -> None:
    incident = await get_open_incident(session, monitor.id)
    if incident is None:
        return
    session.add(
        IncidentEvent(
            incident_id=incident.id,
            kind="config_changed",
            occurred_at=now,
            message="Configuração alterada com incidente aberto ("
            + ", ".join(changed)
            + "); contadores reiniciados. A recuperação passa a exigir "
            f"{monitor.recovery_threshold} sucesso(s) consecutivo(s).",
            details={**_snapshot(monitor), "changed_fields": changed},
        )
    )
```

- Linhas 148–181: encerramento pela operadora (pausa ou arquivamento). O status é `closed`, diferente de `resolved`: o texto deixa claro que **não** houve recuperação observada. Também gera alerta (`incident.closed`).
- Linhas 184–201: mudança de configuração com incidente aberto: um evento `config_changed` com os campos alterados e o novo limiar de recuperação. Não gera alerta (não é transição de estado).

Falhas possíveis. Um segundo incidente aberto violaria o índice único: `IntegrityError` e rollback de tudo (o check fica `running` até a manutenção). Não há caso conhecido que leve a isso, porque o monitor está travado.

`src/pulseops/services/alerts.py`

Finalidade. O outbox de alertas: criação do evento e das entregas, reserva das entregas vencidas e gravação do resultado de cada tentativa (RF06, ADR 0007).

Dados. Incidente e monitor (para montar o payload); configurações de tentativas e espera; `SendResult` do notificador. **Relações.** Chamado por `services/incidents.py` (criação) e `worker/runner.py` (reserva e gravação); `SendResult` e `is_permanent_status` são usados por `notifications/webhook.py`. **Testes.** `tests/integration/test_delivery.py` (13 funções, 17 casos).

src/pulseops/services/alerts.py, linhas 22–89

```python
def event_key(incident_id: int, kind: str) -> str:
    return f"incident:{incident_id}:{kind.removeprefix('incident.')}"


async def enqueue_incident_alert(
    session: AsyncSession,
    *,
    kind: str,
    incident: Incident,
    monitor: Monitor,
    explanation: str,
    now: datetime,
) -> AlertEvent:
    """Cria o evento e uma entrega por destino ativo. Não faz commit: quem chama decide.

    Deve ser chamada dentro da mesma transação que mudou o incidente.
    """
    key = event_key(incident.id, kind)
    payload: dict[str, Any] = {
        "event_id": key,
        "type": kind,
        "occurred_at": now.isoformat(),
        "explanation": explanation,
        "monitor": {
            "id": monitor.id,
            "name": monitor.name,
            "url": redact_text(monitor.url),
        },
        "incident": {
            "id": incident.id,
            "status": incident.status,
            "started_at": incident.started_at.isoformat(),
            "confirmed_at": incident.confirmed_at.isoformat(),
            "ended_at": incident.ended_at.isoformat() if incident.ended_at else None,
            "failure_count": incident.failure_count,
            "failure_threshold": incident.failure_threshold,
            "recovery_threshold": incident.recovery_threshold,
            "last_outcome": incident.last_outcome,
            "last_explanation": incident.last_explanation,
        },
    }
    event = AlertEvent(
        event_key=key,
        kind=kind,
        incident_id=incident.id,
        monitor_id=monitor.id,
        payload=payload,
        created_at=now,
    )
```

```
    session.add(event)
    await session.flush()

    destinations = await session.scalars(
        select(AlertDestination).where(AlertDestination.enabled.is_(True))
    )
    for destination in destinations:
        session.add(
            Delivery(
                event_id=event.id,
                destination_id=destination.id,
                status="pending",
                attempt_count=0,
                next_attempt_at=now,
                created_at=now,
                updated_at=now,
            )
        )
    return event
```

- Linhas 22–23: a chave estável: `incident:42:opened`. `removeprefix` tira o `incident.` do tipo.
- Linhas 40–62: o payload. A URL do monitor vai **mascarada** (linha 48). As datas em ISO 8601 com fuso.
- Linhas 63–72: grava o evento e faz `flush` para ter o `id`.
- Linhas 74–88: uma entrega `pending` por destino **ativo**, para tentar já (`next_attempt_at=now`). Sem destino, o evento fica gravado sem entregas.

src/pulseops/services/alerts.py, linhas 92–99

```
def backoff_delay(attempt: int, base_seconds: float, max_seconds: float) -> float:
    """Espera antes da próxima tentativa após `attempt` falhas: base × 2^(attempt−1), com teto."""
    return float(min(base_seconds * 2 ** (attempt - 1), max_seconds))


def is_permanent_status(status_code: int) -> bool:
    """4xx indica pedido recusado pelo destino; repetir não ajuda. 408 e 429 são temporários."""
    return 400 <= status_code < 500 and status_code not in (408, 429)
```

Espera exponencial com teto e classificação de status permanentes (volume 1, parte G). `2 ** (attempt - 1)` é potência: 1, 2, 4, 8…

src/pulseops/services/alerts.py, linhas 102–143

```
@dataclass(frozen=True)
class ClaimedDelivery:
    delivery_id: int
    attempt_number: int
    url: str
    event_key: str
    payload: dict[str, Any]


async def claim_due_deliveries(
    sessions: SessionFactory, *, now: datetime, limit: int, lease_seconds: float
) -> list[ClaimedDelivery]:
    """Reserva entregas vencidas. Uma reserva vencida ('sending' antigo) volta a ser elegível,
    o que cobre o worker que morreu no meio do envio (e pode gerar duplicata: RF06.8)."""
    async with sessions.begin() as session:
        rows = (
            await session.execute(
                select(Delivery, AlertEvent, AlertDestination)
                .join(AlertEvent, AlertEvent.id == Delivery.event_id)
                .join(AlertDestination, AlertDestination.id == Delivery.destination_id)
                .where(Delivery.status.in_(("pending", "sending")), Delivery.next_attempt_at <= now)
                .order_by(Delivery.next_attempt_at, Delivery.id)
                .limit(limit)
```

```
                .with_for_update(of=Delivery, skip_locked=True)
            )
        ).all()
        claimed = []
        for delivery, event, destination in rows:
            delivery.status = "sending"
            delivery.attempt_count += 1
            delivery.next_attempt_at = now + timedelta(seconds=lease_seconds)
            delivery.updated_at = now
            claimed.append(
                ClaimedDelivery(
                    delivery_id=delivery.id,
                    attempt_number=delivery.attempt_count,
                    url=destination.url,
                    event_key=event.event_key,
                    payload=event.payload,
                )
            )
        return claimed
```

- Linhas 102–108: o que o worker precisa para enviar, copiado para um objeto simples (não do ORM), porque o envio acontece **depois** do commit.
- Linhas 116–127: busca as entregas vencidas com o evento e o destino (`JOIN`), travando só as entregas e pulando as já travadas (SQL na parte F do volume 1). Inclui `sending` com prazo vencido: reserva abandonada.
- Linhas 129–133: marca como `sending`, soma a tentativa e põe o fim da reserva em `next_attempt_at`.

`src/pulseops/services/alerts.py, linhas 146–204`

```
@dataclass(frozen=True)
class SendResult:
    ok: bool
    permanent: bool
    status_code: int | None
    duration_ms: int
    error: str | None


async def record_delivery_result(
    sessions: SessionFactory,
    claimed: ClaimedDelivery,
    result: SendResult,
    *,
    now: datetime,
    max_attempts: int,
    backoff_base: float,
    backoff_max: float,
) -> str:
    """Grava a tentativa e decide o novo estado. Devolve o status final da entrega."""
    if result.ok:
        attempt_result, status = "success", "delivered"
    elif result.permanent or claimed.attempt_number >= max_attempts:
        attempt_result, status = (
            "permanent_error" if result.permanent else "retryable_error",
            "failed",
        )
    else:
        attempt_result, status = "retryable_error", "pending"

    async with sessions.begin() as session:
        session.add(
            DeliveryAttempt(
                delivery_id=claimed.delivery_id,
                attempt_number=claimed.attempt_number,
                attempted_at=now,
                duration_ms=result.duration_ms,
```

```
                status_code=result.status_code,
                result=attempt_result,
                error=result.error,
            )
        )
        values: dict[str, Any] = {"status": status, "updated_at": now, "last_error": result.error}
        if status == "delivered":
            values["delivered_at"] = now
        elif status == "pending":
            delay = backoff_delay(claimed.attempt_number, backoff_base, backoff_max)
            values["next_attempt_at"] = now + timedelta(seconds=delay)
        # Só altera se a entrega ainda estiver reservada por esta tentativa.
        await session.execute(
            update(Delivery)
            .where(
                Delivery.id == claimed.delivery_id,
                Delivery.status == "sending",
                Delivery.attempt_count == claimed.attempt_number,
            )
            .values(**values)
        )
    return status
```

- Linhas 146–152: o resultado do envio.
- Linhas 166–174: decide o desfecho **antes** de abrir a transação: sucesso vira `delivered`; erro permanente ou última tentativa vira `failed`; o resto volta a `pending`.
- Linhas 176–187: grava a tentativa.
- Linhas 188–193: valores a alterar; `pending` ganha o próximo horário com espera exponencial.
- Linhas 194–203: o `UPDATE` condicional (controle otimista; parte F do volume 1). Se a condição não bate, nenhuma linha muda, e a tentativa fica registrada só no histórico.

Falhas possíveis. Banco fora do ar durante a gravação: a exceção sobe para o worker, a entrega fica `sending` e volta à fila quando a reserva vence (pode duplicar, RF06.8).

`src/pulseops/services/processing.py`

Finalidade. O ciclo de um check no banco: reservar, gravar o resultado e aplicar as regras; também a manutenção (checks abandonados e retenção).

Dados. O horário atual (do relógio do worker); `CheckObservation` do verificador; colunas do monitor. **Relações.** Chamado só por `worker/runner.py`; usa `domain/health.py`, `services/incidents.py`. **Testes.** `tests/integration/test_consistency.py`, `test_incident_lifecycle.py`, `test_retention.py`, `test_first_slice.py`.

src/pulseops/services/processing.py, linhas 33–47

```
# Folga além do timeout antes de considerar uma execução 'running' como interrompida.
ABANDON_GRACE = timedelta(seconds=30)


@dataclass(frozen=True)
class ClaimedRun:
    check_id: int
    monitor_id: int
    run_key: str
    config_version: int
    target: CheckTarget


```

```python
def make_run_key(monitor_id: int, scheduled_at: datetime) -> str:
    return f"{monitor_id}:{scheduled_at.isoformat(timespec='milliseconds')}"
```

- Linha 34: folga de 30 s além do timeout antes de considerar um check `running` como abandonado.
- Linhas 37–43: a execução reservada, como objeto simples (imutável), entregue à tarefa do check.
- Linhas 46–47: o `run_key`, com o horário em ISO e milissegundos.

src/pulseops/services/processing.py, linhas 50–77

```python
async def claim_due_runs(
    sessions: SessionFactory, *, now: datetime, limit: int
) -> list[ClaimedRun]:
    """Reserva até `limit` execuções vencidas e grava cada check como 'running'.

    `FOR UPDATE SKIP LOCKED` evita que duas transações peguem o mesmo monitor; o índice
    único parcial de check 'running' impede sobreposição mesmo se algo escapar disso.
    """
    if limit <= 0:
        return []
    running = exists().where(Check.monitor_id == Monitor.id, Check.status == "running")
    async with sessions.begin() as session:
        monitors = (
            await session.scalars(
                select(Monitor)
                .where(Monitor.state == "active", Monitor.next_run_at <= now, ~running)
                .order_by(Monitor.next_run_at, Monitor.id)
                .limit(limit)
                .with_for_update(skip_locked=True)
            )
        ).all()

        claimed: list[ClaimedRun] = []
        for monitor in monitors:
            run = await _reserve(session, monitor, now)
            if run is not None:
                claimed.append(run)
        return claimed
```

- Linhas 58–59: sem vagas, nem consulta o banco.
- Linha 60: o `NOT EXISTS` de check em andamento, montado como expressão reutilizável.
- Linhas 61–70: a consulta com `FOR UPDATE SKIP LOCKED` (SQL na parte F).
- Linhas 72–77: reserva cada monitor; `_reserve` pode devolver `None` (execução já existente).

src/pulseops/services/processing.py, linhas 80–136

```python
async def _reserve(session: AsyncSession, monitor: Monitor, now: datetime) -> ClaimedRun | None:
    if monitor.next_run_at is None:  # o filtro da consulta já exclui; protege a tipagem
        return None
    interval = timedelta(seconds=monitor.interval_seconds)
    scheduled_at = monitor.next_run_at
    trigger = "manual" if monitor.check_requested else "scheduled"
    monitor.check_requested = False

    # Retomada sem avalanche (CB4): atrasado mais de um intervalo → agenda para agora
    # e registra o período sem checks.
    if now - scheduled_at > interval:
        session.add(
            CoverageGap(
                monitor_id=monitor.id,
                gap_start=scheduled_at,
                gap_end=now,
                reason="scheduler_behind",
            )
```

```
 98         )
 99         logger.warning(
100             "lacuna de verificação registrada",
101             extra={"monitor_id": monitor.id, "gap_seconds": (now - scheduled_at).total_seconds()},
102         )
103         scheduled_at = now
104
105     run_key = make_run_key(monitor.id, scheduled_at)
106     check_id = await session.scalar(
107         insert(Check)
108         .values(
109             monitor_id=monitor.id,
110             run_key=run_key,
111             trigger=trigger,
112             status="running",
113             config_version=monitor.config_version,
114             scheduled_at=scheduled_at,
115             started_at=now,
116         )
117         .on_conflict_do_nothing(index_elements=["run_key"])
118         .returning(Check.id)
119     )
120     monitor.next_run_at = max(scheduled_at + interval, now + timedelta(seconds=1))
121     if check_id is None:
122         # Esta execução já existe (reprocessamento da mesma identidade): não duplica.
123         return None
124     return ClaimedRun(
125         check_id=check_id,
126         monitor_id=monitor.id,
127         run_key=run_key,
128         config_version=monitor.config_version,
129         target=CheckTarget(
130             url=monitor.url,
131             method=monitor.method,
132             timeout_seconds=monitor.timeout_seconds,
133             latency_threshold_ms=monitor.latency_threshold_ms,
134             accepted=parse_status_codes(monitor.accepted_status_codes),
135         ),
136     )
```

- Linhas 81–82: proteção para o mypy (a consulta já garante que não é nulo).
- Linhas 85–86: lê e limpa o pedido de “verificar agora”.
- Linhas 88–103: retomada sem avalanche (CB4). Se o atraso passou de um intervalo, registra a lacuna, loga um aviso e agenda para agora.
- Linhas 105–119: o `INSERT ... ON CONFLICT DO NOTHING RETURNING id`.
- Linha 120: o próximo agendamento: agendado + intervalo, mas nunca menos de 1 s no futuro.
- Linhas 121–123: execução já existente: não devolve nada.
- Linhas 124–136: a execução reservada, com o alvo já interpretado.

`src/pulseops/services/processing.py, linhas 139–178`

```
139 async def record_result(
140     sessions: SessionFactory, run: ClaimedRun, observation: CheckObservation
141 ) -> bool:
142     """Grava o resultado e aplica as regras de domínio em uma única transação (CB1).
143
144     Devolve False se esta execução já tinha sido gravada (reprocessamento ignorado).
145     """
146     with log_context(monitor_id=run.monitor_id, run_key=run.run_key):
147         async with sessions.begin() as session:
148             # Trava o monitor: serializa com edição/pausa feitas ao mesmo tempo pela API.
149             monitor = await session.get(Monitor, run.monitor_id, with_for_update=True)
150             check = await session.get(Check, run.check_id, with_for_update=True)
151             if monitor is None or check is None or check.status != "running":
152                 logger.info("resultado ignorado: execução já gravada")
153                 return False
154
```

```
155             check.status = "completed"
156             check.started_at = observation.started_at
157             check.finished_at = observation.finished_at
158             check.duration_ms = observation.duration_ms
159             check.status_code = observation.status_code
160             check.outcome = observation.outcome
161             check.explanation = observation.explanation[:300]
162             monitor.last_check_at = observation.finished_at
163
164             if monitor.state != "active" or monitor.config_version != run.config_version:
165                 # Check iniciado antes de pausa/edição: fica no histórico, mas não decide nada.
166                 logger.info("resultado gravado sem efeito: monitor mudou durante a execução")
167                 return True
168
169             await session.flush()
170             await _apply_domain(session, monitor, check, observation)
171             logger.info(
172                 "check gravado",
173                 extra={
174                     "outcome": observation.outcome.value,
175                     "duration_ms": observation.duration_ms,
176                 },
177             )
178             return True
```

- Linha 146: `log_context` acrescenta monitor e `run_key` a todos os logs do bloco.
- Linhas 149–153: trava monitor e check; se o check já não está `running`, foi gravado antes: devolve `False` (reprocessamento).
- Linhas 155–162: conclui o check com os dados observados (explicação cortada em 300 caracteres, o tamanho da coluna).
- Linhas 164–167: monitor pausado ou com outra versão: o check fica no histórico, mas não decide nada.
- Linha 169: `flush` grava o check antes de aplicar as regras (os eventos apontam para ele).

`src/pulseops/services/processing.py, linhas 181–236`

```
181 async def _apply_domain(
182     session: AsyncSession, monitor: Monitor, check: Check, observation: CheckObservation
183 ) -> None:
184     open_incident = await incidents.get_open_incident(session, monitor.id)
185     state = TrackerState(
186         health=Health(monitor.health),
187         consecutive_failures=monitor.consecutive_failures,
188         consecutive_successes=monitor.consecutive_successes,
189         first_failure_at=monitor.first_failure_at,
190         incident_open=open_incident is not None,
191     )
192     tracker = HealthTracker(Thresholds(monitor.failure_threshold, monitor.recovery_threshold))
193     decision = tracker.apply(
194         state, Observation(observation.outcome, observation.finished_at, observation.explanation)
195     )
196
197     new = decision.state
198     monitor.consecutive_failures = new.consecutive_failures
199     monitor.consecutive_successes = new.consecutive_successes
200     monitor.first_failure_at = new.first_failure_at
201
202     if decision.incident_action is IncidentAction.OPEN:
203         incident = await incidents.open_incident(
204             session,
205             monitor=monitor,
206             check=check,
207             first_failure_at=new.first_failure_at or observation.finished_at,
208             failure_count=new.consecutive_failures,
209             reason=decision.reason,
210         )
211         logger.warning("incidente aberto", extra={"incident_id": incident.id})
```

```
    elif decision.incident_action is IncidentAction.UPDATE and open_incident is not None:
        await incidents.record_failure_on_open_incident(
            session, incident=open_incident, monitor=monitor, check=check
        )
    elif decision.incident_action is IncidentAction.RESOLVE and open_incident is not None:
        await incidents.resolve_incident(
            session, incident=open_incident, monitor=monitor, check=check, reason=decision.reason
        )
        logger.warning("incidente resolvido", extra={"incident_id": open_incident.id})

    if observation.outcome is not Outcome.INTERNAL_ERROR:
        monitor.health_reason = decision.reason
    if decision.health_changed:
        monitor.health = new.health
        monitor.health_changed_at = observation.finished_at
        session.add(
            HealthChange(
                monitor_id=monitor.id,
                changed_at=observation.finished_at,
                from_health=decision.previous_health,
                to_health=new.health,
                reason=decision.reason,
                check_id=check.id,
            )
        )
```

- Linhas 184–195: monta o estado da regra a partir das colunas do monitor e da existência de incidente aberto, e aplica `HealthTracker`.
- Linhas 197–200: grava os contadores novos.
- Linhas 202–220: executa a ação decidida. Os `and open_incident is not None` são defesas: a regra só pede `UPDATE`/`RESOLVE` quando há incidente.
- Linhas 222–223: o motivo da saúde só muda se **não** for erro interno: um erro do PulseOps não apaga a explicação anterior.
- Linhas 224–236: se a saúde mudou, atualiza e registra em `health_changes`, com o check que causou. Teste: `test_every_health_change_is_explained`.

`src/pulseops/services/processing.py, linhas 239–279`

```
async def mark_abandoned_runs(sessions: SessionFactory, *, now: datetime) -> int:
    """Checks 'running' esquecidos (worker caiu entre a requisição e a gravação) viram
    `internal_error`. Isso não conta como falha do alvo (RF02.3, RF03.8)."""
    async with sessions.begin() as session:
        rows = (
            await session.execute(
                select(Check, Monitor.timeout_seconds)
                .join(Monitor, Monitor.id == Check.monitor_id)
                .where(Check.status == "running")
                .with_for_update(of=Check, skip_locked=True)
            )
        ).all()
        count = 0
        for check, timeout in rows:
            if now - check.started_at < timedelta(seconds=timeout) + ABANDON_GRACE:
                continue
            check.status = "completed"
            check.outcome = Outcome.INTERNAL_ERROR
            check.finished_at = now
            check.explanation = "execução interrompida antes de gravar o resultado"
            count += 1
        if count:
            logger.warning(
                "checks interrompidos marcados como erro interno", extra={"count": count}
            )
        return count
```

```python
async def purge_old_checks(
    sessions: SessionFactory, *, now: datetime, retention_days: int, batch_size: int = 5000
) -> int:
    """Retenção (CB6): apaga checks concluídos antigos em lotes. Incidentes ficam."""
    cutoff = now - timedelta(days=retention_days)
    async with sessions.begin() as session:
        ids = (
            select(Check.id)
            .where(Check.started_at < cutoff, Check.status == "completed")
            .limit(batch_size)
        )
        result = await session.execute(delete(Check).where(Check.id.in_(ids.scalar_subquery())))
        return int(getattr(result, "rowcount", 0) or 0)
```

- Linhas 239–264: checks abandonados viram `internal_error`. Linha 248 trava só os checks (`of=Check`) e pula os já travados. Linha 253: só os que passaram do timeout mais a folga. Não mexe em contadores: não é falha do alvo.
- Linhas 267–279: retenção em lotes (parte F). `getattr(result, "rowcount", 0)` lê o número de linhas apagadas de forma segura para a tipagem.

Falhas possíveis. Banco indisponível em qualquer função: exceção para o worker, que registra e tenta de novo com espera crescente. Nenhum resultado é contado pela metade (transação).

`src/pulseops/services/queries.py`

Finalidade. Todas as consultas de leitura: lista de monitores, resumo do painel, histórico paginado, incidentes, alertas, métricas e série do gráfico.

Dados. Parâmetros de janela e paginação vindos das rotas (já validados); devolve objetos do ORM ou *dataclasses* de visualização. **Relações.** Chamado por `web/api.py` e `web/pages.py`. **Testes.** `tests/integration/test_api_history.py`, `test_metrics.py`, `test_dashboard.py`, `test_stopped_worker_makes_data_stale`.

src/pulseops/services/queries.py, linhas 27–58

```python
MAX_PAGE_SIZE = 200
DEFAULT_PAGE_SIZE = 50
STALE_GRACE = timedelta(seconds=30)


# ---- paginação por cursor ------------------------------------------------------


def encode_cursor(moment: datetime, row_id: int) -> str:
    raw = f"{moment.isoformat()}|{row_id}".encode()
    return base64.urlsafe_b64encode(raw).decode().rstrip("=")


def decode_cursor(cursor: str) -> tuple[datetime, int]:
    try:
        padded = cursor + "=" * (-len(cursor) % 4)
        moment_text, id_text = base64.urlsafe_b64decode(padded).decode().split("|")
        moment = datetime.fromisoformat(moment_text)
        if moment.tzinfo is None:
            raise ValueError("cursor sem fuso")
        return moment, int(id_text)
    except (ValueError, UnicodeDecodeError, binascii.Error) as exc:
        raise ValueError("cursor inválido") from exc


@dataclass
class Page[T]:
```

```
    items: list[T]
    next_cursor: str | None


# ---- worker e "dados desatualizados" ------------------------------------------
```

- Linhas 27–29: tamanho máximo e padrão de página; folga de 30 s para “desatualizado”.
- Linhas 35–37: o cursor: `"data|id"` em base64 seguro para URL, sem o `=` final.
- Linhas 40–50: o caminho inverso. Repõe o `=` (linha 42), decodifica, separa, exige data com fuso (linhas 45–46) e transforma qualquer erro em `ValueError("cursor inválido")`, que a rota devolve como 422.
- Linhas 53–56: uma página genérica: itens e o cursor da próxima (ou `None`). `Page[T]` usa a sintaxe de genéricos do Python 3.12.

`src/pulseops/services/queries.py, linhas 61–122`

```
@dataclass(frozen=True)
class WorkerStatus:
    last_seen_at: datetime | None
    age_seconds: float | None
    alive: bool


async def worker_status(session: AsyncSession, *, now: datetime, stale_after: int) -> WorkerStatus:
    last_seen = await session.scalar(select(func.max(WorkerHeartbeat.last_seen_at)))
    if last_seen is None:
        return WorkerStatus(None, None, False)
    age = (now - last_seen).total_seconds()
    return WorkerStatus(last_seen, age, age <= stale_after)


def is_stale(monitor: Monitor, *, now: datetime, worker: WorkerStatus) -> bool:
    """Dados desatualizados (RF03): worker sem sinal ou último check velho demais."""
    if monitor.state != "active":
        return False
    if not worker.alive:
        return True
    reference = monitor.last_check_at or monitor.updated_at
    limit = timedelta(seconds=2 * monitor.interval_seconds + monitor.timeout_seconds) + STALE_GRACE
    return now - reference > limit


# ---- monitores e resumo -------------------------------------------------------


async def list_monitors(session: AsyncSession, *, include_archived: bool = False) -> list[Monitor]:
    query = select(Monitor).order_by(func.lower(Monitor.name), Monitor.id)
    if not include_archived:
        query = query.where(Monitor.state != "archived")
    return list((await session.scalars(query)).all())


@dataclass
class Summary:
    total: int = 0
    by_health: dict[str, int] = field(default_factory=dict)
    stale: int = 0
    open_incidents: int = 0
    pending_deliveries: int = 0
    failed_deliveries: int = 0


async def summary(session: AsyncSession, *, now: datetime, worker: WorkerStatus) -> Summary:
    result = Summary()
    for monitor in await list_monitors(session):
        result.total += 1
        if is_stale(monitor, now=now, worker=worker):
```

```
            result.stale += 1
            continue
        result.by_health[monitor.health] = result.by_health.get(monitor.health, 0) + 1
    result.open_incidents = int(
        await session.scalar(select(func.count()).where(Incident.status == "open")) or 0
    )
    rows = await session.execute(select(Delivery.status, func.count()).group_by(Delivery.status))
    counts: dict[str, int] = {status: count for status, count in rows.tuples()}
    result.pending_deliveries = counts.get("pending", 0) + counts.get("sending", 0)
    result.failed_deliveries = counts.get("failed", 0)
    return result
```

- Linhas 61–73: estado do worker: o heartbeat mais recente, a idade em segundos e se está dentro do limite.
- Linhas 76–85: “desatualizado” (volume 1, H21). Monitor não ativo nunca está desatualizado. Sem check ainda, usa a data da última alteração.
- Linhas 90–94: lista ordenada por nome sem diferenciar maiúsculas.
- Linhas 97–122: o resumo do painel: contagem por saúde (os desatualizados à parte, porque a saúde deles não é confiável), incidentes abertos e entregas pendentes e falhas. A linha 119 é a que corrigiu o bug B2.

`src/pulseops/services/queries.py, linhas 128–185`

```
async def list_checks(
    session: AsyncSession,
    *,
    monitor_id: int,
    limit: int = DEFAULT_PAGE_SIZE,
    cursor: str | None = None,
    start: datetime | None = None,
    end: datetime | None = None,
    outcome: str | None = None,
) -> Page[Check]:
    """Checks concluídos, mais recentes primeiro, ordem estável (started_at, id)."""
    limit = max(1, min(limit, MAX_PAGE_SIZE))
    query = select(Check).where(Check.monitor_id == monitor_id, Check.status == "completed")
    if start is not None:
        query = query.where(Check.started_at >= start)
    if end is not None:
        query = query.where(Check.started_at < end)
    if outcome is not None:
        query = query.where(Check.outcome == outcome)
    if cursor:
        moment, row_id = decode_cursor(cursor)
        query = query.where(tuple_(Check.started_at, Check.id) < tuple_(moment, row_id))
    query = query.order_by(Check.started_at.desc(), Check.id.desc()).limit(limit + 1)
    return _page(list((await session.scalars(query)).all()), limit, lambda c: (c.started_at, c.id))


async def list_incidents(
    session: AsyncSession,
    *,
    status: str | None = None,
    monitor_id: int | None = None,
    limit: int = DEFAULT_PAGE_SIZE,
    cursor: str | None = None,
) -> Page[tuple[Incident, Monitor]]:
    limit = max(1, min(limit, MAX_PAGE_SIZE))
    query: Select[Incident, Monitor] = select(Incident, Monitor).join(
        Monitor, Monitor.id == Incident.monitor_id
    )
    if status == "closed":  # "encerrados" na interface = resolvidos ou fechados
        query = query.where(Incident.status != "open")
    elif status is not None:
        query = query.where(Incident.status == status)
    if monitor_id is not None:
        query = query.where(Incident.monitor_id == monitor_id)
```

```
    if cursor:
        moment, row_id = decode_cursor(cursor)
        query = query.where(tuple_(Incident.confirmed_at, Incident.id) < tuple_(moment, row_id))
    query = query.order_by(Incident.confirmed_at.desc(), Incident.id.desc()).limit(limit + 1)
    rows = [(r[0], r[1]) for r in (await session.execute(query)).all()]
    return _page(rows, limit, lambda r: (r[0].confirmed_at, r[0].id))


def _page[T](rows: list[T], limit: int, key: Any) -> Page[T]:
    if len(rows) > limit:
        rows = rows[:limit]
        moment, row_id = key(rows[-1])
        return Page(rows, encode_cursor(moment, row_id))
    return Page(rows, None)
```

- Linhas 128–151: histórico. Limita o tamanho da página (linha 139), filtra por janela e resultado, aplica o cursor (linhas 147–149) e pede `limit + 1` linhas.
- Linhas 154–177: incidentes com o monitor (`JOIN`). O filtro `closed` da interface significa “encerrados”, isto é, resolvidos **ou** fechados (linha 164).
- Linhas 180–185: se veio mais que o limite, corta e gera o cursor do último item.

`src/pulseops/services/queries.py, linhas 188–275`

```
@dataclass
class DeliveryView:
    delivery: Delivery
    destination: AlertDestination
    attempts: list[DeliveryAttempt]


@dataclass
class AlertView:
    event: AlertEvent
    deliveries: list[DeliveryView]


@dataclass
class IncidentDetail:
    incident: Incident
    monitor: Monitor
    events: list[IncidentEvent]
    alerts: list[AlertView]


async def incident_detail(session: AsyncSession, incident_id: int) -> IncidentDetail | None:
    row = (
        await session.execute(
            select(Incident, Monitor)
            .join(Monitor, Monitor.id == Incident.monitor_id)
            .where(Incident.id == incident_id)
        )
    ).first()
    if row is None:
        return None
    events = (
        await session.scalars(
            select(IncidentEvent)
            .where(IncidentEvent.incident_id == incident_id)
            .order_by(IncidentEvent.occurred_at, IncidentEvent.id)
        )
    ).all()
    alert_events = (
        await session.scalars(
            select(AlertEvent)
            .where(AlertEvent.incident_id == incident_id)
            .order_by(AlertEvent.created_at, AlertEvent.id)
        )
    ).all()
```

```
    return IncidentDetail(
        row[0], row[1], list(events), await _alert_views(session, list(alert_events))
    )


async def list_alerts(session: AsyncSession, *, limit: int = DEFAULT_PAGE_SIZE) -> list[AlertView]:
    limit = max(1, min(limit, MAX_PAGE_SIZE))
    events = (
        await session.scalars(
            select(AlertEvent)
            .order_by(AlertEvent.created_at.desc(), AlertEvent.id.desc())
            .limit(limit)
        )
    ).all()
    return await _alert_views(session, list(events))


async def _alert_views(session: AsyncSession, events: list[AlertEvent]) -> list[AlertView]:
    if not events:
        return []
    rows = (
        await session.execute(
            select(Delivery, AlertDestination)
            .join(AlertDestination, AlertDestination.id == Delivery.destination_id)
            .where(Delivery.event_id.in_([e.id for e in events]))
            .order_by(Delivery.id)
        )
    ).all()
    attempts = (
        await session.scalars(
            select(DeliveryAttempt)
            .where(DeliveryAttempt.delivery_id.in_([r[0].id for r in rows] or [0]))
            .order_by(DeliveryAttempt.delivery_id, DeliveryAttempt.attempt_number)
        )
    ).all()
    by_delivery: dict[int, list[DeliveryAttempt]] = {}
    for attempt in attempts:
        by_delivery.setdefault(attempt.delivery_id, []).append(attempt)
    by_event: dict[int, list[DeliveryView]] = {}
    for delivery, destination in rows:
        view = DeliveryView(delivery, destination, by_delivery.get(delivery.id, []))
        by_event.setdefault(delivery.event_id, []).append(view)
    return [AlertView(e, by_event.get(e.id, [])) for e in events]
```

- Linhas 188–206: *dataclasses* de visualização (entrega com destino e tentativas; evento com entregas; detalhe do incidente).
- Linhas 209–235: detalhe do incidente: o incidente com o monitor, os eventos em ordem e os alertas.
- Linhas 238–247: os alertas mais recentes.
- Linhas 250–275: monta as visões com **três** consultas no total (eventos, entregas, tentativas), em vez de uma consulta por entrega (o problema "N+1"). A linha 264 usa `or [0]` porque `IN ()` vazio não é SQL válido. As linhas 268–274 agrupam em dicionários com `setdefault`.

`src/pulseops/services/queries.py, linhas 278–313`

```
async def health_changes(
    session: AsyncSession, monitor_id: int, *, limit: int = 20
) -> list[HealthChange]:
    return list(
        (
            await session.scalars(
                select(HealthChange)
                .where(HealthChange.monitor_id == monitor_id)
                .order_by(HealthChange.changed_at.desc(), HealthChange.id.desc())
                .limit(limit)
            )
        ).all()
```

```
290     )
291
292
293 # ---- métricas ----------------------------------------------------------------
294
295 RESPONSE_OUTCOMES = ("healthy", "slow", "unexpected_status")
296
297
298 @dataclass
299 class Metrics:
300     start: datetime
301     end: datetime
302     evaluated: int  # N: checks na janela, sem erros internos
303     internal_errors: int
304     by_outcome: dict[str, int]
305     available_pct: float | None
306     healthy_pct: float | None
307     latency_samples: int
308     latency_avg_ms: float | None
309     latency_p95_ms: float | None
310     incident_count: int
311     incident_duration: timedelta
312     last_check_at: datetime | None
313     gaps: list[CoverageGap]
```

- Linhas 278–290: as últimas mudanças de saúde de um monitor.
- Linha 295: `RESPONSE_OUTCOMES`, os resultados que têm tempo de resposta HTTP: saudável, lento e status inesperado. É a base da latência média, do p95 e, desde a correção do bug B10, também do resumo do gráfico.
- Linhas 298–313: o resultado das métricas, com `None` onde não há base.

`src/pulseops/services/queries.py, linhas 316–421`

```
316 async def monitor_metrics(
317     session: AsyncSession, monitor_id: int, *, start: datetime, end: datetime, now: datetime
318 ) -> Metrics:
319     """Fórmulas em docs/requisitos.md (RF05). Janela [start, end)."""
320     in_window = (
321         Check.monitor_id == monitor_id,
322         Check.status == "completed",
323         Check.started_at >= start,
324         Check.started_at < end,
325     )
326     rows = await session.execute(
327         select(Check.outcome, func.count()).where(*in_window).group_by(Check.outcome)
328     )
329     by_outcome: dict[str, int] = {outcome or "": count for outcome, count in rows.tuples()}
330     internal = by_outcome.get("internal_error", 0)
331     evaluated = sum(by_outcome.values()) - internal
332     available = by_outcome.get("healthy", 0) + by_outcome.get("slow", 0)
333
334     # percentile_disc(0.95) devolve o primeiro valor com distribuição acumulada >= 0,95:
335     # é o mesmo resultado do nearest-rank (ADR 0011).
336     latency = (
337         await session.execute(
338             select(
339                 func.count(Check.duration_ms),
340                 func.avg(Check.duration_ms),
341                 func.percentile_disc(0.95).within_group(Check.duration_ms),
342             ).where(*in_window, Check.outcome.in_(RESPONSE_OUTCOMES))
343         )
344     ).one()
345     samples = int(latency[0])
346
347     incidents = (
348         await session.scalars(
349             select(Incident).where(
350                 Incident.monitor_id == monitor_id,
```

```
                Incident.started_at < end,
                (Incident.ended_at.is_(None)) | (Incident.ended_at > start),
            )
        )
    ).all()
    duration = sum(
        (clipped_duration(start, end, i.started_at, i.ended_at, now) for i in incidents),
        timedelta(0),
    )

    last_check_at = await session.scalar(
        select(func.max(Check.finished_at)).where(
            Check.monitor_id == monitor_id, Check.status == "completed"
        )
    )
    gaps = (
        await session.scalars(
            select(CoverageGap)
            .where(
                CoverageGap.monitor_id == monitor_id,
                CoverageGap.gap_start < end,
                CoverageGap.gap_end > start,
            )
            .order_by(CoverageGap.gap_start)
        )
    ).all()

    return Metrics(
        start=start,
        end=end,
        evaluated=evaluated,
        internal_errors=internal,
        by_outcome=by_outcome,
        available_pct=ratio(available, evaluated),
        healthy_pct=ratio(by_outcome.get("healthy", 0), evaluated),
        latency_samples=samples,
        latency_avg_ms=float(latency[1]) if samples else None,
        latency_p95_ms=float(latency[2]) if samples else None,
        incident_count=len(incidents),
        incident_duration=duration,
        last_check_at=last_check_at,
        gaps=list(gaps),
    )


@dataclass(frozen=True)
class LatencyPoint:
    at: datetime
    duration_ms: int
    outcome: str


async def latency_series(
    session: AsyncSession, monitor_id: int, *, start: datetime, end: datetime, limit: int = 300
) -> list[LatencyPoint]:
    """Pontos do gráfico: os `limit` checks mais recentes da janela, em ordem cronológica."""
    rows = (
        await session.execute(
            select(Check.started_at, Check.duration_ms, Check.outcome)
            .where(
                Check.monitor_id == monitor_id,
                Check.status == "completed",
                Check.started_at >= start,
                Check.started_at < end,
                Check.outcome != "internal_error",
            )
            .order_by(Check.started_at.desc(), Check.id.desc())
            .limit(limit)
        )
    ).all()
    return [LatencyPoint(r[0], r[1] or 0, r[2] or "") for r in reversed(rows)]
```

- Linhas 320–325: a janela como tupla de condições, reutilizada nas consultas.
- Linhas 326–331: contagem por resultado (`GROUP BY`; SQL na parte F).
- Linhas 332–334: `N` exclui erros internos; disponíveis são saudáveis mais lentos.
- Linhas 336–346: contagem, média e p95 numa consulta só.
- Linhas 348–359: incidentes que **se sobrepõem** à janela (começaram antes do fim e não terminaram antes do início) e a soma das durações recortadas.
- Linhas 361–377: último check e lacunas que se sobrepõem à janela.
- Linhas 403–421: a série do gráfico: os 300 checks mais recentes da janela (sem erros internos), invertidos para ordem cronológica. `r[1] or 0` cobre duração nula.

Falhas possíveis. Cursor inválido: `ValueError`, 422 na API. Janela muito grande: consultas mais lentas, mas limitadas pelos índices e pelo `limit`.

D7 — Worker: o processo que verifica e avisa

`src/pulseops/worker/runner.py`

Finalidade. A classe `Worker`: três laços (checks, entregas, manutenção) no mesmo processo, em tarefas `asyncio` separadas, com limite de checks simultâneos.

Dados. Recebe sessões, verificador, notificador, relógio e configuração. Produz linhas no banco (via serviços) e logs. **Relações.** Montado em `worker/__main__.py`; usa `services/processing.py` e `services/alerts.py`. **Testes.** `tests/integration/test_consistency.py` (`test_worker_loop_survives_unreachable_database`, `test_dispatcher_refills_freed_slots_without_waiting_a_tick`), `test_slow_webhook_does_not_block_checks`, `test_maintenance_runs_retention` e quase todos os testes de integração, que usam `run_checks_once` e `run_deliveries_once`.

src/pulseops/worker/runner.py, linhas 1–30

```
"""Worker: agenda verificações, entrega alertas e faz manutenção (ADR 0002).

Os três laços rodam no mesmo processo, mas em tarefas `asyncio` separadas. Assim
um webhook lento não atrasa nenhuma verificação (RF06.3).
"""

import asyncio
import contextlib
import logging
import os
import socket
from collections.abc import Awaitable, Callable
from datetime import timedelta

from sqlalchemy.dialects.postgresql import insert

from pulseops.checks.checker import Checker, CheckObservation
from pulseops.clock import Clock
from pulseops.config import Settings
from pulseops.db.models import WorkerHeartbeat
from pulseops.db.session import SessionFactory
from pulseops.domain.outcomes import Outcome
from pulseops.logs import log_context
from pulseops.notifications.webhook import Notifier
from pulseops.services import alerts, processing

logger = logging.getLogger(__name__)

DELIVERY_BATCH = 10
RETENTION_EVERY = timedelta(hours=1)
```

- Linhas 1–5: a docstring dá a razão dos três laços separados: um webhook lento não atrasa verificação nenhuma (RF06.3).
- Linha 29: até 10 entregas por rodada.
- Linha 30: a retenção roda no máximo uma vez por hora.

src/pulseops/worker/runner.py, linhas 33–54

```
class Worker:
    def __init__(
        self,
        *,
        sessions: SessionFactory,
```

```
        checker: Checker,
        notifier: Notifier,
        clock: Clock,
        settings: Settings,
        worker_id: str | None = None,
    ) -> None:
        self._sessions = sessions
        self._checker = checker
        self._notifier = notifier
        self._clock = clock
        self._settings = settings
        self.worker_id = worker_id or f"{socket.gethostname()}:{os.getpid()}"
        self._started_at = clock.now()
        self._inflight: set[asyncio.Task[None]] = set()
        # Sinalizado sempre que um check termina e libera uma vaga (ver _wait_after_dispatch).
        self._slot_freed = asyncio.Event()
        self._last_retention = self._started_at - RETENTION_EVERY
```

- Linhas 34–43: tudo é recebido pronto, com argumentos só nomeados (`*`).
- Linha 49: identificação `hostname:pid`, usada no heartbeat.
- Linha 51: o conjunto das tarefas de check em andamento. O tamanho dele é o número de vagas ocupadas.
- Linha 53: o evento “uma vaga abriu” (correção do bug B8).
- Linha 54: começa “uma hora atrás”, para a retenção rodar logo na primeira manutenção.

`src/pulseops/worker/runner.py, linhas 58–100`

```
    async def dispatch_checks(self) -> int:
        """Reserva execuções para as vagas livres e dispara cada uma em uma tarefa.

        O número de tarefas em andamento nunca passa de `worker_concurrency`: esse é o
        limite de concorrência das verificações.
        """
        claimed, _ = await self._dispatch()
        return claimed

    async def _dispatch(self) -> tuple[int, int]:
        """Devolve (reservadas, vagas pedidas)."""
        free = self._settings.worker_concurrency - len(self._inflight)
        runs = await processing.claim_due_runs(self._sessions, now=self._clock.now(), limit=free)
        for run in runs:
            task = asyncio.create_task(self._run_one(run), name=f"check:{run.run_key}")
            self._inflight.add(task)
            task.add_done_callback(self._release_slot)
        return len(runs), free

    def _release_slot(self, task: asyncio.Task[None]) -> None:
        self._inflight.discard(task)
        self._slot_freed.set()

    async def _wait_after_dispatch(self, claimed: int, requested: int) -> None:
        """Se todas as vagas pedidas foram preenchidas, pode haver fila: volta assim que
        uma vaga abrir, em vez de dormir o tick inteiro (bug B8). Se vieram menos
        execuções que vagas, nada mais está vencido e o tick normal basta."""
        tick = self._settings.worker_tick_seconds
        if claimed < requested:
            await asyncio.sleep(tick)
            return
        with contextlib.suppress(TimeoutError):
            await asyncio.wait_for(self._slot_freed.wait(), timeout=tick)

    async def wait_inflight(self) -> None:
        if self._inflight:
            await asyncio.gather(*self._inflight, return_exceptions=True)

    async def run_checks_once(self) -> int:
        """Usado pelos testes: reserva, executa e grava tudo que estiver vencido agora."""
        count = await self.dispatch_checks()
```

```
 99        await self.wait_inflight()
100        return count
```

- Linhas 58–65: interface pública que devolve só quantas foram reservadas.
- Linhas 67–75: calcula as vagas livres, reserva até esse número e cria uma tarefa por execução. `add_done_callback` registra uma função chamada quando a tarefa termina, com sucesso ou erro.
- Linhas 77–79: ao terminar, a tarefa sai do conjunto e o evento é sinalizado.
- Linhas 81–90: a espera depois de cada rodada (volume 1, H29). Se reservou menos que as vagas pedidas, não há fila: dorme o tick. Se preencheu tudo, espera uma vaga abrir, com o tick como limite (`wait_for` com `timeout`; `contextlib.suppress(TimeoutError)` ignora o estouro, que é normal aqui).
- Linhas 92–94: espera as tarefas em andamento; `return_exceptions=True` evita que a falha de uma interrompa a espera das outras.
- Linhas 96–100: usado nos testes: reserva, executa e espera tudo.

`src/pulseops/worker/runner.py, linhas 102–122`

```
102    async def _run_one(self, run: processing.ClaimedRun) -> None:
103        with log_context(monitor_id=run.monitor_id, run_key=run.run_key):
104            try:
105                observation = await self._checker.check(run.target)
106            except Exception as exc:  # o verificador já trata erros; isto é a última barreira
107                logger.exception("erro inesperado no verificador")
108                now = self._clock.now()
109                observation = CheckObservation(
110                    Outcome.INTERNAL_ERROR,
111                    now,
112                    now,
113                    0,
114                    None,
115                    f"erro interno ({type(exc).__name__})",
116                )
117            try:
118                await processing.record_result(self._sessions, run, observation)
119            except Exception:
120                # Banco fora do ar, por exemplo. O check fica 'running' e a manutenção o marca
121                # como interrompido depois; nada é contado como falha do alvo.
122                logger.exception("falha ao gravar resultado do check")
```

- Linha 103: contexto de log com monitor e `run_key`.
- Linhas 104–116: o verificador já trata os próprios erros; este `except` é a última barreira. Uma exceção vira observação de erro interno, não falha do alvo. `logger.exception` registra o *traceback*.
- Linhas 117–122: se a gravação falhar (banco fora do ar), só registra. O check fica `running`; a manutenção o marca como interrompido depois.

`src/pulseops/worker/runner.py, linhas 126–159`

```
126    async def run_deliveries_once(self) -> int:
127        s = self._settings
128        claimed = await alerts.claim_due_deliveries(
129            self._sessions,
130            now=self._clock.now(),
131            limit=DELIVERY_BATCH,
132            lease_seconds=s.webhook_timeout_seconds + 30,
133        )
134        await asyncio.gather(*(self._deliver(item) for item in claimed))
135        return len(claimed)
136
137    async def _deliver(self, item: alerts.ClaimedDelivery) -> None:
138        s = self._settings
```

```
139         with log_context(delivery_id=item.delivery_id, event_id=item.event_key):
140             result = await self._notifier.send(item.url, item.event_key, item.payload)
141             try:
142                 status = await alerts.record_delivery_result(
143                     self._sessions,
144                     item,
145                     result,
146                     now=self._clock.now(),
147                     max_attempts=s.webhook_max_attempts,
148                     backoff_base=s.webhook_backoff_base_seconds,
149                     backoff_max=s.webhook_backoff_max_seconds,
150                 )
151             except Exception:
152                 logger.exception("falha ao gravar tentativa de entrega")
153                 return
154             level = logging.INFO if result.ok else logging.WARNING
155             logger.log(
156                 level,
157                 "tentativa de entrega",
158                 extra={"attempt": item.attempt_number, "status": status, "error": result.error},
159             )
```

- Linhas 126–135: reserva até 10 entregas, com reserva valendo o timeout do webhook mais 30 s, e envia todas **em paralelo** (`gather`).
- Linhas 137–159: envia, grava o resultado e loga a tentativa (INFO se deu certo, WARNING se não).

`src/pulseops/worker/runner.py, linhas 163–188`

```
163     async def heartbeat(self) -> None:
164         now = self._clock.now()
165         statement = insert(WorkerHeartbeat).values(
166             worker_id=self.worker_id,
167             hostname=socket.gethostname(),
168             pid=os.getpid(),
169             started_at=self._started_at,
170             last_seen_at=now,
171         )
172         statement = statement.on_conflict_do_update(
173             index_elements=["worker_id"], set_={"last_seen_at": now}
174         )
175         async with self._sessions.begin() as session:
176             await session.execute(statement)
177 
178     async def run_maintenance_once(self) -> None:
179         await self.heartbeat()
180         now = self._clock.now()
181         await processing.mark_abandoned_runs(self._sessions, now=now)
182         if now - self._last_retention >= RETENTION_EVERY:
183             removed = await processing.purge_old_checks(
184                 self._sessions, now=now, retention_days=self._settings.check_retention_days
185             )
186             self._last_retention = now
187             if removed:
188                 logger.info("retenção: checks antigos removidos", extra={"count": removed})
```

- Linhas 163–176: heartbeat com "upsert": `INSERT ... ON CONFLICT (worker_id) DO UPDATE SET last_seen_at = ...`. A primeira vez insere; as seguintes atualizam.
- Linhas 178–188: a manutenção: heartbeat, checks abandonados e, a cada hora, retenção.

`src/pulseops/worker/runner.py, linhas 192–245`

```
192     async def run(self, stop: asyncio.Event) -> None:
193         s = self._settings
194         logger.info("worker iniciado", extra={"worker_id": self.worker_id})
195         loops = [
```

```
196             self._check_loop(stop),
197             self._loop("deliveries", self.run_deliveries_once, s.worker_tick_seconds, stop),
198             self._loop(
199                 "maintenance", self.run_maintenance_once, s.heartbeat_interval_seconds, stop
200             ),
201         ]
202         await asyncio.gather(*loops)
203         logger.info("aguardando checks em andamento terminarem")
204         await self.wait_inflight()
205 
206     async def _check_loop(self, stop: asyncio.Event) -> None:
207         failures = 0
208         while not stop.is_set():
209             self._slot_freed.clear()
210             try:
211                 claimed, requested = await self._dispatch()
212                 failures = 0
213             except Exception:
214                 failures += 1
215                 delay = min(self._settings.worker_tick_seconds * 2**failures, 30.0)
216                 logger.exception(
217                     "erro no laço do worker", extra={"loop": "checks", "retry_in": delay}
218                 )
219                 with contextlib.suppress(TimeoutError):
220                     await asyncio.wait_for(stop.wait(), timeout=delay)
221                 continue
222             waiter = asyncio.create_task(self._wait_after_dispatch(claimed, requested))
223             stopper = asyncio.create_task(stop.wait())
224             await asyncio.wait({waiter, stopper}, return_when=asyncio.FIRST_COMPLETED)
225             waiter.cancel()
226             stopper.cancel()
227 
228     async def _loop(
229         self, name: str, step: Callable[[], Awaitable[object]], every: float, stop: asyncio.Event
230     ) -> None:
231         failures = 0
232         while not stop.is_set():
233             try:
234                 await step()
235                 failures = 0
236                 delay = every
237             except Exception:
238                 # Banco indisponível, por exemplo: registra e tenta de novo com espera crescente,
239                 # sem derrubar o processo e sem afetar os outros laços.
240                 failures += 1
241                 delay = min(every * 2**failures, 30.0)
242                 logger.exception("erro no laço do worker", extra={"loop": name, "retry_in": delay})
243             # Espera o intervalo, mas acorda na hora se o processo receber sinal de parada.
244             with contextlib.suppress(TimeoutError):
245                 await asyncio.wait_for(stop.wait(), timeout=delay)
```

- Linhas 192–204: `run` executa os três laços até o evento `stop`. Depois de parar, espera os checks em andamento terminarem (desligamento gracioso: o resultado é gravado em vez de ficar `running`).
- Linhas 206–226: o laço de checks. Linha 209 limpa o evento antes de reservar. Linhas 213–221: erro (banco fora do ar) espera `tick × 2^falhas`, até 30 s, mas acorda na hora se receber `stop`. Linhas 222–226: espera o que vier primeiro, o fim da espera normal ou o sinal de parada, e cancela a outra tarefa.
- Linhas 228–245: o laço genérico das entregas e da manutenção, com o mesmo tratamento de erro. A espera usa `wait_for(stop.wait(), timeout=delay)`: dorme `delay`, mas acorda antes se `stop` for sinalizado.

Falhas possíveis. Banco indisponível: os laços registram, esperam e tentam de novo (`test_worker_loop_survives_unreachable_database`). Um bug que lance exceção fora desses `try` derrubaria o processo; o Compose o reinicia (`restart: unless-stopped`).

`src/pulseops/worker/__main__.py`

Finalidade. Ponto de entrada do processo (`python -m pulseops.worker`): monta as peças, garante um worker só por banco e trata os sinais de parada.

Dados. Configuração do ambiente. **Relações.** É o comando do serviço `worker` no `compose.yaml`. **Testes.** `try_acquire_worker_lock` em `test_second_worker_is_refused`; o resto é exercitado pela demonstração e pelo job `docker` do CI (a cobertura deste arquivo nos testes é baixa, 44%; volume 1, parte G).

src/pulseops/worker/**main**.py, linhas 24–49

```
24 # Chave do advisory lock que garante um único worker por banco (ADR 0002).
25 WORKER_LOCK_KEY = 0x50554C53  # "PULS"
26
27
28 async def try_acquire_worker_lock(conn: AsyncConnection) -> bool:
29     """Lock de sessão do PostgreSQL: só uma conexão por banco consegue segurá-lo."""
30     got = await conn.scalar(text("SELECT pg_try_advisory_lock(:k)"), {"k": WORKER_LOCK_KEY})
31     await conn.commit()  # o lock é de sessão: continua valendo depois do commit
32     return bool(got)
33
34
35 async def main() -> int:
36     settings = Settings()
37     configure_logging(settings.log_level)
38     engine = create_engine(settings.database_url, pool_size=settings.worker_concurrency + 5)
39     sessions = create_session_factory(engine)
40     clock = SystemClock()
41     policy = DestinationPolicy(settings.allowed_port_set, settings.demo_allowlist)
42     user_agent = f"PulseOps/{__version__} (+monitoramento)"
43
44     # Conexão dedicada que segura o lock enquanto o processo vive.
45     async with engine.connect() as lock_conn:
46         if not await try_acquire_worker_lock(lock_conn):
47             logger.error("outro worker já está rodando neste banco; apenas um é suportado")
48             await engine.dispose()
49             return 1
```

- Linhas 24–25: a chave do advisory lock: os bytes de “PULS” como número.
- Linhas 28–32: tenta o lock (volume 1, parte F). O `commit` da linha 31 fecha a transação aberta pelo `SELECT`; o lock de sessão continua.
- Linhas 35–41: configuração, logs, engine com conexões suficientes (uma por check simultâneo, mais 5), sessões, relógio, política e `User-Agent`.
- Linhas 45–49: uma conexão dedicada segura o lock enquanto o processo vive (o `async with` só termina no fim). Se outro worker já tem o lock, registra o erro e sai com código 1.

src/pulseops/worker/**main**.py, linhas 51–80

```
51         check_client = build_guarded_client(
52             policy, max_connections=settings.worker_concurrency, user_agent=user_agent
53         )
54         webhook_client = build_guarded_client(policy, max_connections=10, user_agent=user_agent)
55         worker = Worker(
56             sessions=sessions,
57             checker=HttpChecker(check_client, clock),
```

```
58              notifier=WebhookNotifier(
59                  webhook_client, clock, policy, settings.webhook_timeout_seconds
60              ),
61              clock=clock,
62              settings=settings,
63          )
64
65          stop = asyncio.Event()
66          loop = asyncio.get_running_loop()
67          for sig in (signal.SIGINT, signal.SIGTERM):
68              loop.add_signal_handler(sig, stop.set)
69          try:
70              await worker.run(stop)
71          finally:
72              await check_client.aclose()
73              await webhook_client.aclose()
74      await engine.dispose()
75      logger.info("worker encerrado")
76      return 0
77
78
79  if __name__ == "__main__":
80      sys.exit(asyncio.run(main()))
```

- Linhas 51–54: dois clientes HTTP protegidos: um para checks (tantas conexões quanto checks simultâneos) e outro para webhooks (10).
- Linhas 55–63: a composição de tudo.
- Linhas 65–68: `SIGINT` (Ctrl+C) e `SIGTERM` (`docker compose stop`) sinalizam o evento `stop`, em vez de matar o processo no meio de uma gravação.
- Linhas 69–73: roda até parar e fecha os clientes em qualquer caso.
- Linhas 74–76: fecha o engine (libera o lock) e sai com 0.
- Linhas 79–80: `asyncio.run(main())` e o código de saída para o sistema.

Falhas possíveis. Banco inacessível na inicialização: a conexão do lock falha e o processo sai com erro; o Compose tenta de novo. Segundo worker: sai com código 1 e a mensagem “outro worker já está rodando neste banco; apenas um é suportado”.

D8 — Web: API JSON, painel e login

A camada web é o processo `api`. Ela não verifica nada: lê o que o worker gravou, aceita cadastros e mostra tudo em HTML ou JSON. As regras ficam nos serviços (D6); as rotas validam a entrada, chamam um serviço, fazem `commit` e escolhem a resposta.

`src/pulseops/web/__init__.py`

Arquivo vazio (0 linhas). Só marca a pasta como pacote Python, para que `from pulseops.web import api` funcione. Não há o que explicar linha a linha.

`src/pulseops/web/app.py`

Finalidade. A fábrica `create_app`: monta a aplicação FastAPI, liga as rotas, define o tratamento de erros e acrescenta os cabeçalhos de segurança a toda resposta.

Dados. Recebe `Settings` e, opcionalmente, sessões, relógio e resolvedor (os testes passam versões próprias). Produz um objeto `FastAPI`. **Relações.** É chamada pelo comando da imagem (`uvicorn pulseops.web.app:create_app --factory`, no `Dockerfile`, ver D10) e por `tests/conftest.py`. Usa `deps.py`, `api.py`, `pages.py`, `health.py` e `templating.py`. **Testes.** `test_security_headers`, `test_not_found_page_is_html` e `test_pages_redirect_to_login`, além de todo teste de integração que usa o cliente HTTP da aplicação.

src/pulseops/web/app.py, linhas 30–57

```
30 def create_app(
31     settings: Settings | None = None,
32     *,
33     sessions: SessionFactory | None = None,
34     clock: Clock | None = None,
35     resolver: Resolver | None = None,
36 ) -> FastAPI:
37     """Os testes passam `sessions`, `clock` e `resolver`; em produção tudo vem das variáveis."""
38     settings = settings or Settings()
39
40     @asynccontextmanager
41     async def lifespan(app: FastAPI) -> AsyncIterator[None]:
42         engine = None
43         factory = sessions
44         if factory is None:
45             configure_logging(settings.log_level)
46             engine = create_engine(settings.database_url)
47             factory = create_session_factory(engine)
48         app.state.pulseops = AppState(
49             settings=settings,
50             sessions=factory,
51             clock=clock or SystemClock(),
52             policy=DestinationPolicy(settings.allowed_port_set, settings.demo_allowlist),
53             resolver=resolver,
54         )
55         yield
56         if engine is not None:
57             await engine.dispose()
```

- Linhas 30–36: o `*` obriga a passar `sessions`, `clock` e `resolver` pelo nome. É **injeção de dependência** feita à mão: quem chama decide as peças.

- Linhas 40–57: o *lifespan*, código que roda uma vez ao subir (antes do `yield`) e outra ao desligar (depois). Linhas 44–47: em produção, cria o engine e as sessões; nos testes, usa as sessões recebidas e não cria engine. Linhas 48–54: guarda tudo num `AppState` (definido em `deps.py`) dentro de `app.state`. Linhas 56–57: fecha o engine só se foi criado aqui.

src/pulseops/web/app.py, linhas 59–75

```
59    app = FastAPI(
60        title="PulseOps",
61        version=__version__,
62        lifespan=lifespan,
63        # A documentação interativa fica desligada: a API é de uso do próprio operador.
64        docs_url=None,
65        redoc_url=None,
66        openapi_url=None,
67    )
68    app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
69    app.include_router(health.router)
70    app.include_router(api.router)
71    app.include_router(pages.router)
72
73    @app.exception_handler(LoginRequiredError)
74    async def login_redirect(request: Request, exc: LoginRequiredError) -> Response:
75        return RedirectResponse(f"/login?next={quote(exc.next_path)}", status_code=303)
```

- Linhas 59–67: a documentação interativa (`/docs`, `/redoc`, `/openapi.json`) fica desligada. A API é do próprio operador, e expor o esquema não traria nada além de um mapa das rotas.
- Linha 68: arquivos estáticos (`style.css`, `app.js`) em `/static`.
- Linhas 69–71: as três coleções de rotas. A ordem importa pouco, porque os prefixos não se sobrepõem (`/health`, `/api/v1` e o resto).
- Linhas 73–75: quando uma página exige login e não há sessão, `deps.py` lança `LoginRequiredError`; aqui ela vira um redirecionamento `303` para `/login?next=<caminho>`. `quote` escapa o caminho para caber na URL.

src/pulseops/web/app.py, linhas 77–114

```
77    @app.exception_handler(StarletteHTTPException)
78    async def http_error(request: Request, exc: StarletteHTTPException) -> Response:
79        """Na API, JSON; nas páginas, uma página de erro legível (sem detalhes internos)."""
80        if request.url.path.startswith(("/api/", "/health/")):
81            return JSONResponse(
82                {"detail": exc.detail}, status_code=exc.status_code, headers=exc.headers
83            )
84        titles = {403: "Acesso negado", 404: "Página não encontrada", 409: "Operação não permitida"}
85        return templates.TemplateResponse(
86            request,
87            "error.html",
88            {
89                "title": titles.get(exc.status_code, "Erro"),
90                "message": str(exc.detail),
91                "tz": settings.timezone,
92                "tz_name": settings.display_timezone,
93                "demo_mode": settings.demo_mode,
94            },
95            status_code=exc.status_code,
96        )
97
98    @app.exception_handler(DBAPIError)
99    async def database_unavailable(request: Request, exc: DBAPIError) -> Response:
100       logger.error("erro de banco ao atender requisição", extra={"path": request.url.path})
101       if request.url.path.startswith("/api/"):
102           return JSONResponse({"detail": "banco de dados indisponível"}, status_code=503)
103       return templates.TemplateResponse(
104           request,
```

```
105             "error.html",
106             {
107                 "title": "Banco de dados indisponível",
108                 "message": "Tente novamente em instantes.",
109                 "tz": settings.timezone,
110                 "tz_name": settings.display_timezone,
111                 "demo_mode": settings.demo_mode,
112             },
113             status_code=503,
114         )
```

- Linhas 77–96: erros HTTP (404, 403, 409…). Na API e no `/health`, JSON no formato padrão do FastAPI (`{"detail": ...}`). Nas páginas, o template `error.html` com um título em português. A mensagem é a do próprio erro, que o código escreveu; nada de *traceback*.
- Linhas 98–114: `DBAPIError` é a exceção do SQLAlchemy para falhas do driver (banco fora do ar, conexão perdida). Registra no log e responde 503, em JSON na API ou numa página "Banco de dados indisponível". Sem isso, o usuário veria um 500 genérico.

`src/pulseops/web/app.py, linhas 116–129`

```
116     @app.middleware("http")
117     async def security_headers(request: Request, call_next):  # type: ignore[no-untyped-def]
118         response: Response = await call_next(request)
119         response.headers.setdefault("X-Content-Type-Options", "nosniff")
120         response.headers.setdefault("X-Frame-Options", "DENY")
121         response.headers.setdefault("Referrer-Policy", "same-origin")
122         response.headers.setdefault(
123             "Content-Security-Policy",
124             "default-src 'self'; img-src 'self' data:; style-src 'self'; script-src 'self'; "
125             "frame-ancestors 'none'; form-action 'self'",
126         )
127         return response
128
129     return app
```

- Linha 116: um *middleware* roda em volta de toda requisição. `call_next` chama a rota; depois disso, a resposta pode ser alterada.
- Linha 117: o `type: ignore` existe porque o FastAPI não oferece um tipo exato para `call_next`; é o único `type: ignore` desta camada.
- Linhas 119–121: `nosniff` impede o navegador de "adivinhar" o tipo de um arquivo; `X-Frame-Options: DENY` impede que o painel seja exibido dentro de um `<iframe>` de outro site (*clickjacking*); `Referrer-Policy: same-origin` não envia a URL do painel a outros sites.
- Linhas 122–126: a *Content Security Policy* (CSP). Só carrega scripts, estilos e imagens do próprio servidor (`'self'`); `data:` é permitido em imagens. Não há `unsafe-inline`: um `<script>` injetado no HTML **não** rodaria, mesmo que o escape falhasse. É a segunda camada contra XSS, depois do `autoescape` do Jinja.
- `setdefault` não sobrescreve um cabeçalho que a rota já tenha definido.

Falhas possíveis. Um erro sem tratador (por exemplo, um `KeyError` num bug) vira 500 do próprio Starlette, sem detalhes para o usuário; o log registra.

`src/pulseops/web/deps.py`

Finalidade. As **dependências** do FastAPI: funções que as rotas declaram nos parâmetros e que o framework executa antes da rota, como "abra uma sessão de banco" ou "exija login".

Dados. Lê o cookie de sessão e cabeçalhos. Produz `AppState`, `AsyncSession` e `AuthContext`. **Relações.** Usado por `api.py`, `pages.py` e `health.py`; chama `services/auth.py`. **Testes.** `tests/integration/test_auth.py` inteiro (16 funções, 26 casos), em especial `test_api_mutation_requires_csrf_header`, `test_form_action_requires_csrf_field` e `test_session_expires`.

src/pulseops/web/deps.py, linhas 18–47

```python
SESSION_COOKIE = "pulseops_session"
LOGIN_CSRF_COOKIE = "pulseops_login_csrf"
UNSAFE_METHODS = frozenset({"POST", "PUT", "PATCH", "DELETE"})


@dataclass
class AppState:
    settings: Settings
    sessions: SessionFactory
    clock: Clock
    policy: DestinationPolicy
    resolver: Resolver | None = None


def get_state(request: Request) -> AppState:
    state: AppState = request.app.state.pulseops
    return state


State = Annotated[AppState, Depends(get_state)]


async def get_db(state: State) -> AsyncIterator[AsyncSession]:
    """Uma sessão por requisição. As rotas que alteram dados chamam commit() explicitamente;
    o que não foi confirmado é desfeito quando a sessão fecha."""
    async with state.sessions() as session:
        yield session


Db = Annotated[AsyncSession, Depends(get_db)]
```

- Linhas 18–19: nomes dos dois cookies. `pulseops_session` guarda o token de sessão; `pulseops_login_csrf` só existe entre abrir a página de login e enviar o formulário.
- Linha 20: os métodos que alteram dados. Só eles exigem token CSRF; `GET` não deve alterar nada.
- Linhas 23–29: o pacote de objetos compartilhados pela aplicação inteira.
- Linhas 32–37: `Annotated[AppState, Depends(get_state)]` é um **apelido de tipo**: uma rota que declara `state: State` recebe o `AppState` sem repetir o `Depends`.
- Linhas 40–47: uma sessão de banco por requisição. A docstring é a regra da camada: quem altera dados chama `commit()`; o que não foi confirmado é desfeito ao fechar.

src/pulseops/web/deps.py, linhas 50–77

```python
@dataclass(frozen=True)
class AuthContext:
    """Valores copiados da sessão no início da requisição.

    Não guarda objetos do ORM: se a rota desfizer a transação (rollback), objetos
    carregados expiram e lê-los de novo no template exigiria I/O fora de contexto
    assíncrono (bug B7).
    """

    operator_id: int
    username: str
    csrf_token: str
```

```python

async def current_auth(request: Request, db: Db, state: State) -> AuthContext | None:
    token = request.cookies.get(SESSION_COOKIE)
    if not token:
        return None
    loaded = await auth.load_session(db, token, now=state.clock.now())
    if loaded is None:
        return None
    operator_session, operator = loaded
    return AuthContext(
        operator_id=operator.id, username=operator.username, csrf_token=operator_session.csrf_token
    )


MaybeAuth = Annotated[AuthContext | None, Depends(current_auth)]
```

- Linhas 50–61: `AuthContext` copia três valores **simples** da sessão (id, nome e token CSRF). A docstring conta o porquê: o bug B7. Guardar o objeto `Operator` do ORM fazia o template tentar recarregá-lo depois de um `rollback`, o que exige I/O fora do contexto assíncrono e quebrava a página.
- Linhas 64–74: lê o cookie; sem cookie, ou com sessão inválida ou expirada (`load_session` devolve `None`), não há usuário. O relógio vem do `AppState`, o que permite ao teste `test_session_expires` avançar o tempo.
- Linha 77: `MaybeAuth` é "talvez logado". A página de login usa este, porque precisa funcionar sem sessão.

`src/pulseops/web/deps.py, linhas 80–117`

```python
def tokens_match(expected: str | None, received: str | None) -> bool:
    if not expected or not received:
        return False
    return hmac.compare_digest(expected, received)


async def require_api_auth(request: Request, ctx: MaybeAuth) -> AuthContext:
    """API JSON: 401 sem sessão; métodos que alteram dados exigem o header X-CSRF-Token."""
    if ctx is None:
        raise HTTPException(status_code=401, detail="autenticação necessária")
    if request.method in UNSAFE_METHODS and not tokens_match(
        ctx.csrf_token, request.headers.get("X-CSRF-Token")
    ):
        raise HTTPException(status_code=403, detail="token CSRF ausente ou inválido")
    return ctx


ApiAuth = Annotated[AuthContext, Depends(require_api_auth)]


class LoginRequiredError(Exception):
    def __init__(self, next_path: str) -> None:
        self.next_path = next_path


async def require_page_auth(request: Request, ctx: MaybeAuth) -> AuthContext:
    """Páginas: sem sessão, redireciona para o login (tratado em app.py)."""
    if ctx is None:
        raise LoginRequiredError(request.url.path)
    return ctx


PageAuth = Annotated[AuthContext, Depends(require_page_auth)]


def verify_form_csrf(ctx: AuthContext, submitted: str | None) -> None:
```

```
116     if not tokens_match(ctx.csrf_token, submitted):
117         raise HTTPException(status_code=403, detail="token CSRF ausente ou inválido")
```

- Linhas 80–83: compara tokens com `hmac.compare_digest`, que leva o mesmo tempo qualquer que seja a posição da primeira diferença. Um `==` comum para no primeiro caractere diferente, e o tempo de resposta poderia revelar o token aos poucos (ataque de *timing*). Token vazio nunca confere.
- Linhas 86–94: a regra da API. Sem sessão, 401. Com sessão e método que altera dados, exige o cabeçalho `X-CSRF-Token` igual ao da sessão, senão 403. Um site malicioso pode fazer o navegador enviar o cookie, mas não consegue ler o token para pôr no cabeçalho.
- Linhas 100–109: nas páginas, a falta de sessão vira redirecionamento ao login (o tratador está em `app.py`, linhas 73–75), não um 401 seco.
- Linhas 115–117: nos formulários HTML, o token vem num campo escondido `csrf_token`, conferido do mesmo jeito.

Falhas possíveis. Se o banco cair, `current_auth` falha na consulta da sessão e o tratador de `DBAPIError` responde 503.

`src/pulseops/web/health.py`

Finalidade. Os dois endpoints de saúde do próprio PulseOps (RF09), sem login.

Dados. Nenhuma entrada. Produz JSON. **Relações.** O `compose.yaml` usa `/health/ready` no *healthcheck* do contêiner `api` (linha 50); é ele que faz o `docker compose up --wait` esperar a API ficar pronta. `/health/live` fica disponível para um orquestrador que reinicie processos travados. **Testes.** `test_live_and_ready`, `test_ready_returns_503_when_database_is_down` e `test_health_endpoints_are_public`.

src/pulseops/web/health.py

```python
"""Liveness e readiness do próprio PulseOps (RF09). Não exigem login."""

import asyncio
from typing import Any

from fastapi import APIRouter
from fastapi.responses import JSONResponse
from sqlalchemy import text

from pulseops.services import queries
from pulseops.web.deps import State

router = APIRouter(prefix="/health", tags=["health"])

READY_TIMEOUT_SECONDS = 2.0


@router.get("/live")
async def live() -> dict[str, str]:
    """O processo está respondendo. Não consulta o banco de propósito."""
    return {"status": "ok"}


@router.get("/ready")
async def ready(state: State) -> JSONResponse:
    """Pronto para atender: o banco responde em até 2 s. O worker é informado, mas a API
    continua pronta sem ele (o painel mostra os dados como desatualizados)."""
    body: dict[str, Any] = {"status": "ok", "database": "ok"}
    try:
```

```
30          async with asyncio.timeout(READY_TIMEOUT_SECONDS):
31              async with state.sessions() as session:
32                  await session.execute(text("SELECT 1"))
33                  worker = await queries.worker_status(
34                      session,
35                      now=state.clock.now(),
36                      stale_after=state.settings.worker_stale_after_seconds,
37                  )
38      except Exception:
39          body.update(status="unavailable", database="unavailable")
40          return JSONResponse(body, status_code=503)
41      body["worker"] = {
42          "alive": worker.alive,
43          "last_seen_at": worker.last_seen_at.isoformat() if worker.last_seen_at else None,
44      }
45      return JSONResponse(body)
```

- Linhas 18–21: *liveness*: “o processo responde”. Não consulta o banco **de propósito**: se consultasse, um orquestrador que reinicia processos “mortos” reiniciaria a API em laço sempre que o banco caísse, sem resolver nada.
- Linhas 24–45: *readiness*: “posso atender”. Linha 30: `asyncio.timeout(2.0)` cancela tudo que passar de 2 s, para o endpoint não ficar pendurado num banco lento. Linha 32: o `SELECT 1` mais simples possível. Linhas 33–37: aproveita para informar o estado do worker. Linhas 38–40: qualquer erro vira 503 com `"unavailable"`. O `except Exception` amplo é intencional aqui: o endpoint existe justamente para dizer “não”, e não para quebrar.
- Linhas 41–45: o worker parado **não** torna a API “não pronta”. A API continua servindo o painel, que mostra os dados como desatualizados.

`src/pulseops/web/schemas.py`

Finalidade. Os contratos JSON da API: o formato aceito na entrada e o formato devolvido na saída, como classes Pydantic.

Dados. Classes `*In` (entrada), `*Out` (saída) e `*Page` (página com cursor). **Relações.** Usado só por `api.py`. Os limites de tamanho vêm de `domain/monitor_rules.py`. **Testes.** `test_unknown_fields_and_methods_rejected`, `test_invalid_configuration_is_rejected` e os testes de `test_api_history.py`.

`src/pulseops/web/schemas.py, linhas 11–57`

```
11  class MonitorIn(BaseModel):
12      """Limites de tipo e tamanho aqui; regras combinadas em domain/monitor_rules.py."""
13
14      model_config = ConfigDict(extra="forbid")
15
16      name: str = Field(max_length=MAX_NAME)
17      url: str = Field(max_length=MAX_URL)
18      description: str = Field(default="", max_length=MAX_DESCRIPTION)
19      method: Literal["GET", "HEAD"] = "GET"
20      interval_seconds: int = 60
21      timeout_seconds: int = 5
22      accepted_status_codes: str = Field(default="200-299", max_length=100)
23      latency_threshold_ms: int = 1000
24      failure_threshold: int = 3
25      recovery_threshold: int = 2
26
27      def to_config(self) -> MonitorConfig:
28          return MonitorConfig(**self.model_dump())
29
30
```

```python
class MonitorOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    description: str
    url: str
    method: str
    interval_seconds: int
    timeout_seconds: int
    accepted_status_codes: str
    latency_threshold_ms: int
    failure_threshold: int
    recovery_threshold: int
    state: str
    health: str
    health_reason: str
    stale: bool = False
    consecutive_failures: int
    consecutive_successes: int
    config_version: int
    next_run_at: datetime | None
    last_check_at: datetime | None
    created_at: datetime
    updated_at: datetime
    archived_at: datetime | None
```

- Linha 12: a divisão de trabalho. O Pydantic confere **tipo e tamanho**; as regras que combinam campos (por exemplo, timeout menor que o intervalo) ficam em `domain/monitor_rules.py`, que o formulário HTML também usa. Assim, a API e o painel aplicam a mesma regra.
- Linha 14: `extra="forbid"` recusa campos desconhecidos com 422. Sem isso, um campo digitado errado (`interval_secs`) seria ignorado em silêncio e o valor padrão entraria no lugar.
- Linha 19: `Literal["GET", "HEAD"]` aceita só esses dois métodos.
- Linhas 27–28: converte para o `MonitorConfig` do domínio.
- Linha 32: `from_attributes=True` permite montar a saída direto de um objeto do ORM (`MonitorOut.model_validate(monitor)`).
- Linha 48: `stale` não é coluna do banco; `api.py` calcula e preenche.

`src/pulseops/web/schemas.py, linhas 153–203`

```python
class MetricsOut(BaseModel):
    start: datetime
    end: datetime
    evaluated_checks: int
    internal_errors: int
    by_outcome: dict[str, int]
    available_checks_pct: float | None
    healthy_checks_pct: float | None
    latency_samples: int
    latency_avg_ms: float | None
    latency_p95_ms: float | None
    incident_count: int
    incident_duration_seconds: float
    last_check_at: datetime | None
    gaps: list[tuple[datetime, datetime]]


class DestinationIn(BaseModel):
    model_config = ConfigDict(extra="forbid")

    name: str = Field(min_length=1, max_length=100)
    url: str = Field(max_length=MAX_URL)
    enabled: bool = True
```

```python
class DestinationPatch(BaseModel):
    model_config = ConfigDict(extra="forbid")

    enabled: bool


class DestinationOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    name: str
    url: str
    enabled: bool
    created_at: datetime


class SummaryOut(BaseModel):
    total: int
    by_health: dict[str, int]
    stale: int
    open_incidents: int
    pending_deliveries: int
    failed_deliveries: int
    worker_alive: bool
    worker_last_seen_at: datetime | None
    generated_at: datetime
```

- Linhas 153–167: as métricas. As porcentagens e latências são `float | None`: `None` vira `null` no JSON quando não há dados, em vez de um 0 que pareceria "0% disponível".
- Linhas 170–182: entrada de destinos de webhook; o `PATCH` só aceita `enabled`.
- Linhas 184–191: saída de destinos. **Ponto de atenção:** `url` sai completa. Se a URL de um webhook contiver um segredo (muitos serviços colocam o token no caminho), ele aparece para quem estiver logado. Como o único usuário é o operador que cadastrou o destino, isso foi aceito; com vários usuários, seria preciso mascarar.
- Linhas 194–203: o resumo usado pelo `app.js`.

As classes das linhas 59–150 (`CheckOut`, `IncidentOut`, `IncidentEventOut`, `AttemptOut`, `DeliveryOut`, `AlertOut`, `IncidentDetailOut`) seguem o mesmo padrão de `MonitorOut`: uma lista de campos com tipos e `from_attributes=True` quando são montadas de objetos do ORM.

`src/pulseops/web/api.py`

Finalidade. A API JSON em `/api/v1`: monitores, histórico de checks, métricas, incidentes, alertas, destinos e resumo. Toda rota exige sessão (`ApiAuth`).

Dados. JSON validado por `schemas.py` e parâmetros de consulta. Produz JSON. **Relações.** Chama `services/monitors.py`, `services/queries.py` e `security/transport.py`. Usada pelo `app.js` (só `/summary`), por `scripts/demo_flow.py` e por quem quiser automatizar. **Testes.** `tests/integration/test_api_monitors.py` (17 casos) e `test_api_history.py` (8).

src/pulseops/web/api.py, linhas 37–69

```python
router = APIRouter(prefix="/api/v1", tags=["api"])

Limit = Annotated[int, Query(ge=1, le=queries.MAX_PAGE_SIZE)]


def _http_error(exc: Exception) -> HTTPException:
    if isinstance(exc, NotFoundError):
        return HTTPException(404, str(exc))
```

```python
    if isinstance(exc, ConflictError):
        return HTTPException(409, str(exc))
    if isinstance(exc, ValidationFailedError):
        detail = [
            {"loc": ["body", e.field], "msg": e.message, "type": "value_error"} for e in exc.errors
        ]
        return HTTPException(422, detail)
    raise exc


async def _monitor_out(
    db: Db, state: State, monitor_ids: list[int] | None = None
) -> list[MonitorOut]:
    now = state.clock.now()
    worker = await queries.worker_status(
        db, now=now, stale_after=state.settings.worker_stale_after_seconds
    )
    result = []
    for monitor in await queries.list_monitors(db, include_archived=True):
        if monitor_ids is not None and monitor.id not in monitor_ids:
            continue
        out = MonitorOut.model_validate(monitor)
        out.stale = queries.is_stale(monitor, now=now, worker=worker)
        result.append(out)
    return result
```

- Linha 39: `Limit` limita o tamanho da página entre 1 e 200 (`MAX_PAGE_SIZE`). Um `?limit=201` recebe 422, sem chegar ao banco (`test_pagination_limits`).
- Linhas 42–52: traduz as exceções dos serviços para HTTP: não encontrado 404, conflito 409, validação 422 no mesmo formato que o FastAPI usa para erros do Pydantic. Linha 52: uma exceção inesperada é relançada, não engolida.
- Linhas 55–69: monta a saída dos monitores com o campo `stale`. **Ponto de atenção:** carrega **todos** os monitores (linha 63) e filtra em Python, mesmo quando a rota quer um só (`get_monitor`, linha 99). Com dezenas de monitores não faz diferença; com milhares, seria uma consulta por id. É ineficiência, não bug: o resultado está certo.

`src/pulseops/web/api.py, linhas 72–142`

```python
@router.get("/monitors")
async def list_monitors(
    db: Db, state: State, _: ApiAuth, include_archived: bool = False
) -> list[MonitorOut]:
    items = await _monitor_out(db, state)
    return [m for m in items if include_archived or m.state != "archived"]


@router.post("/monitors", status_code=201)
async def create_monitor(body: MonitorIn, db: Db, state: State, _: ApiAuth) -> MonitorOut:
    try:
        monitor = await monitor_service.create_monitor(
            db,
            body.to_config(),
            now=state.clock.now(),
            policy=state.policy,
            min_interval_seconds=state.settings.min_interval_seconds,
            resolver=state.resolver,
        )
    except ValidationFailedError as exc:
        raise _http_error(exc) from exc
    await db.commit()
    return (await _monitor_out(db, state, [monitor.id]))[0]


@router.get("/monitors/{monitor_id}")
async def get_monitor(monitor_id: int, db: Db, state: State, _: ApiAuth) -> MonitorOut:
    items = await _monitor_out(db, state, [monitor_id])
```

```
    if not items:
        raise HTTPException(404, f"monitor {monitor_id} não encontrado")
    return items[0]


@router.put("/monitors/{monitor_id}")
async def update_monitor(
    monitor_id: int, body: MonitorIn, db: Db, state: State, _: ApiAuth
) -> MonitorOut:
    try:
        await monitor_service.update_monitor(
            db,
            monitor_id,
            body.to_config(),
            now=state.clock.now(),
            policy=state.policy,
            min_interval_seconds=state.settings.min_interval_seconds,
            resolver=state.resolver,
        )
    except (NotFoundError, ConflictError, ValidationFailedError) as exc:
        raise _http_error(exc) from exc
    await db.commit()
    return (await _monitor_out(db, state, [monitor_id]))[0]


@router.post("/monitors/{monitor_id}/{action}")
async def monitor_action(
    monitor_id: int, action: str, db: Db, state: State, _: ApiAuth
) -> MonitorOut:
    operations = {
        "pause": monitor_service.pause_monitor,
        "resume": monitor_service.resume_monitor,
        "archive": monitor_service.archive_monitor,
        "check": monitor_service.request_check_now,
    }
    if action not in operations:
        raise HTTPException(404, "ação desconhecida")
    try:
        await operations[action](db, monitor_id, now=state.clock.now())
    except (NotFoundError, ConflictError) as exc:
        raise _http_error(exc) from exc
    await db.commit()
    return (await _monitor_out(db, state, [monitor_id]))[0]
```

- Linhas 72–77: lista; arquivados só com `?include_archived=true`.
- Linhas 80–94: cria. A política SSRF e o resolvedor vão para o serviço, que valida a URL (D4, D6). Só depois de `commit` a resposta é montada, com status 201.
- Linhas 97–102: busca um; 404 se não existe.
- Linhas 105–122: atualiza com `PUT` (a configuração inteira). Um monitor arquivado dá 409.
- Linhas 125–142: as quatro ações num caminho só (`/monitors/7/pause`). Linhas 129–136: um dicionário de funções em vez de quatro rotas quase iguais; uma ação fora da lista dá 404.

`src/pulseops/web/api.py, linhas 145–211`

```
@router.get("/monitors/{monitor_id}/checks")
async def list_checks(
    monitor_id: int,
    db: Db,
    _: ApiAuth,
    limit: Limit = queries.DEFAULT_PAGE_SIZE,
    cursor: str | None = None,
    start: Annotated[datetime | None, Query(alias="from")] = None,
    end: Annotated[datetime | None, Query(alias="to")] = None,
    outcome: Outcome | None = None,
) -> CheckPage:
    _require_aware(start, end)
    try:
```

```
158         page = await queries.list_checks(
159             db,
160             monitor_id=monitor_id,
161             limit=limit,
162             cursor=cursor,
163             start=start,
164             end=end,
165             outcome=outcome.value if outcome else None,
166         )
167     except ValueError as exc:
168         raise HTTPException(422, str(exc)) from exc
169     return CheckPage(
170         items=[CheckOut.model_validate(c) for c in page.items], next_cursor=page.next_cursor
171     )
172
173
174 @router.get("/monitors/{monitor_id}/metrics")
175 async def monitor_metrics(
176     monitor_id: int,
177     db: Db,
178     state: State,
179     _: ApiAuth,
180     start: Annotated[datetime | None, Query(alias="from")] = None,
181     end: Annotated[datetime | None, Query(alias="to")] = None,
182 ) -> MetricsOut:
183     _require_aware(start, end)
184     now = state.clock.now()
185     end = end or now
186     start = start or end - timedelta(hours=24)
187     if start >= end:
188         raise HTTPException(422, "'from' deve ser anterior a 'to'")
189     m = await queries.monitor_metrics(db, monitor_id, start=start, end=end, now=now)
190     return MetricsOut(
191         start=m.start,
192         end=m.end,
193         evaluated_checks=m.evaluated,
194         internal_errors=m.internal_errors,
195         by_outcome=m.by_outcome,
196         available_checks_pct=m.available_pct,
197         healthy_checks_pct=m.healthy_pct,
198         latency_samples=m.latency_samples,
199         latency_avg_ms=m.latency_avg_ms,
200         latency_p95_ms=m.latency_p95_ms,
201         incident_count=m.incident_count,
202         incident_duration_seconds=m.incident_duration.total_seconds(),
203         last_check_at=m.last_check_at,
204         gaps=[(g.gap_start, g.gap_end) for g in m.gaps],
205     )
206
207
208 def _require_aware(*values: datetime | None) -> None:
209     """Datas sem fuso seriam ambíguas; a API exige UTC explícito (por exemplo, sufixo Z)."""
210     if any(v is not None and v.tzinfo is None for v in values):
211         raise HTTPException(422, "datas precisam de fuso horário (use Z para UTC)")
```

- Linhas 145–171: histórico de checks com cursor e filtros. Linhas 152–153: `Query(alias="from")` porque `from` é palavra reservada do Python e não pode ser nome de parâmetro. Linha 154: `outcome` tipado como `Outcome`, então um valor inválido já dá 422. Linhas 167–168: cursor malformado vira 422. **Ponto de atenção:** para um `monitor_id` que não existe, a resposta é uma página vazia com 200, não 404. Não é falso (não há checks), mas é menos informativo que a rota de métricas também não acusar.
- Linhas 174–205: métricas. Linhas 185–186: sem janela, últimas 24 h até agora. Linhas 187–188: janela invertida dá 422. O resto copia os campos para o contrato.
- Linhas 208–211: datas sem fuso são recusadas. `2026-10-22T10:00` é ambíguo (10 h em qual fuso?); `2026-10-22T10:00Z` não é (`test_naive_datetimes_are_rejected`).

src/pulseops/web/api.py, linhas 214–272

```python
@router.get("/incidents")
async def list_incidents(
    db: Db,
    _: ApiAuth,
    status: Annotated[str | None, Query(pattern="^(open|resolved|closed)$")] = None,
    monitor_id: int | None = None,
    limit: Limit = queries.DEFAULT_PAGE_SIZE,
    cursor: str | None = None,
) -> IncidentPage:
    try:
        page = await queries.list_incidents(
            db, status=status, monitor_id=monitor_id, limit=limit, cursor=cursor
        )
    except ValueError as exc:
        raise HTTPException(422, str(exc)) from exc
    return IncidentPage(
        items=[IncidentOut.model_validate(i) for i, _m in page.items], next_cursor=page.next_cursor
    )


def _alert_out(view: queries.AlertView) -> AlertOut:
    return AlertOut(
        event_id=view.event.event_key,
        kind=view.event.kind,
        incident_id=view.event.incident_id,
        created_at=view.event.created_at,
        deliveries=[
            DeliveryOut(
                id=d.delivery.id,
                destination=d.destination.name,
                status=d.delivery.status,
                attempt_count=d.delivery.attempt_count,
                next_attempt_at=d.delivery.next_attempt_at,
                delivered_at=d.delivery.delivered_at,
                last_error=d.delivery.last_error,
                attempts=[AttemptOut.model_validate(a) for a in d.attempts],
            )
            for d in view.deliveries
        ],
    )


@router.get("/incidents/{incident_id}")
async def get_incident(incident_id: int, db: Db, _: ApiAuth) -> IncidentDetailOut:
    detail = await queries.incident_detail(db, incident_id)
    if detail is None:
        raise HTTPException(404, f"incidente {incident_id} não encontrado")
    return IncidentDetailOut(
        incident=IncidentOut.model_validate(detail.incident),
        events=[IncidentEventOut.model_validate(e) for e in detail.events],
        alerts=[_alert_out(a) for a in detail.alerts],
    )


@router.get("/alerts")
async def list_alerts(
    db: Db, _: ApiAuth, limit: Limit = queries.DEFAULT_PAGE_SIZE
) -> list[AlertOut]:
    return [_alert_out(a) for a in await queries.list_alerts(db, limit=limit)]
```

- Linhas 214–231: incidentes. Linha 218: `pattern` limita `status` a três valores. **Ponto de atenção:** `status=closed` devolve incidentes `resolved` **e** `closed`, porque `queries.list_incidents` trata "closed" como "encerrados" (a interface usa o mesmo filtro). Já `status=resolved` devolve só os resolvidos. Quem usar a API esperando só os encerrados manualmente terá uma surpresa; o comportamento está no comentário de `services/queries.py`, linha 166.

- Linhas 234–253: converte a visão de um alerta (evento, entregas e tentativas) no contrato JSON. O `event_id` exposto é a `event_key` estável, a mesma enviada no webhook, para o receptor deduplicar.
- Linhas 256–265: detalhe de um incidente com a linha do tempo e os alertas.
- Linhas 268–272: últimos alertas.

`src/pulseops/web/api.py, linhas 275–337`

```
275 @router.get("/destinations")
276 async def list_destinations(db: Db, _: ApiAuth) -> list[DestinationOut]:
277     rows = await db.scalars(select(AlertDestination).order_by(AlertDestination.id))
278     return [DestinationOut.model_validate(d) for d in rows]
279
280
281 @router.post("/destinations", status_code=201)
282 async def create_destination(
283     body: DestinationIn, db: Db, state: State, _: ApiAuth
284 ) -> DestinationOut:
285     try:
286         await validate_destination(state.policy, body.url, state.resolver)
287     except UnsafeDestinationError as exc:
288         raise HTTPException(
289             422, [{"loc": ["body", "url"], "msg": str(exc), "type": "value_error"}]
290         ) from exc
291     now = state.clock.now()
292     destination = AlertDestination(
293         name=body.name.strip(),
294         url=body.url.strip(),
295         enabled=body.enabled,
296         created_at=now,
297         updated_at=now,
298     )
299     db.add(destination)
300     try:
301         await db.commit()
302     except IntegrityError as exc:
303         raise HTTPException(409, "já existe um destino com este nome") from exc
304     return DestinationOut.model_validate(destination)
305
306
307 @router.patch("/destinations/{destination_id}")
308 async def update_destination(
309     destination_id: int, body: DestinationPatch, db: Db, state: State, _: ApiAuth
310 ) -> DestinationOut:
311     destination = await db.get(AlertDestination, destination_id)
312     if destination is None:
313         raise HTTPException(404, "destino não encontrado")
314     destination.enabled = body.enabled
315     destination.updated_at = state.clock.now()
316     await db.commit()
317     return DestinationOut.model_validate(destination)
318
319
320 @router.get("/summary")
321 async def get_summary(db: Db, state: State, _: ApiAuth) -> SummaryOut:
322     now = state.clock.now()
323     worker = await queries.worker_status(
324         db, now=now, stale_after=state.settings.worker_stale_after_seconds
325     )
326     s = await queries.summary(db, now=now, worker=worker)
327     return SummaryOut(
328         total=s.total,
329         by_health=s.by_health,
330         stale=s.stale,
331         open_incidents=s.open_incidents,
332         pending_deliveries=s.pending_deliveries,
333         failed_deliveries=s.failed_deliveries,
334         worker_alive=worker.alive,
335         worker_last_seen_at=worker.last_seen_at,
```

```
336         generated_at=now,
337     )
```

- Linhas 281–304: cria destino. Linha 286: a mesma validação SSRF usada para monitores, porque o worker também fará requisições para esse endereço. Linhas 300–303: nome repetido viola a restrição única do banco; o `IntegrityError` vira 409. É o banco quem garante a unicidade, não uma consulta prévia, que teria uma janela de corrida.
- Linhas 307–317: liga ou desliga um destino.
- Linhas 320–337: o resumo do painel, com o estado do worker e a hora em que foi gerado.

Falhas possíveis. Banco indisponível: 503 pelo tratador de `app.py`. Sessão expirada: 401. Mutação sem `X-CSRF-Token`: 403.

`src/pulseops/web/pages.py`

Finalidade. As páginas HTML (RF07) e o login (RF08). Faz o mesmo que a API, mas lendo formulários e devolvendo HTML ou redirecionamentos.

Dados. Formulários `application/x-www-form-urlencoded` e parâmetros de consulta. Produz HTML e redirecionamentos 303. **Relações.** Usa `deps.py`, `templating.py`, `charts.py` e os serviços. **Testes.** `tests/integration/test_dashboard.py` (12 casos) e `test_auth.py` (26).

src/pulseops/web/pages.py, linhas 35–69

```python
router = APIRouter(include_in_schema=False)

WINDOWS = {"1h": timedelta(hours=1), "24h": timedelta(hours=24), "7d": timedelta(days=7)}
Csrf = Annotated[str, Form()]


def render(
    request: Request,
    state: State,
    template: str,
    ctx: AuthContext | None,
    status_code: int = 200,
    **context: Any,
) -> HTMLResponse:
    base = {
        "tz": state.settings.timezone,
        "tz_name": state.settings.display_timezone,
        "demo_mode": state.settings.demo_mode,
        "username": ctx.username if ctx else None,
        "csrf_token": ctx.csrf_token if ctx else None,
    }
    return templates.TemplateResponse(
        request, template, {**base, **context}, status_code=status_code
    )


def redirect(path: str) -> RedirectResponse:
    return RedirectResponse(path, status_code=303)


def _safe_next(path: str | None) -> str:
    """Só aceita caminhos internos no ?next=, para o login não virar um redirecionador aberto."""
    if not path or not path.startswith("/") or path.startswith("//") or urlsplit(path).netloc:
        return "/"
    return path
```

- Linha 35: `include_in_schema=False`, porque páginas não fazem parte da API.
- Linha 37: as três janelas de tempo do detalhe do monitor.
- Linhas 41–58: `render` junta o contexto comum a toda página (fuso, modo demonstração, usuário e token CSRF) com o contexto da página.
- Linhas 61–62: 303 *See Other* depois de um `POST` faz o navegador abrir a página seguinte com `GET`. Recarregar a página não reenvia o formulário (padrão *Post/Redirect/Get*).
- Linhas 65–69: `_safe_next` só aceita caminhos internos. Sem isso, `/login?next=https://site-falso.example` mandaria a pessoa, recém-logada, para outro site (*open redirect*). `//site.example` também é recusado, porque o navegador o lê como outro domínio (`test_next_parameter_cannot_redirect_outside`).

`src/pulseops/web/pages.py, linhas 75–148`

```
75 @router.get("/login")
76 async def login_form(request: Request, state: State, ctx: MaybeAuth, next: str = "/") -> Response:
77     if ctx is not None:
78         return redirect(_safe_next(next))
79     token = secrets.token_urlsafe(32)
80     response = render(request, state, "login.html", None, login_csrf=token, next=_safe_next(next))
81     response.set_cookie(
82         LOGIN_CSRF_COOKIE,
83         token,
84         httponly=True,
85         samesite="strict",
86         secure=state.settings.cookie_secure,
87     )
88     return response
89
90
91 @router.post("/login")
92 async def login(
93     request: Request,
94     db: Db,
95     state: State,
96     username: Annotated[str, Form(max_length=50)],
97     password: Annotated[str, Form(max_length=1024)],
98     csrf_token: Csrf,
99     next: Annotated[str, Form()] = "/",
100 ) -> Response:
101     if not tokens_match(request.cookies.get(LOGIN_CSRF_COOKIE), csrf_token):
102         raise HTTPException(403, "token CSRF ausente ou inválido")
103     operator = await auth.authenticate(db, username, password)
104     if operator is None:
105         token = secrets.token_urlsafe(32)
106         response = render(
107             request,
108             state,
109             "login.html",
110             None,
111             status_code=401,
112             error="Usuário ou senha inválidos.",
113             login_csrf=token,
114             next=_safe_next(next),
115         )
116         response.set_cookie(
117             LOGIN_CSRF_COOKIE,
118             token,
119             httponly=True,
120             samesite="strict",
121             secure=state.settings.cookie_secure,
122         )
123         return response
124     ttl = timedelta(minutes=state.settings.session_ttl_minutes)
125     new = await auth.start_session(db, operator, now=state.clock.now(), ttl=ttl)
126     await db.commit()
127     success = redirect(_safe_next(next))
```

```
128     success.set_cookie(
129         SESSION_COOKIE,
130         new.token,
131         max_age=int(ttl.total_seconds()),
132         httponly=True,
133         samesite="lax",
134         secure=state.settings.cookie_secure,
135         path="/",
136     )
137     success.delete_cookie(LOGIN_CSRF_COOKIE)
138     return success
139
140
141 @router.post("/logout")
142 async def logout(request: Request, db: Db, ctx: PageAuth, csrf_token: Csrf) -> Response:
143     verify_form_csrf(ctx, csrf_token)
144     await auth.end_session(db, request.cookies.get(SESSION_COOKIE, ""))
145     await db.commit()
146     response = redirect("/login")
147     response.delete_cookie(SESSION_COOKIE, path="/")
148     return response
```

- Linhas 75–88: o formulário de login. Quem já está logado vai direto ao destino. Linha 79: gera um token aleatório, que vai **duas vezes** para o navegador: num cookie e num campo escondido do formulário. É o padrão *double submit cookie*: antes do login não existe sessão onde guardar o token. Linha 85: o cookie é `SameSite=Strict`, então não é enviado em requisições que partem de outro site.
- Linhas 91–102: o login confere se cookie e campo coincidem. Um site malicioso pode enviar um formulário para `/login`, mas não consegue ler nem definir esse cookie. Isso impede o *login CSRF* (logar a vítima numa conta do atacante).
- Linhas 103–123: senha errada ou usuário inexistente: a **mesma** mensagem e o status 401 (`test_unknown_user_gets_same_message`), com um token novo.
- Linhas 124–138: sucesso. Cria a sessão e confirma (`commit`). O cookie `pulseops_session` tem `max_age` igual à duração da sessão, `HttpOnly` (o JavaScript não lê), `SameSite=Lax` (linha 133) e `Secure` conforme a configuração. `Lax` permite que um link de outro site abra o painel já logado, mas não envia o cookie em `POST` vindo de outro site. A proteção dos formulários continua sendo o token CSRF. Linha 137: apaga o cookie de CSRF do login, que não serve mais.
- Linhas 141–148: logout exige token CSRF (senão um site qualquer poderia deslogar a pessoa), apaga a sessão no banco e o cookie.

`src/pulseops/web/pages.py, linhas 154–207`

```
154 @router.get("/")
155 async def dashboard(
156     request: Request, db: Db, state: State, ctx: PageAuth, archived: bool = False
157 ) -> Response:
158     now = state.clock.now()
159     worker = await queries.worker_status(
160         db, now=now, stale_after=state.settings.worker_stale_after_seconds
161     )
162     monitors = await queries.list_monitors(db, include_archived=archived)
163     rows = [(m, queries.is_stale(m, now=now, worker=worker)) for m in monitors]
164     return render(
165         request,
166         state,
167         "dashboard.html",
168         ctx,
169         summary=await queries.summary(db, now=now, worker=worker),
170         worker=worker,
171         rows=rows,
172         show_archived=archived,
173     )
```

```
174
175
176 def _form_config(form: dict[str, str]) -> tuple[MonitorConfig | None, list[FieldError]]:
177     errors: list[FieldError] = []
178     numbers: dict[str, int] = {}
179     for field in (
180         "interval_seconds",
181         "timeout_seconds",
182         "latency_threshold_ms",
183         "failure_threshold",
184         "recovery_threshold",
185     ):
186         try:
187             numbers[field] = int(form.get(field, ""))
188         except ValueError:
189             errors.append(FieldError(field, "informe um número inteiro"))
190     if errors:
191         return None, errors
192     return (
193         MonitorConfig(
194             name=form.get("name", "")[:200],
195             url=form.get("url", "")[:3000],
196             description=form.get("description", "")[:1000],
197             method=form.get("method", "GET"),
198             accepted_status_codes=form.get("accepted_status_codes", "")[:200],
199             **numbers,
200         ),
201         [],
202     )
203
204
205 async def _read_form(request: Request) -> dict[str, str]:
206     form = await request.form()
207     return {k: v for k, v in form.items() if isinstance(v, str)}
```

- Linhas 154–173: o painel. Calcula, para cada monitor, se os dados estão desatualizados.
- Linhas 176–202: `_form_config` converte o formulário em `MonitorConfig`. Linhas 179–189: campos numéricos que não são inteiros viram erro de campo, não uma exceção. Linhas 194–198: os textos são cortados num tamanho bem acima do limite. O corte serve só para não processar textos enormes; o limite de verdade é conferido em `monitor_rules.py`, que dá a mensagem de erro.
- Linhas 205–207: lê o formulário e descarta campos que não são texto (arquivos).

`src/pulseops/web/pages.py, linhas 218–246`

```
218 @router.post("/monitors/new")
219 async def create_monitor(request: Request, db: Db, state: State, ctx: PageAuth) -> Response:
220     form = await _read_form(request)
221     verify_form_csrf(ctx, form.get("csrf_token"))
222     config, errors = _form_config(form)
223     if config is not None:
224         try:
225             monitor = await monitor_service.create_monitor(
226                 db,
227                 config,
228                 now=state.clock.now(),
229                 policy=state.policy,
230                 min_interval_seconds=state.settings.min_interval_seconds,
231                 resolver=state.resolver,
232             )
233             await db.commit()
234             return redirect(f"/monitors/{monitor.id}")
235         except ValidationFailedError as exc:
236             errors = exc.errors
237     return render(
238         request,
239         state,
```

```
240         "monitor_form.html",
241         ctx,
242         status_code=422,
243         form=form,
244         errors={e.field: e.message for e in errors},
245         monitor=None,
246     )
```

- Linha 221: o token CSRF é a primeira coisa conferida.
- Linhas 223–236: se a conversão deu certo, o serviço valida e cria. Erro de validação volta como lista de erros por campo.
- Linhas 237–246: com erro, a página é exibida de novo com status 422, os valores digitados (`form=form`) e as mensagens ao lado de cada campo. A pessoa não perde o que digitou.

`src/pulseops/web/pages.py, linhas 249–297`

```
249 @router.get("/monitors/{monitor_id}")
250 async def monitor_detail(
251     request: Request,
252     monitor_id: int,
253     db: Db,
254     state: State,
255     ctx: PageAuth,
256     window: Annotated[str, Query(pattern="^(1h|24h|7d)$")] = "24h",
257     outcome: Outcome | None = None,
258     cursor: str | None = None,
259 ) -> Response:
260     try:
261         monitor = await monitor_service.get_monitor(db, monitor_id)
262     except NotFoundError as exc:
263         raise HTTPException(404, str(exc)) from exc
264     now = state.clock.now()
265     start = now - WINDOWS[window]
266     worker = await queries.worker_status(
267         db, now=now, stale_after=state.settings.worker_stale_after_seconds
268     )
269     try:
270         checks = await queries.list_checks(
271             db,
272             monitor_id=monitor_id,
273             limit=25,
274             cursor=cursor,
275             outcome=outcome.value if outcome else None,
276         )
277     except ValueError as exc:
278         raise HTTPException(422, str(exc)) from exc
279     points = await queries.latency_series(db, monitor_id, start=start, end=now)
280     incidents = await queries.list_incidents(db, monitor_id=monitor_id, limit=10)
281     return render(
282         request,
283         state,
284         "monitor_detail.html",
285         ctx,
286         monitor=monitor,
287         stale=queries.is_stale(monitor, now=now, worker=worker),
288         worker=worker,
289         metrics=await queries.monitor_metrics(db, monitor_id, start=start, end=now, now=now),
290         chart=build_latency_chart(points, monitor.latency_threshold_ms),
291         changes=await queries.health_changes(db, monitor_id),
292         checks=checks,
293         incidents=[i for i, _m in incidents.items],
294         window=window,
295         outcome=outcome.value if outcome else "",
296         outcomes=list(Outcome),
297     )
```

- Linha 256: `window` limitado a `1h`, `24h` ou `7d` por expressão regular.

- Linhas 269–278: últimos 25 checks com cursor e filtro de resultado.
- Linha 279: os pontos do gráfico (até 300, sem `internal_error`; ver `queries.latency_series`).
- Linhas 281–297: métricas da janela, o gráfico já montado (`charts.py`), as mudanças de saúde, os checks e os incidentes.

As demais rotas seguem os mesmos padrões e não trazem ideia nova: `edit_monitor_form` e `edit_monitor` (linhas 300–348) repetem o fluxo de criação com `update_monitor`; `monitor_action` (351–371) é a versão HTML da rota de ações da API, com `verify_form_csrf`; `incidents_page` e `incident_page` (377–404) listam e detalham; `notifications_page` (407–421) lista destinos e alertas.

`src/pulseops/web/pages.py, linhas 424–473`

```
424 @router.post("/notifications/destinations")
425 async def create_destination(request: Request, db: Db, state: State, ctx: PageAuth) -> Response:
426     form = await _read_form(request)
427     verify_form_csrf(ctx, form.get("csrf_token"))
428     name = form.get("name", "").strip()[:100]
429     url = form.get("url", "").strip()[:2048]
430     errors: dict[str, str] = {}
431     if not name:
432         errors["name"] = "obrigatório"
433     try:
434         await validate_destination(state.policy, url, state.resolver)
435     except UnsafeDestinationError as exc:
436         errors["url"] = str(exc)
437     if not errors:
438         now = state.clock.now()
439         db.add(AlertDestination(name=name, url=url, enabled=True, created_at=now, updated_at=now))
440         try:
441             await db.commit()
442             return redirect("/notifications")
443         except IntegrityError:
444             await db.rollback()
445             errors["name"] = "já existe um destino com este nome"
446     destinations = list(
447         (await db.scalars(select(AlertDestination).order_by(AlertDestination.id))).all()
448     )
449     return render(
450         request,
451         state,
452         "notifications.html",
453         ctx,
454         status_code=422,
455         alerts=await queries.list_alerts(db, limit=50),
456         destinations=destinations,
457         errors=errors,
458         form=form,
459     )
460
461
462 @router.post("/notifications/destinations/{destination_id}/toggle")
463 async def toggle_destination(
464     destination_id: int, db: Db, state: State, ctx: PageAuth, csrf_token: Csrf
465 ) -> Response:
466     verify_form_csrf(ctx, csrf_token)
467     destination = await db.get(AlertDestination, destination_id)
468     if destination is None:
469         raise HTTPException(404, "destino não encontrado")
470     destination.enabled = not destination.enabled
471     destination.updated_at = state.clock.now()
472     await db.commit()
473     return redirect("/notifications")
```

- Linhas 428–436: valida nome e URL (com a política SSRF) e junta os erros.

- Linhas 440–445: nome repetido. Linha 444: depois de um erro no `commit`, o SQLAlchemy exige `rollback` antes de usar a sessão de novo (a consulta da linha 447 precisa dela). O `rollback` expira os objetos do ORM carregados na sessão, e foi isso que causou o bug B7 (ver `deps.py`, `AuthContext`).
- Linhas 462–473: inverte `enabled` de um destino.

Falhas possíveis. Monitor inexistente: página 404 (`test_not_found_page_is_html`). Monitor arquivado em edição: 409. Banco fora do ar: página 503.

`src/pulseops/web/templating.py`

Finalidade. Configura o Jinja2 e registra os **filtros**, funções que formatam valores nos templates (`{{ valor | pct }}`).

Dados. Valores do banco (datas em UTC, números, códigos). Produz texto em português. **Relações.** `templates` é usado por `app.py` e `pages.py`; os filtros aparecem em todos os templates. **Testes.** `test_times_are_shown_in_display_timezone`, `test_monitor_name_is_escaped` e `test_monitor_detail_with_real_data`.

src/pulseops/web/templating.py, linhas 15–47

```
15 # autoescape=True: todo valor inserido no HTML é escapado (proteção contra XSS, T12).
16 _env = Environment(loader=FileSystemLoader(Path(__file__).parent / "templates"), autoescape=True)
17 templates = Jinja2Templates(env=_env)
18
19
20 @pass_context
21 def local_datetime(context: Context, value: datetime | None) -> str:
22     """Converte de UTC para o fuso de exibição configurado (RF05.6)."""
23     if value is None:
24         return "—"
25     tz: ZoneInfo = context["tz"]
26     return value.astimezone(tz).strftime("%d/%m/%Y %H:%M:%S")
27
28
29 def percent(value: float | None) -> str:
30     return "N/A" if value is None else f"{value:.1f}%".replace(".", ",")
31
32
33 def milliseconds(value: float | None) -> str:
34     return "N/A" if value is None else f"{value:.0f} ms"
35
36
37 def duration(value: timedelta | None) -> str:
38     if value is None:
39         return "—"
40     total = int(value.total_seconds())
41     hours, rest = divmod(total, 3600)
42     minutes, seconds = divmod(rest, 60)
43     if hours:
44         return f"{hours} h {minutes:02d} min"
45     if minutes:
46         return f"{minutes} min {seconds:02d} s"
47     return f"{seconds} s"
```

- Linhas 15–17: `autoescape=True`. Todo `{{ ... }}` é escapado: `<` vira `<` e assim por diante. O bug B3 explica a forma: a versão instalada do Starlette não aceita mais `Jinja2Templates(autoescape=True)`, então o projeto cria o próprio `Environment` e o entrega pronto

(`env=`). No `Environment` do Jinja, o padrão é **sem** escape; por isso o `autoescape=True` explícito é indispensável, e `test_monitor_name_is_escaped` confere que ele está ligado.

- Linhas 20–26: `@pass_context` dá ao filtro acesso ao contexto do template, de onde vem o fuso `tz`. As datas ficam em UTC no banco e só são convertidas na exibição.
- Linhas 29–34: `None` vira "N/A", nunca 0. A vírgula decimal é trocada à mão (`replace`), sem depender da *locale* do sistema.
- Linhas 37–47: duração legível: "2 h 05 min", "3 min 07 s" ou "42 s".

`src/pulseops/web/templating.py, linhas 50–95`

```
50 def health_label(value: str) -> str:
51     try:
52         return Health(value).label
53     except ValueError:
54         return value
55
56
57 def outcome_label(value: str | None) -> str:
58     if value is None:
59         return "—"
60     try:
61         return Outcome(value).label
62     except ValueError:
63         return value
64
65
66 STATE_LABELS = {"active": "Ativo", "paused": "Pausado", "archived": "Arquivado"}
67 INCIDENT_LABELS = {"open": "Aberto", "resolved": "Resolvido", "closed": "Encerrado sem recuperação"}
68 DELIVERY_LABELS = {
69     "pending": "Aguardando",
70     "sending": "Enviando",
71     "delivered": "Entregue",
72     "failed": "Falhou",
73 }
74 EVENT_LABELS = {
75     "first_failure": "Primeira falha",
76     "opened": "Incidente confirmado",
77     "error_changed": "Tipo de falha mudou",
78     "config_changed": "Configuração alterada",
79     "resolved": "Recuperado",
80     "closed": "Encerrado pelo operador",
81 }
82
83 filters: dict[str, Any] = {
84     "dt": local_datetime,
85     "pct": percent,
86     "ms": milliseconds,
87     "duration": duration,
88     "health_label": health_label,
89     "outcome_label": outcome_label,
90     "state_label": STATE_LABELS.get,
91     "incident_label": INCIDENT_LABELS.get,
92     "delivery_label": DELIVERY_LABELS.get,
93     "event_label": EVENT_LABELS.get,
94 }
95 templates.env.filters.update(filters)
```

- Linhas 50–63: rótulos em português vindos dos enums do domínio. Um valor desconhecido aparece cru, em vez de quebrar a página.
- Linhas 66–81: rótulos dos estados guardados como texto no banco.
- Linhas 83–95: o registro dos filtros. `STATE_LABELS.get` já é uma função: recebe a chave e devolve o rótulo (ou `None`).

`src/pulseops/web/charts.py`

Finalidade. Monta o gráfico de latência como dados para um SVG desenhado no servidor, sem biblioteca de JavaScript (ADR 0010).

Dados. Recebe `LatencyPoint` (hora, duração, resultado) e o limite de latência. Produz um `LatencyChart` com coordenadas e o texto de resumo. **Relações.** Chamado em `pages.py`, linha 290; desenhado em `monitor_detail.html`. **Testes.** `tests/unit/test_charts.py` (4 testes, dois deles de regressão de B9 e B10).

src/pulseops/web/charts.py

```python
"""Gráfico de latência em SVG, montado no servidor a partir dos checks reais (ADR 0010)."""

from dataclasses import dataclass

from pulseops.services.queries import RESPONSE_OUTCOMES, LatencyPoint

WIDTH, HEIGHT = 720, 220
PAD_LEFT, PAD_RIGHT, PAD_TOP, PAD_BOTTOM = 56, 12, 12, 28
ACCEPTED = ("healthy", "slow")  # status aceito; o resto é desenhado como falha


@dataclass(frozen=True)
class ChartDot:
    x: float
    y: float
    outcome: str
    label: str
    failed: bool


@dataclass(frozen=True)
class LatencyChart:
    width: int
    height: int
    polyline: str
    dots: list[ChartDot]
    threshold_y: float
    y_ticks: list[tuple[float, str]]
    plot_left: int
    plot_right: int
    plot_bottom: int
    summary: str


def build_latency_chart(points: list[LatencyPoint], threshold_ms: int) -> LatencyChart | None:
    if not points:
        return None
    top_value = max(max(p.duration_ms for p in points), threshold_ms) * 1.15
    plot_w = WIDTH - PAD_LEFT - PAD_RIGHT
    plot_h = HEIGHT - PAD_TOP - PAD_BOTTOM
    bottom = PAD_TOP + plot_h

    def y_of(value: float) -> float:
        return round(bottom - value / top_value * plot_h, 1)

    step = plot_w / max(len(points) - 1, 1)
    dots = [
        ChartDot(
            x=round(PAD_LEFT + i * step, 1),
            y=y_of(p.duration_ms),
            outcome=p.outcome,
            label=f"{p.at.isoformat(timespec='seconds')} — {p.duration_ms} ms ({p.outcome})",
            failed=p.outcome not in ACCEPTED,
        )
        for i, p in enumerate(points)
```

```
56      ]
57      ticks = [(y_of(top_value * f), f"{top_value * f:.0f} ms") for f in (0, 0.25, 0.5, 0.75)]
58      # Mesma base da latência média e do p95 (RF05): checks que receberam resposta HTTP,
59      # inclusive com status fora da lista. Timeout e erro de conexão ficam fora do resumo,
60      # porque a duração deles mede o prazo ou a falha, não a resposta (B9, B10).
61      responses = [p.duration_ms for p in points if p.outcome in RESPONSE_OUTCOMES]
62      failures = sum(1 for d in dots if d.failed)
63      latency = (
64          f"latência das respostas HTTP: mínima {min(responses)} ms, máxima {max(responses)} ms"
65          if responses
66          else "nenhuma resposta HTTP"
67      )
68      summary = f"{len(points)} checks ({failures} com falha); {latency}; limite {threshold_ms} ms."
69      return LatencyChart(
70          width=WIDTH,
71          height=HEIGHT,
72          polyline=" ".join(f"{d.x},{d.y}" for d in dots),
73          dots=dots,
74          threshold_y=y_of(threshold_ms),
75          y_ticks=ticks,
76          plot_left=PAD_LEFT,
77          plot_right=WIDTH - PAD_RIGHT,
78          plot_bottom=bottom,
79          summary=summary,
80      )
```

- Linhas 7–9: tamanho do desenho e margens, em unidades do `viewBox` do SVG (o navegador escala). Linha 9: `ACCEPTED` decide o **formato** do ponto: círculo para status aceito e quadrado para falha.
- Linhas 12–32: classes de dados imutáveis (`frozen=True`). O template só lê.
- Linhas 36–37: sem pontos, sem gráfico (`None`); o template mostra “sem dados”.
- Linha 38: o topo do eixo é o maior valor entre os pontos e o limite, mais 15%, para a linha do limite sempre aparecer e o ponto mais alto não encostar na borda.
- Linhas 43–44: converte milissegundos em altura. No SVG, o `y` cresce para **baixo**, por isso `bottom - ...`.
- Linha 46: espaço horizontal igual entre pontos. `max(..., 1)` evita divisão por zero com um ponto só. **Consequência:** o eixo x é a **ordem** dos checks, não o tempo. Uma falta de checks de uma hora não aparece como espaço vazio no gráfico; ela aparece na lista de lacunas das métricas.
- Linhas 47–56: cada ponto com o texto de dica (`<title>` no SVG).
- Linha 57: quatro marcas no eixo y (0, 25%, 50% e 75% do topo).
- Linhas 58–68: o resumo em texto, que vai na `<desc>` do SVG (lida por leitores de tela) e na legenda. A latência mínima e a máxima usam `RESPONSE_OUTCOMES` (healthy, slow e unexpected_status), a **mesma base** da latência média e do p95 das métricas. Antes da correção B10, o resumo usava só `healthy` e `slow`, e as duas partes da mesma página descreviam conjuntos diferentes. Timeout e erro de conexão ficam fora, porque a duração deles mede o prazo, não uma resposta (B9).
- Linhas 69–80: a linha (`polyline`) liga todos os pontos na ordem.

Templates (`src/pulseops/web/templates/`)

São 11 arquivos HTML com Jinja2. Os padrões se repetem, então mostramos os quatro que definem a estrutura e resumimos os outros.

Testes. `tests/integration/test_dashboard.py` faz as requisições e confere o HTML devolvido; `test_monitor_name_is_escaped` e `test_sql_injection_attempts_are_plain_data` cadastram textos maliciosos e conferem que aparecem como texto.

src/pulseops/web/templates/base.html

```
 1 <!doctype html>
 2 <html lang="pt-BR">
 3 <head>
 4   <meta charset="utf-8">
 5   <meta name="viewport" content="width=device-width, initial-scale=1">
 6   <title>{% block title %}PulseOps{% endblock %} · PulseOps</title>
 7   <link rel="stylesheet" href="/static/style.css">
 8   <script src="/static/app.js" defer></script>
 9 </head>
10 <body>
11   <a class="skip-link" href="#conteudo">Pular para o conteúdo</a>
12   {% if demo_mode %}
13   <div class="demo-banner" role="note">
14     <strong>Modo demonstração.</strong> Intervalos curtos e alvos do laboratório local liberados. Não
   use em produção.
15   </div>
16   {% endif %}
17   <header class="topbar">
18     <a class="brand" href="/">PulseOps</a>
19     {% if username %}
20     <nav aria-label="Principal">
21       <ul>
22         <li><a href="/">Monitores</a></li>
23         <li><a href="/incidents">Incidentes</a></li>
24         <li><a href="/notifications">Notificações</a></li>
25       </ul>
26     </nav>
27     <form method="post" action="/logout" class="logout">
28       <input type="hidden" name="csrf_token" value="{{ csrf_token }}">
29       <span class="muted">{{ username }}</span>
30       <button type="submit" class="link-button">Sair</button>
31     </form>
32     {% endif %}
33   </header>
34   <main id="conteudo" tabindex="-1">
35     {% block content %}{% endblock %}
36   </main>
37   <footer class="footer muted">
38     Horários exibidos em {{ tz_name }} (armazenados em UTC).
39   </footer>
40 </body>
41 </html>
```

- Linha 2: `lang="pt-BR"`, para leitores de tela pronunciarem em português.
- Linha 8: `defer` executa o script depois de montar a página.
- Linha 11: o link “Pular para o conteúdo” fica escondido até receber foco pelo teclado (CSS `.skip-link`); permite pular o menu.
- Linhas 12–16: a faixa de modo demonstração, só quando `PULSEOPS_DEMO_MODE` está ligado (`test_demo_banner_only_in_demo_mode`).
- Linhas 19–32: menu e logout só para quem está logado. O logout é um **formulário `POST`** com token CSRF, e não um link: um link (`GET`) poderia ser disparado por uma imagem em outro site.
- Linha 34: `tabindex="-1"` permite que o link da linha 11 mova o foco para o conteúdo.
- Linha 38: o rodapé diz em que fuso os horários estão.

src/pulseops/web/templates/_macros.html

```
1 {% macro health_badge(health, stale=False) -%}
2   {%- if stale -%}
```

```
  <span class="badge badge-stale">Dados desatualizados</span>
  {%- else -%}
  <span class="badge badge-{{ health }}">{{ health | health_label }}</span>
  {%- endif -%}
{%- endmacro %}

{% macro outcome_badge(outcome) -%}
  <span class="badge badge-outcome-{{ outcome }}">{{ outcome | outcome_label }}</span>
{%- endmacro %}

{% macro incident_badge(status) -%}
  <span class="badge badge-incident-{{ status }}">{{ status | incident_label }}</span>
{%- endmacro %}

{% macro field_error(errors, name) -%}
  {%- if errors.get(name) -%}
  <p class="field-error" id="{{ name }}-error">{{ errors[name] }}</p>
  {%- endif -%}
{%- endmacro %}

{% macro action_button(url, label, csrf_token, style="secondary") -%}
<form method="post" action="{{ url }}" class="inline-form">
  <input type="hidden" name="csrf_token" value="{{ csrf_token }}">
  <button type="submit" class="button button-{{ style }}">{{ label }}</button>
</form>
{%- endmacro %}
```

- **Macros** são funções de template. Linhas 1–7: o selo de saúde; se os dados estão desatualizados, mostra isso **em vez** da saúde, que pode estar velha.
- Linhas 17–21: a mensagem de erro de um campo tem `id="<campo>-error"`. O campo do formulário aponta para ela com `aria-describedby`, e o leitor de tela lê a mensagem junto com o campo.
- Linhas 23–28: toda ação (pausar, arquivar…) é um formulário `POST` pequeno com o token CSRF.
- Os `-` em `{%-` e `-%}` removem espaços em branco em volta, para o HTML não ficar com quebras de linha sobrando dentro dos selos.

`src/pulseops/web/templates/dashboard.html, linhas 10–35`

```
{% if not worker.alive %}
<p class="alert alert-warning" role="alert">
  {% if worker.last_seen_at %}
  Worker sem sinal desde {{ worker.last_seen_at | dt }} (há {{ worker.age_seconds | int }} s).
  {% else %}
  Nenhum sinal do worker até agora.
  {% endif %}
  Os estados abaixo não estão sendo atualizados e aparecem como <strong>dados desatualizados</strong>.
</p>
{% endif %}

<section aria-labelledby="resumo-titulo" class="summary" data-summary-url="/api/v1/summary">
  <h2 id="resumo-titulo" class="visually-hidden">Resumo</h2>
  <div class="summary-grid">
    <div class="stat"><span class="stat-value" data-key="total">{{ summary.total }}</span><span class="stat-label">monitores</span></div>
    <div class="stat stat-healthy"><span class="stat-value" data-key="healthy">{{ summary.by_health.get('healthy', 0) }}</span><span class="stat-label">saudáveis</span></div>
    <div class="stat stat-degraded"><span class="stat-value" data-key="degraded">{{ summary.by_health.get('degraded', 0) }}</span><span class="stat-label">degradados</span></div>
    <div class="stat stat-down"><span class="stat-value" data-key="down">{{ summary.by_health.get('down', 0) }}</span><span class="stat-label">indisponíveis</span></div>
    <div class="stat"><span class="stat-value" data-key="no_data">{{ summary.by_health.get('no_data', 0) }}</span><span class="stat-label">sem dados</span></div>
    <div class="stat"><span class="stat-value" data-key="paused">{{ summary.by_health.get('paused', 0) }}</span><span class="stat-label">pausados</span></div>
    <div class="stat stat-stale"><span class="stat-value" data-key="stale">{{ summary.stale }}</span><span class="stat-label">desatualizados</span></div>
    <div class="stat"><span class="stat-value" data-key="open_incidents">{{ summary.open_incidents }}</span><span class="stat-label">incidentes abertos</span></div>
```

```
    <div class="stat"><span class="stat-value" data-key="failed_deliveries">{{ summary.failed_deliveries }}</span><span class="stat-label">alertas com falha</span></div>
  </div>
  <p class="muted small" aria-live="polite" data-summary-status>Resumo gerado no carregamento da página.</p>
</section>
```

- Linhas 10–19: o aviso de worker parado, com `role="alert"`, e há quanto tempo (RF07.5).
- Linhas 21–35: o resumo. `data-summary-url` e `data-key` são os ganchos do `app.js`; sem JavaScript, os números do carregamento continuam lá. Linha 34: `aria-live="polite"` faz o leitor de tela anunciar “Resumo atualizado” sem interromper a pessoa.

`src/pulseops/web/templates/monitor_detail.html, linhas 84–105`

```
<section class="card" aria-labelledby="grafico-titulo">
  <h2 id="grafico-titulo">Latência dos checks</h2>
  {% if chart %}
  <figure class="chart">
    <svg viewBox="0 0 {{ chart.width }} {{ chart.height }}" role="img" aria-labelledby="grafico-titulo grafico-desc" preserveAspectRatio="none">
      <desc id="grafico-desc">{{ chart.summary }}</desc>
      {% for y, label in chart.y_ticks %}
      <line x1="{{ chart.plot_left }}" x2="{{ chart.plot_right }}" y1="{{ y }}" y2="{{ y }}" class="grid"/>
      <text x="{{ chart.plot_left - 6 }}" y="{{ y + 4 }}" class="axis-label" text-anchor="end">{{ label }}</text>
      {% endfor %}
      <line x1="{{ chart.plot_left }}" x2="{{ chart.plot_right }}" y1="{{ chart.threshold_y }}" y2="{{ chart.threshold_y }}" class="threshold"/>
      <polyline points="{{ chart.polyline }}" class="series"/>
      {% for dot in chart.dots %}
      {% if dot.failed %}
      <rect x="{{ dot.x - 4 }}" y="{{ dot.y - 4 }}" width="8" height="8" class="dot dot-{{ dot.outcome }}"><title>{{ dot.label }}</title></rect>
      {% else %}
      <circle cx="{{ dot.x }}" cy="{{ dot.y }}" r="3" class="dot dot-{{ dot.outcome }}"><title>{{ dot.label }}</title></circle>
      {% endif %}
      {% endfor %}
    </svg>
    <figcaption class="muted small">{{ chart.summary }} Linha tracejada: limite de latência. Quadrados escuros são checks com falha (a altura é o tempo até a falha); círculos são respostas com status aceito; a tabela abaixo traz os mesmos dados em texto.</figcaption>
  </figure>
```

- Linha 88: `role="img"` e `aria-labelledby` apontando para o título e a descrição dão ao SVG um nome acessível. `preserveAspectRatio="none"` deixa o desenho esticar para a largura da tela.
- Linha 89: `<desc>` com o resumo de `charts.py`.
- Linhas 90–95: linhas de grade, rótulos do eixo, o limite tracejado e a série.
- Linhas 96–102: quadrado para falha e círculo para sucesso. A **forma** distingue os dois, não só a cor, o que serve a quem não distingue cores. `<title>` dá a dica com hora, duração e resultado.
- Linha 104: a legenda explica a leitura e lembra que a tabela abaixo traz os mesmos dados em texto.

Os demais templates:

Arquivo	O que mostra	Padrões a notar
`login.html`	formulário de login	campo escondido `csrf_token` com o token do cookie; `autocomplete` correto para gerenciadores de senha

Arquivo	O que mostra	Padrões a notar
`error.html`	título e mensagem de erro	`role="alert"`; nenhum detalhe interno
`monitor_form.html`	criar e editar monitor	`novalidate` deixa a validação para o servidor, que dá as mensagens; `aria-invalid` e `aria-describedby` nos campos com erro
`monitor_detail.html`	estado, métricas, gráfico, mudanças, incidentes, checks	abas da janela com `aria-current="page"`; ações pelo macro `action_button`
`incidents.html`	lista de incidentes	filtro por abas (`open`, `closed`, `all`) e paginação por cursor
`incident_detail.html`	um incidente	início versus confirmação, linha do tempo com motivos e os alertas
`notifications.html`	destinos e alertas	formulário de destino com erros por campo; botão de ligar e desligar com CSRF
`_alerts_table.html`	tabela de alertas com entregas e tentativas	incluído por `incident_detail.html` e `notifications.html`

`src/pulseops/web/static/app.js`

Finalidade. Atualiza os números do resumo do painel a cada 15 segundos. É o único JavaScript do projeto.

Relações. Carregado por `base.html`; lê `/api/v1/summary`. **Testes.** `test_javascript_has_no_token` confere que o arquivo não contém nenhum token. O comportamento no navegador não tem teste automático; foi conferido nas capturas de tela (`scripts/capturar_telas.mjs`, D9).

src/pulseops/web/static/app.js

```
// Atualiza o resumo do painel a cada 15 s. A página funciona sem este script.
// A requisição usa o cookie de sessão (HttpOnly); nenhum token fica neste arquivo.
(function () {
  "use strict";
  const section = document.querySelector("[data-summary-url]");
  if (!section) return;
  const status = section.querySelector("[data-summary-status]");
  const url = section.getAttribute("data-summary-url");

  function setValue(key, value) {
    const el = section.querySelector('[data-key="' + key + '"]');
    if (el) el.textContent = String(value);
  }

  async function refresh() {
    status.textContent = "Atualizando…";
    try {
      const response = await fetch(url, { credentials: "same-origin", headers: { Accept:
"application/json" } });
      if (!response.ok) throw new Error("HTTP " + response.status);
      const data = await response.json();
      setValue("total", data.total);
      for (const key of ["healthy", "degraded", "down", "no_data", "paused"]) {
        setValue(key, data.by_health[key] || 0);
      }
      setValue("stale", data.stale);
      setValue("open_incidents", data.open_incidents);
      setValue("failed_deliveries", data.failed_deliveries);
      const when = new Date(data.generated_at).toLocaleTimeString("pt-BR");
```

```
      status.textContent = "Resumo atualizado às " + when + "." +
        (data.worker_alive ? "" : " Worker sem sinal: dados desatualizados.");
    } catch (error) {
      status.textContent = "Falha ao atualizar o resumo (" + error.message + "). Recarregue a página.";
    }
  }

  setInterval(refresh, 15000);
})();
```

- Linhas 1–2: a página funciona **sem** este script (*melhoria progressiva*). A autenticação é o cookie `HttpOnly`, que o script nem consegue ler.
- Linha 3: uma função que se executa na hora (IIFE), para as variáveis não vazarem para o escopo global.
- Linha 6: fora do painel, não faz nada.
- Linhas 10–13: `textContent`, e não `innerHTML`: o valor é sempre tratado como texto, nunca como HTML.
- Linha 18: `credentials: "same-origin"` envia o cookie de sessão. É um `GET`, então não precisa de token CSRF.
- Linhas 28–30: mostra a hora da atualização e avisa se o worker parou.
- Linhas 31–33: em erro (sessão expirou, servidor fora), diz o que houve em vez de falhar em silêncio.
- Linha 36: `setInterval` de 15 s.

`src/pulseops/web/static/style.css`

Finalidade. O visual do painel: 136 linhas, sem *framework*. A primeira linha resume as regras: contraste AA, foco visível e cor sempre acompanhada de texto.

src/pulseops/web/static/style.css, linhas 1–34

```
/* PulseOps: estilo sóbrio, contraste AA e foco visível. Cores sempre acompanhadas de texto. */
:root {
  --bg: #f6f7f9;
  --surface: #ffffff;
  --text: #1c2230;
  --muted: #545d6e;
  --border: #d5d9e0;
  --primary: #1f4fbf;
  --primary-text: #ffffff;
  --danger: #a1231b;
  --focus: #f0b400;
  --healthy-bg: #e3f4e8; --healthy-fg: #1d5e33;
  --degraded-bg: #fff3d6; --degraded-fg: #7a4f00;
  --down-bg: #fde4e1; --down-fg: #8c1d14;
  --neutral-bg: #eceef2; --neutral-fg: #3c4454;
  --stale-bg: #ede3f6; --stale-fg: #5a2a82;
}

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  font-size: 16px;
  line-height: 1.5;
  color: var(--text);
  background: var(--bg);
}
a { color: var(--primary); }
code { font-family: ui-monospace, SFMono-Regular, Menlo, monospace; font-size: 0.9em; }

```

```
31 :focus-visible { outline: 3px solid var(--focus); outline-offset: 2px; }
32 .skip-link { position: absolute; left: -999px; top: 0; background: var(--surface); padding: .5rem 1rem;
   }
33 .skip-link:focus { left: 1rem; z-index: 10; }
34 .visually-hidden { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0);
   white-space: nowrap; }
```

- Linhas 2–17: as cores como **variáveis CSS** (`--nome`). Cada estado tem um par fundo e texto; o comentário da linha 1 promete contraste AA (conferido abaixo).
- Conferência do contraste, calculada com a fórmula da WCAG sobre as cores deste trecho: os pares dos selos ficam entre 6,5:1 (degradado) e 8,4:1 (neutro); o texto principal sobre o fundo dá 14,8:1 e o texto secundário (`--muted`), 6,2:1. Todos passam do mínimo AA para texto normal, 4,5:1.
- Linha 31: `:focus-visible` mostra um contorno amarelo grosso em quem recebe foco pelo teclado.
- Linhas 32–33: o link de pular conteúdo fica fora da tela até receber foco.
- Linha 34: `.visually-hidden` esconde da tela, mas não do leitor de tela (usado nos títulos do resumo e da tabela).

`src/pulseops/web/static/style.css, linhas 118–136`

```
118 .chart svg { width: 100%; height: 240px; display: block; }
119 .chart .grid { stroke: var(--border); stroke-width: 1; }
120 .chart .axis-label { font-size: 11px; fill: var(--muted); }
121 .chart .threshold { stroke: var(--degraded-fg); stroke-dasharray: 6 4; stroke-width: 1.5; }
122 .chart .series { fill: none; stroke: var(--primary); stroke-width: 1.5; }
123 .chart .dot { fill: var(--primary); }
124 .chart .dot-slow { fill: var(--degraded-fg); }
125 .chart .dot-unexpected_status, .chart .dot-timeout, .chart .dot-connection_error,
126 .chart .dot-tls_error, .chart .dot-blocked { fill: var(--down-fg); stroke: #000; stroke-width: 1; }
127
128 @media (max-width: 720px) {
129   /* A tabela rola na horizontal em vez de espremer as colunas em uma palavra por linha. */
130   .data-table { display: block; overflow-x: auto; }
131   .data-table td, .data-table th { min-width: 7.5rem; }
132   .data-table td:first-child { min-width: 11rem; }
133   .data-table td.reason { min-width: 16rem; }
134   .topbar { gap: .5rem 1rem; padding: .5rem 1rem; }
135   main { padding: 1rem; }
136 }
```

- Linhas 118–126: o gráfico. A linha do limite é tracejada; os pontos de falha têm cor escura e borda preta, além da forma quadrada.
- Linhas 128–136: em telas estreitas, a tabela rola na horizontal em vez de espremer as colunas.

O restante do arquivo (linhas 35–117) define o cabeçalho, os cartões, os botões, os selos (`.badge-*`), os formulários e as tabelas, sem lógica para explicar.

D9 — Laboratório de falhas e scripts

O laboratório tem três peças: um **alvo** que obedece a comandos (fica lento, dá erro, demora demais), um **receptor** de webhooks que pode "cair" e um **roteiro** que provoca as falhas e confere, passo a passo, o que o PulseOps fez. Tudo roda dentro da rede do Compose; nada disso é publicado no seu computador.

`lab/__init__.py`

Uma linha: a docstring que diz para que serve o pacote. Existe para `uvicorn lab.target:app` encontrar o módulo.

`lab/target.py`

Finalidade. O alvo controlável dos monitores da demonstração.

Dados. Recebe o modo por `POST /control/mode`. Responde em `/service` conforme o modo e conta as requisições. **Relações.** Serviço `lab-target` do `compose.demo.yaml`; o roteiro `scripts/demo_flow.py` troca o modo. **Testes.** Não há teste unitário; ele é exercitado pelo roteiro inteiro, que roda no job `docker` do CI.

lab/target.py

```python
"""Alvo controlável do laboratório.

GET /service responde conforme o modo atual:
- healthy: 200 imediato
- slow: 200 depois de SLOW_SECONDS (acima do limite de latência da demo)
- error: 500
- timeout: espera TIMEOUT_SECONDS (mais que o timeout da demo)

GET /stable responde sempre 200 (segundo monitor da demo, que nunca falha).

POST /control/mode {"mode": "..."} troca o modo. Só existe dentro da rede do Compose.
"""

import asyncio
from typing import Literal

from fastapi import FastAPI
from fastapi.responses import PlainTextResponse
from pydantic import BaseModel

Mode = Literal["healthy", "slow", "error", "timeout"]
SLOW_SECONDS = 1.5
TIMEOUT_SECONDS = 30.0

app = FastAPI(title="PulseOps lab target", docs_url=None, redoc_url=None, openapi_url=None)
app.state.mode = "healthy"
app.state.requests = 0


class ModeIn(BaseModel):
    mode: Mode


@app.api_route("/service", methods=["GET", "HEAD"])
async def service() -> PlainTextResponse:
    app.state.requests += 1
```

```
    mode = app.state.mode
    if mode == "slow":
        await asyncio.sleep(SLOW_SECONDS)
    elif mode == "timeout":
        await asyncio.sleep(TIMEOUT_SECONDS)
    elif mode == "error":
        return PlainTextResponse("erro simulado", status_code=500)
    return PlainTextResponse("ok")


@app.api_route("/stable", methods=["GET", "HEAD"])
async def stable() -> PlainTextResponse:
    return PlainTextResponse("ok")


@app.post("/control/mode")
async def set_mode(body: ModeIn) -> dict[str, str]:
    app.state.mode = body.mode
    return {"mode": body.mode}


@app.get("/control/state")
async def get_state() -> dict[str, object]:
    return {"mode": app.state.mode, "requests": app.state.requests}
```

- Linhas 1–12: a docstring descreve os quatro modos e os endpoints.
- Linhas 21–23: `Literal` restringe o modo a quatro textos (um valor diferente dá 422). 1,5 s fica acima do limite de lentidão da demonstração (800 ms) e abaixo do timeout (2 s), então o resultado é `slow`. 30 s fica muito acima do timeout, então o resultado é `timeout`.
- Linhas 25–27: o estado fica em `app.state`, na memória do processo. Reiniciar o contêiner volta ao modo `healthy`.
- Linhas 34–44: `api_route` aceita `GET` e `HEAD`, os dois métodos que um monitor pode usar. `asyncio.sleep` espera sem bloquear o processo, então outras requisições continuam sendo atendidas durante a espera.
- Linhas 47–49: o monitor de comparação, que nunca falha.
- Linhas 52–60: os controles. Não há autenticação: só são alcançáveis de dentro da rede do Compose, porque o serviço não publica portas.

Falhas possíveis. Se o contêiner não subir, o `healthcheck` (que consulta `/control/state`) falha, e o `demo-runner` não começa.

`lab/receiver.py`

Finalidade. Um receptor de webhooks que se comporta como um receptor real deveria: **deduplica** pelo id do evento e pode ser posto “fora do ar”.

Dados. Recebe `POST /webhook` com o cabeçalho `X-PulseOps-Event-Id`. Guarda os eventos na memória. **Relações.** Serviço `lab-receiver`; destino de alerta criado pelo roteiro. **Testes.** `tests/unit/test_lab_receiver.py` (`test_duplicate_event_is_detected`, `test_unavailable_receiver_rejects_then_accepts`, `test_missing_event_id_is_rejected`).

lab/receiver.py

```
"""Receptor de webhooks do laboratório, com deduplicação por X-PulseOps-Event-Id.

POST /webhook: se `available` for falso, responde 503 (simula destino fora do ar).
Eventos repetidos (mesmo id) são aceitos com 200 mas contados como duplicados,
que é o que um receptor real deve fazer com entrega "pelo menos uma vez".
```

```
"""

from typing import Any

from fastapi import FastAPI, Header, Request
from fastapi.responses import JSONResponse
from pydantic import BaseModel

app = FastAPI(title="PulseOps lab receiver", docs_url=None, redoc_url=None, openapi_url=None)


class Inbox:
    def __init__(self) -> None:
        self.available = True
        self.events: dict[str, dict[str, Any]] = {}
        self.order: list[str] = []
        self.duplicates = 0
        self.rejected = 0

    def receive(self, event_id: str, payload: dict[str, Any]) -> bool:
        """Guarda o evento. Devolve False se o id já tinha sido recebido."""
        if event_id in self.events:
            self.duplicates += 1
            return False
        self.events[event_id] = payload
        self.order.append(event_id)
        return True


inbox = Inbox()


class AvailabilityIn(BaseModel):
    available: bool


@app.post("/webhook")
async def webhook(
    request: Request, x_pulseops_event_id: str | None = Header(default=None)
) -> JSONResponse:
    if not inbox.available:
        inbox.rejected += 1
        return JSONResponse({"detail": "receptor indisponível (simulado)"}, status_code=503)
    if not x_pulseops_event_id:
        return JSONResponse({"detail": "X-PulseOps-Event-Id ausente"}, status_code=400)
    payload = await request.json()
    new = inbox.receive(x_pulseops_event_id, payload)
    return JSONResponse({"duplicate": not new})


@app.post("/control/availability")
async def set_availability(body: AvailabilityIn) -> dict[str, bool]:
    inbox.available = body.available
    return {"available": inbox.available}


@app.get("/control/received")
async def received() -> dict[str, Any]:
    return {
        "available": inbox.available,
        "event_ids": inbox.order,
        "types": [inbox.events[i].get("type") for i in inbox.order],
        "duplicates": inbox.duplicates,
        "rejected": inbox.rejected,
    }
```

- Linhas 17–32: a classe `Inbox` guarda os eventos por id. `receive` devolve `False` quando o id já foi visto, e conta a duplicata. É o lado do receptor da entrega “pelo menos uma vez” (ADR 0007): o PulseOps pode reenviar, e quem recebe ignora o repetido.

- Linhas 46–48: indisponível responde 503 e conta a recusa. O PulseOps trata 503 como erro temporário e tenta de novo.
- Linhas 49–50: sem o id, 400. Um erro 4xx é permanente para o PulseOps, que não tentaria de novo; no laboratório isso nunca acontece, porque o PulseOps sempre manda o cabeçalho.
- Linhas 51–53: aceita com 200 mesmo quando é duplicado, e diz isso na resposta. Responder erro a um duplicado faria o remetente tentar de novo à toa.
- Linhas 62–70: o que o roteiro consulta para conferir as entregas.

`scripts/demo.sh`

Finalidade. O comando único do laboratório: sobe o ambiente, garante a operadora da demonstração e executa o roteiro.

Relações. Usa `compose.yaml` e `compose.demo.yaml`; é o passo principal do job `docker` do CI. **Testes.** O próprio CI.

scripts/demo.sh

```
 1 #!/usr/bin/env bash
 2 # Laboratório de falhas do PulseOps: sobe o ambiente com o perfil "demo" e executa
 3 # o roteiro de 11 passos (scripts/demo_flow.py). Pode ser executado várias vezes.
 4 #
 5 # Uso: scripts/demo.sh           (sobe, prepara e executa o roteiro)
 6 #      scripts/demo.sh --down    (derruba o ambiente da demonstração, mantendo o banco)
 7 set -euo pipefail
 8
 9 cd "$(dirname "$0")/.."
10 COMPOSE=(docker compose -f compose.yaml -f compose.demo.yaml --profile demo)
11
12 if [[ "${1:-}" == "--down" ]]; then
13   "${COMPOSE[@]}" down
14   exit 0
15 fi
16
17 # Senha da operadora da demonstração: gerada uma vez e guardada fora do Git
18 # (.demo/ está no .gitignore), para que reexecutar o script reutilize o mesmo usuário.
19 mkdir -p .demo
20 if [[ ! -s .demo/admin-password ]]; then
21   python3 -c 'import secrets; print(secrets.token_urlsafe(18))' > .demo/admin-password
22   chmod 600 .demo/admin-password
23 fi
24 export PULSEOPS_DEMO_USERNAME="${PULSEOPS_DEMO_USERNAME:-demo}"
25 PULSEOPS_DEMO_PASSWORD="$(cat .demo/admin-password)"
26 export PULSEOPS_DEMO_PASSWORD
27
28 echo "==> Subindo banco, API, worker e serviços do laboratório"
29 "${COMPOSE[@]}" up -d --build --wait db migrate api worker lab-target lab-receiver
30
31 echo "==> Garantindo a operadora '${PULSEOPS_DEMO_USERNAME}'"
32 printf '%s\n' "$PULSEOPS_DEMO_PASSWORD" | "${COMPOSE[@]}" run --rm -T --no-deps api \
33   pulseops create-admin --username "$PULSEOPS_DEMO_USERNAME" --password-stdin --exists-ok
34
35 echo "==> Executando o roteiro"
36 "${COMPOSE[@]}" run --rm --no-deps demo-runner
37
38 echo
39 echo "Painel: http://127.0.0.1:${PULSEOPS_PORT:-8000}  (usuário: ${PULSEOPS_DEMO_USERNAME}, senha em
   .demo/admin-password)"
```

- Linha 7: `set -euo pipefail`: para no primeiro erro (`-e`), trata variável não definida como erro (`-u`) e considera falha de qualquer comando num *pipe* (`pipefail`).
- Linha 9: muda para a raiz do projeto, qualquer que seja a pasta de onde o script foi chamado.

- Linha 10: o comando do Compose num *array* do Bash, para não repetir os arquivos e o perfil. `"${COMPOSE[@]}"` expande cada item como um argumento separado.
- Linhas 12–15: `--down` derruba os contêineres sem `-v`, então o volume do banco é mantido.
- Linhas 17–23: a senha da operadora é gerada **uma vez**, com 18 bytes aleatórios, e guardada em `.demo/admin-password`, que está no `.gitignore`. `chmod 600`: só o dono lê. Rodar de novo reaproveita a mesma senha.
- Linha 29: `--wait` espera todos os *healthchecks* passarem antes de seguir.
- Linhas 31–33: a senha vai pela entrada padrão (`--password-stdin`), não como argumento, porque argumentos aparecem na lista de processos. `--exists-ok` torna o passo repetível.
- Linha 36: roda o roteiro num contêiner de vida curta (`--rm`).

`scripts/demo_flow.py`

Finalidade. O roteiro de 11 passos. Cada passo provoca algo e **confere** o resultado pela API, com prazo. Se uma conferência falhar, imprime `FALHOU:` e sai com código 1.

Dados. Variáveis `PULSEOPS_DEMO_*` (endereços, usuário e senha). Produz a saída no terminal e o código de saída. **Relações.** Executado pelo serviço `demo-runner` (`compose.demo.yaml`). Usa a API JSON e os controles do laboratório. **Testes.** Não há teste automatizado *do roteiro*; ele **é** um teste de ponta a ponta, executado no CI. O mypy confere os tipos (`files` inclui `scripts`).

scripts/demo_flow.py, linhas 20–55

```
20 T = TypeVar("T")
21 TOTAL_STEPS = 11
22 CSRF_FIELD = re.compile(r'name="csrf_token" value="([^"]+)"')
23
24 MAIN: dict[str, Any] = {
25     "name": "Lab: serviço principal",
26     "description": "Alvo controlado pelo roteiro da demonstração.",
27     "url": "http://lab-target:8081/service",
28     "method": "GET",
29     "interval_seconds": 3,
30     "timeout_seconds": 2,
31     "accepted_status_codes": "200-299",
32     "latency_threshold_ms": 800,
33     "failure_threshold": 3,
34     "recovery_threshold": 2,
35 }
36 STABLE: dict[str, Any] = {
37     **MAIN,
38     "name": "Lab: serviço estável",
39     "description": "Responde sempre 200; serve de comparação no painel.",
40     "url": "http://lab-target:8081/stable",
41     "interval_seconds": 5,
42 }
43 DESTINATION = {"name": "Receptor do laboratório", "url": "http://lab-receiver:8082/webhook"}
44
45
46 class StepFailedError(Exception):
47     pass
48
49
50 class Demo:
51     def __init__(self, api: str, target: str, receiver: str) -> None:
52         self.api = httpx.Client(base_url=api, timeout=10)
53         self.target = httpx.Client(base_url=target, timeout=10)
54         self.receiver = httpx.Client(base_url=receiver, timeout=10)
55         self.csrf = ""
```

- Linha 22: uma expressão regular extrai o token CSRF do HTML. É frágil se o HTML mudar, mas o roteiro falha com uma mensagem clara nesse caso (linhas 88–89).
- Linhas 24–42: os dois monitores. O principal verifica a cada 3 s, com timeout de 2 s e limite de 800 ms. O estável repete a configuração (`**MAIN`) e troca nome, URL e intervalo.
- Linhas 51–55: três clientes HTTP **síncronos** (`httpx.Client`). Um roteiro sequencial não ganha nada com `async`.

scripts/demo_flow.py, linhas 67–112

```
67      def expect(self, condition: bool, message: str) -> None:
68          if not condition:
69              raise StepFailedError(message)
70          self.ok(message)
71
72      @staticmethod
73      def wait_for(description: str, probe: Callable[[], T | None], timeout: float) -> T:
74          print(f"    aguardando: {description} (até {timeout:.0f} s)", flush=True)
75          deadline = time.monotonic() + timeout
76          while time.monotonic() < deadline:
77              value = probe()
78              if value:
79                  return value
80              time.sleep(0.5)
81          raise StepFailedError(f"tempo esgotado esperando: {description}")
82
83      # ---- API do PulseOps ----------------------------------------------------------
84
85      def login(self, username: str, password: str) -> None:
86          page = self.api.get("/login")
87          token = CSRF_FIELD.search(page.text)
88          if not token:
89              raise StepFailedError("página de login sem token CSRF")
90          response = self.api.post(
91              "/login",
92              data={"username": username, "password": password, "csrf_token": token.group(1)},
93          )
94          if response.status_code != 303:
95              raise StepFailedError(f"login recusado (HTTP {response.status_code})")
96          token = CSRF_FIELD.search(self.api.get("/").text)
97          if not token:
98              raise StepFailedError("painel sem token CSRF depois do login")
99          self.csrf = token.group(1)
100
101     def get(self, path: str, **params: Any) -> Any:
102         response = self.api.get(f"/api/v1{path}", params=params)
103         response.raise_for_status()
104         return response.json()
105
106     def send(self, method: str, path: str, body: dict[str, Any] | None = None) -> Any:
107         response = self.api.request(
108             method, f"/api/v1{path}", json=body, headers={"X-CSRF-Token": self.csrf}
109         )
110         if response.status_code >= 400:
111             raise StepFailedError(f"{method} {path}: HTTP {response.status_code} {response.text}")
112         return response.json()
```

- Linhas 67–70: `expect` é o `assert` do roteiro: falha com mensagem ou imprime `ok:`.
- Linhas 72–81: `wait_for` chama a função `probe` a cada 0,5 s até ela devolver algo verdadeiro ou o prazo acabar. `time.monotonic()` é um relógio que nunca volta atrás, mesmo que o horário do sistema seja ajustado. `TypeVar` faz o tipo de saída ser o mesmo que a função devolve.
- Linhas 85–99: login como um navegador faria: abre `/login` (o cliente guarda o cookie de CSRF do login), envia o formulário com o token e espera o 303. Depois lê o token CSRF da sessão no painel.

- Linhas 106–112: toda chamada que altera dados leva o cabeçalho `X-CSRF-Token`.

scripts/demo_flow.py, linhas 118–134

```python
    def ensure_monitor(self, spec: dict[str, Any], *, reset: bool) -> dict[str, Any]:
        found = self.monitors_named(spec["name"])
        if not found:
            created: dict[str, Any] = self.send("POST", "/monitors", spec)
            self.ok(f"monitor criado: {spec['name']} (id {created['id']})")
            return created
        monitor = found[0]
        self.send("PUT", f"/monitors/{monitor['id']}", spec)
        if monitor["state"] == "active" and reset:
            # Pausar fecha um incidente que tenha sobrado de execução anterior e zera a
            # contagem; retomar agenda a primeira verificação para já.
            self.send("POST", f"/monitors/{monitor['id']}/pause")
        if monitor["state"] == "paused" or reset:
            self.send("POST", f"/monitors/{monitor['id']}/resume")
        self.ok(f"monitor já existia e foi reaproveitado: {spec['name']} (id {monitor['id']})")
        refreshed: dict[str, Any] = self.get(f"/monitors/{monitor['id']}")
        return refreshed
```

- O roteiro é **idempotente**: pode rodar várias vezes sem duplicar nada. Procura pelo nome; se existe, atualiza (`PUT`). Linhas 126–131: pausar e retomar zera a contagem e fecha um incidente que tenha ficado aberto de uma execução anterior, para o roteiro começar do zero.

scripts/demo_flow.py, linhas 179–223

```python
def run(demo: Demo, username: str, password: str) -> None:
    demo.step(1, "Ambiente no ar: API pronta, banco acessível e worker ativo")
    ready = demo.api.get("/health/ready")
    demo.expect(ready.status_code == 200, f"/health/ready respondeu {ready.status_code}")
    demo.login(username, password)
    demo.ok(f"login como '{username}'")
    demo.wait_for("worker enviar sinal de vida", lambda: demo.get("/summary")["worker_alive"], 30)
    demo.ok("worker ativo segundo /api/v1/summary")

    demo.step(2, "Preparar monitores sem duplicar (idempotente)")
    demo.set_target("healthy")
    demo.set_receiver(True)
    demo.ensure_destination()
    main = demo.ensure_monitor(MAIN, reset=True)
    demo.ensure_monitor(STABLE, reset=False)
    for spec in (MAIN, STABLE):
        count = len(demo.monitors_named(spec["name"]))
        demo.expect(
            count == 1, f"existe exatamente 1 monitor '{spec['name']}' (encontrados {count})"
        )
    mid = main["id"]
    baseline = demo.last_check_id(mid)

    demo.step(3, "Verificações saudáveis")
    demo.wait_for(
        "2 checks com sucesso e saúde 'healthy'",
        lambda: (
            sum(c["outcome"] == "healthy" for c in demo.checks_after(mid, baseline)) >= 2
            and demo.get(f"/monitors/{mid}")["health"] == "healthy"
        ),
        30,
    )
    demo.ok(f"motivo exibido: {demo.get(f'/monitors/{mid}')['health_reason']}")
    demo.set_target("slow")
    mark = demo.last_check_id(mid)
    slow = demo.wait_for(
        "um check lento (acima de 800 ms, abaixo do timeout de 2 s)",
        lambda: [c for c in demo.checks_after(mid, mark) if c["outcome"] == "slow"],
        20,
    )
    monitor = demo.get(f"/monitors/{mid}")
```

```
220     demo.expect(
221         monitor["health"] == "degraded" and not demo.open_incidents(mid),
222         f"lentidão ({slow[0]['duration_ms']} ms) deixa o monitor degradado sem abrir incidente",
223     )
```

- Passo 1 (linhas 180–186): `/health/ready`, login e sinal de vida do worker.
- Passo 2 (188–200): prepara tudo e confere que existe **exatamente um** monitor de cada nome. Linha 200: `baseline` é o id do último check; os passos seguintes só olham checks mais novos que a marca, para não confundir com execuções anteriores.
- Passo 3 (202–223): dois checks saudáveis; depois, modo lento e a conferência de que lentidão **degrada sem abrir incidente** (ADR 0004).

scripts/demo_flow.py, linhas 225–268

```
225     demo.step(4, "Provocar uma falha (timeout)")
226     demo.set_target("timeout")
227     mark = demo.last_check_id(mid)
228     demo.wait_for(
229         "um check com resultado 'timeout'",
230         lambda: [c for c in demo.checks_after(mid, mark) if c["outcome"] == "timeout"],
231         20,
232     )
233     monitor = demo.get(f"/monitors/{mid}")
234     demo.expect(not demo.open_incidents(mid), "uma falha isolada ainda não abre incidente")
235     demo.ok(f"motivo exibido: {monitor['health_reason']}")
236
237     demo.step(5, "Limiar de abertura: 3 falhas consecutivas confirmam o incidente")
238     demo.set_target("error")
239     incident = demo.wait_for("incidente aberto", lambda: demo.open_incidents(mid), 40)[0]
240     demo.expect(
241         incident["opening_reason"].startswith("Incidente confirmado após 3 falhas consecutivas"),
242         f"motivo de abertura: {incident['opening_reason']}",
243     )
244     detail = demo.get(f"/incidents/{incident['id']}")
245     kinds = [e["kind"] for e in detail["events"]]
246     demo.expect(
247         kinds[:2] == ["first_failure", "opened"],
248         f"linha do tempo começa com primeira falha e confirmação: {kinds}",
249     )
250     demo.wait_for(
251         "alerta incident.opened chegar ao receptor",
252         lambda: f"incident:{incident['id']}:opened" in demo.received()["event_ids"],
253         20,
254     )
255     demo.ok("receptor recebeu incident.opened")
256
257     demo.step(6, "Falhas seguintes não abrem outro incidente")
258     mark = demo.last_check_id(mid)
259     demo.wait_for(
260         "mais 2 checks com falha",
261         lambda: sum(c["outcome"] == "unexpected_status" for c in demo.checks_after(mid, mark)) >= 2,
262         20,
263     )
264     still_open = demo.open_incidents(mid)
265     demo.expect(
266         len(still_open) == 1 and still_open[0]["id"] == incident["id"],
267         f"existe apenas UM incidente aberto (id {incident['id']})",
268     )
```

- Passo 4: um timeout isolado não abre incidente.
- Passo 5: modo erro (500). Espera o incidente abrir e confere o motivo (“Incidente confirmado após 3 falhas consecutivas”) e o começo da linha do tempo (`first_failure` e depois `opened`). Depois confere que o alerta chegou ao receptor pelo id estável `incident:<id>:opened`.
- Passo 6: mais duas falhas e ainda **um só** incidente aberto, com o mesmo id.

scripts/demo_flow.py, linhas 270–292

```
270    demo.step(7, "Restaurar o alvo")
271    demo.set_target("healthy")
272    mark = demo.last_check_id(mid)
273    demo.wait_for(
274        "1 check com sucesso",
275        lambda: [c for c in demo.checks_after(mid, mark) if c["outcome"] == "healthy"],
276        20,
277    )
278    monitor = demo.get(f"/monitors/{mid}")
279    if demo.open_incidents(mid):
280        demo.ok(f"um sucesso ainda não encerra: {monitor['health_reason']}")
281
282    demo.step(8, "Recuperação: 2 sucessos consecutivos encerram o incidente")
283    demo.wait_for("incidente resolvido", lambda: not demo.open_incidents(mid), 20)
284    resolved = demo.get(f"/incidents/{incident['id']}")["incident"]
285    demo.expect(resolved["status"] == "resolved", f"motivo: {resolved['resolution_explanation']}")
286    demo.expect(demo.get(f"/monitors/{mid}")["health"] == "healthy", "monitor voltou a 'healthy'")
287    demo.wait_for(
288        "alerta incident.resolved chegar ao receptor",
289        lambda: f"incident:{incident['id']}:resolved" in demo.received()["event_ids"],
290        20,
291    )
292    demo.ok("receptor recebeu incident.resolved")
```

- Passo 7: alvo restaurado e um sucesso. **Ponto de atenção:** a linha 279 é um `if`, não um `expect`. Se a consulta acontecer depois do segundo sucesso, o incidente já terá sido encerrado e o passo só não imprime a frase. Isso evita uma falha falsa por causa do tempo, mas significa que o passo 7, sozinho, não **prova** que um sucesso não encerra. Quem prova é o passo 8 (encerra com 2) e os testes de `test_health.py`.
- Passo 8: incidente resolvido, monitor `healthy` e alerta `resolved` entregue.

scripts/demo_flow.py, linhas 294–352

```
294    demo.step(9, "Indisponibilizar o receptor de alertas e provocar novo incidente")
295    demo.set_receiver(False)
296    rejected_before = demo.received()["rejected"]
297    demo.set_target("error")
298    second = demo.wait_for("novo incidente aberto", lambda: demo.open_incidents(mid), 40)[0]
299    event_id = f"incident:{second['id']}:opened"
300
301    def failed_delivery() -> dict[str, Any] | None:
302        for alert in demo.get(f"/incidents/{second['id']}")["alerts"]:
303            for delivery in alert["deliveries"]:
304                if alert["event_id"] == event_id and delivery["attempt_count"] >= 1:
305                    return dict(delivery)
306        return None
307
308    delivery = demo.wait_for("tentativa de entrega recusada pelo receptor", failed_delivery, 20)
309    demo.expect(
310        delivery["status"] == "pending" and "503" in (delivery["last_error"] or ""),
311        f"entrega pendente, último erro: {delivery['last_error']}",
312    )
313    demo.expect(demo.received()["rejected"] > rejected_before, "receptor registrou a recusa")
314
315    demo.step(10, "O monitoramento continua com o receptor fora do ar")
316    mark = demo.last_check_id(mid)
317    demo.wait_for("mais 2 checks gravados", lambda: len(demo.checks_after(mid, mark)) >= 2, 20)
318    demo.expect(
319        event_id not in demo.received()["event_ids"],
320        "checks continuam sendo gravados enquanto o alerta espera",
321    )
322
323    demo.step(11, "Restaurar o receptor e ver a retentativa entregar o alerta")
324    demo.set_receiver(True)
325    demo.wait_for(
326        "entrega concluída",
```

```
        lambda: event_id in demo.received()["event_ids"],
        30,
    )
    final = next(
        d
        for a in demo.get(f"/incidents/{second['id']}")["alerts"]
        if a["event_id"] == event_id
        for d in a["deliveries"]
    )
    demo.expect(
        final["status"] == "delivered" and final["attempt_count"] >= 2,
        f"entregue na tentativa {final['attempt_count']}",
    )
    # Entrega "pelo menos uma vez": se o mesmo evento chegar de novo, o receptor o
    # reconhece pelo id e não processa duas vezes.
    payload = demo.received()
    repost = demo.receiver.post(
        "/webhook", json={"type": "incident.opened"}, headers={"X-PulseOps-Event-Id": event_id}
    ).json()
    after = demo.received()
    demo.expect(
        repost["duplicate"] and after["duplicates"] == payload["duplicates"] + 1,
        "reenvio do mesmo evento foi reconhecido como duplicado",
    )
    demo.set_target("healthy")
    print("\nDemonstração concluída: os 11 passos foram verificados.", flush=True)
```

- Passo 9: receptor fora do ar e um novo incidente. Linhas 301–306: procura a entrega do alerta que já teve ao menos uma tentativa. Linhas 309–313: a entrega está `pending` (vai tentar de novo), o erro registrado contém 503 e o receptor contou a recusa.
- Passo 10: com o alerta parado, o monitoramento **continua**: mais dois checks são gravados (RF06.3).
- Passo 11: receptor de volta. A retentativa entrega (status `delivered`, pelo menos 2 tentativas). Linhas 340–350: o roteiro reenvia o **mesmo** evento ao receptor e confere que ele foi reconhecido como duplicado. Repare que quem reenvia é o roteiro, não o PulseOps: o passo mostra a deduplicação do lado do receptor.

`scripts/demo_flow.py, linhas 355–374`

```
def main() -> int:
    password = os.environ.get("PULSEOPS_DEMO_PASSWORD", "")
    if not password:
        print("erro: defina PULSEOPS_DEMO_PASSWORD (o scripts/demo.sh faz isso)", file=sys.stderr)
        return 2
    demo = Demo(
        os.environ.get("PULSEOPS_DEMO_API", "http://api:8000"),
        os.environ.get("PULSEOPS_DEMO_TARGET", "http://lab-target:8081"),
        os.environ.get("PULSEOPS_DEMO_RECEIVER", "http://lab-receiver:8082"),
    )
    try:
        run(demo, os.environ.get("PULSEOPS_DEMO_USERNAME", "demo"), password)
    except (StepFailedError, httpx.HTTPError) as exc:
        print(f"\nFALHOU: {exc}", file=sys.stderr, flush=True)
        return 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

- Linhas 356–359: sem senha, sai com código 2 (erro de uso), diferente do 1 (passo falhou).
- Linhas 365–369: `StepFailedError` e erros de HTTP viram `FALHOU: ...` e código 1. Qualquer outra exceção aparece com *traceback*, o que é o desejado para um bug do próprio roteiro.

`scripts/ensaio_carga.py`

Finalidade. O ensaio de carga local: cria N monitores, deixa o worker rodar por um tempo fixo enquanto clientes consultam a API, e mede o que aconteceu. Foi com ele que o bug B8 apareceu.

Dados. Argumentos `--monitors` (200), `--interval` (10 s), `--duration` (120 s) e `--api-clients` (4). Produz um relatório em Markdown na saída padrão. **Relações.** Chamado por `scripts/ensaio_carga.sh` dentro do `demo-runner`. Os resultados estão em `docs/evidencias/ensaio-de-carga.md` e `docs/evidencias/ensaio-de-carga-antes-B8.md`. **Testes.** Não há; é uma ferramenta de medição.

scripts/ensaio_carga.py, linhas 32–36

```python
def percentile(values: list[float], p: float) -> float:
    """Nearest-rank, o mesmo critério do p95 do produto."""
    ordered = sorted(values)
    rank = max(1, -(-len(ordered) * p // 100))
    return ordered[int(rank) - 1]
```

- O percentil *nearest-rank*, o mesmo critério do p95 do produto (volume 1, parte F). `-(-a // b)` é um truque para arredondar a divisão **para cima** só com divisão inteira.

scripts/ensaio_carga.py, linhas 112–151

```python
async def main_async(args: argparse.Namespace) -> int:
    api = os.environ.get("PULSEOPS_DEMO_API", "http://api:8000")
    target = os.environ.get("PULSEOPS_DEMO_TARGET", "http://lab-target:8081")
    async with httpx.AsyncClient(base_url=api, timeout=30) as client:
        csrf = await login(
            client,
            os.environ.get("PULSEOPS_DEMO_USERNAME", "demo"),
            os.environ["PULSEOPS_DEMO_PASSWORD"],
        )
        client.headers["X-CSRF-Token"] = csrf
        ids = await prepare(client, args.monitors, args.interval, target)

        # Aquecimento: a primeira rodada de todos os monitores sai de uma vez.
        await asyncio.sleep(args.interval)
        since = datetime.now(UTC)
        stop = asyncio.Event()
        timings: dict[str, list[float]] = {}
        samplers = [
            asyncio.create_task(sample_api(client, stop, timings)) for _ in range(args.api_clients)
        ]
        await asyncio.sleep(args.duration)
        stop.set()
        await asyncio.gather(*samplers)
        until = datetime.now(UTC)
        await asyncio.sleep(args.interval)  # deixa terminar os checks iniciados na janela

        checks = await collect(client, ids, since, until)
        summary = (await client.get("/api/v1/summary")).json()
        gaps = 0
        for monitor_id in ids[: min(len(ids), 50)]:
            metrics = (
                await client.get(
                    f"/api/v1/monitors/{monitor_id}/metrics",
                    params={"from": since.isoformat(), "to": until.isoformat()},
                )
            ).json()
            gaps += len(metrics["gaps"])

        for monitor_id in ids:
            await client.post(f"/api/v1/monitors/{monitor_id}/archive")
```

- Linhas 115–121: login e token CSRF num cabeçalho padrão do cliente, usado em todas as chamadas.
- Linhas 124–125: aquecimento de um intervalo. Na primeira rodada, todos os monitores vencem juntos; medir essa rajada distorceria o atraso.
- Linhas 126–135: a janela medida. Clientes consultam a API em paralelo (`sample_api`, linhas 82–95) até o evento `stop`.
- Linha 136: espera mais um intervalo, para os checks iniciados dentro da janela terminarem de gravar.
- Linhas 138–148: coleta os checks da janela e as lacunas dos primeiros 50 monitores. **Ponto de atenção:** `collect` (linhas 98–109) pede no máximo 200 checks por monitor e não segue o cursor. Com os valores padrão são 12 checks por monitor, longe do limite; uma janela com mais de 200 checks por monitor seria contada a menos.
- Linhas 150–151: arquiva os monitores do ensaio, para não ficarem verificando depois.

scripts/ensaio_carga.py, linhas 153–192

```
153    window = (until - since).total_seconds()
154    expected = args.monitors * window / args.interval
155    completed = [c for c in checks if c["outcome"] is not None]
156    lateness = [
157        (
158            datetime.fromisoformat(c["started_at"]) - datetime.fromisoformat(c["scheduled_at"])
159        ).total_seconds()
160        * 1000
161        for c in completed
162    ]
163    durations = [c["duration_ms"] for c in completed if c["duration_ms"] is not None]
164    outcomes: dict[str, int] = {}
165    for c in completed:
166        outcomes[c["outcome"]] = outcomes.get(c["outcome"], 0) + 1
167
168    print(f"- Monitores: {args.monitors}, intervalo de {args.interval} s, alvo `{target}/stable`")
169    print(f"- Janela medida: {window:.0f} s (após {args.interval} s de aquecimento)")
170    print(f"- Clientes consultando a API em paralelo: {args.api_clients}")
171    print(f"- Checks esperados na janela: {expected:.0f}; gravados: {len(completed)}")
172    print(f"- Resultados: {outcomes}")
173    if lateness:
174        print(
175            "- Atraso de início (started_at - scheduled_at): "
176            f"mediana {statistics.median(lateness):.0f} ms, "
177            f"p95 {percentile(lateness, 95):.0f} ms, máximo {max(lateness):.0f} ms"
178        )
179    if durations:
180        print(
181            f"- Duração dos checks: mediana {statistics.median(durations):.0f} ms, "
182            f"p95 {percentile(durations, 95):.0f} ms"
183        )
184    print(f"- Lacunas de cobertura registradas (primeiros 50 monitores): {gaps}")
185    print(f"- Worker ativo ao final: {summary['worker_alive']}")
186    for path, values in sorted(timings.items()):
187        print(
188            f"- API `{path}`: {len(values)} requisições, mediana "
189            f"{statistics.median(values):.0f} ms, p95 {percentile(values, 95):.0f} ms, "
190            f"máximo {max(values):.0f} ms"
191        )
192    return 0
```

- Linha 154: checks esperados = monitores × janela ÷ intervalo.
- Linhas 156–162: o atraso de cada check é `started_at - scheduled_at`, em ms.
- O restante imprime as medidas: mediana, p95 e máximo.

`scripts/ensaio_carga.sh`

Finalidade. Embrulha o ensaio: registra a máquina, roda o Python e grava o relatório em `docs/evidencias/ensaio-de-carga.md`.

scripts/ensaio_carga.sh

```
#!/usr/bin/env bash
# Ensaio de carga local. Requer o ambiente da demonstração (scripts/demo.sh já executado).
# Uso: scripts/ensaio_carga.sh [--monitors N] [--interval S] [--duration S] [--api-clients N]
# Grava o relatório em docs/evidencias/ensaio-de-carga.md.
set -euo pipefail

cd "$(dirname "$0")/.."
COMPOSE=(docker compose -f compose.yaml -f compose.demo.yaml --profile demo)
OUT=docs/evidencias/ensaio-de-carga.md
mkdir -p docs/evidencias
PULSEOPS_DEMO_PASSWORD="$(cat .demo/admin-password)"
export PULSEOPS_DEMO_PASSWORD

{
  echo "# Ensaio de carga local"
  echo
  echo "Gerado por \`scripts/ensaio_carga.sh $*\` em $(date -u +%Y-%m-%dT%H:%MZ)."
  echo "Commit: \`$(git rev-parse --short HEAD 2>/dev/null || echo desconhecido)\`."
  echo
  echo "## Máquina"
  echo
  echo "- Sistema: $(uname -srm)"
  echo "- CPUs visíveis: $(nproc)"
  echo "- Memória total: $(awk '/MemTotal/ {printf "%.1f GiB", $2/1048576}' /proc/meminfo)"
  echo "- Docker: $(docker version --format '{{.Server.Version}}')"
  echo "- Worker: concorrência $("${COMPOSE[@]}" exec -T worker printenv PULSEOPS_WORKER_CONCURRENCY
2>/dev/null || echo '20 (padrão)')"
  echo
  echo "## Resultado"
  echo
} > "$OUT"

# Amostra de CPU e memória dos containers no meio da janela medida.
( sleep 70; docker stats --no-stream --format '| {{.Name}} | {{.CPUPerc}} | {{.MemUsage}} |' \
    > /tmp/pulseops-stats.txt ) &
STATS_PID=$!

"${COMPOSE[@]}" run --rm --no-deps -T demo-runner python scripts/ensaio_carga.py "$@" | tee -a "$OUT"
wait "$STATS_PID" || true

{
  echo
  echo "## Uso de recursos (amostra única no meio da janela)"
  echo
  echo "| Container | CPU | Memória |"
  echo "|---|---|---|"
  grep pulseops /tmp/pulseops-stats.txt || echo "| (sem amostra) | | |"
  echo
  echo "## Como ler"
  echo
  echo "Os números valem para esta máquina, esta configuração e esta duração. Não indicam"
  echo "capacidade máxima nem comportamento em produção; o alvo é local e responde em"
  echo "poucos milissegundos, o que não acontece com APIs reais."
} >> "$OUT"
echo "Relatório: $OUT"
```

- Linhas 14–30: o cabeçalho do relatório: data, commit, sistema, CPUs, memória, versão do Docker e a concorrência real do worker (lida de dentro do contêiner).

- Linhas 33–35: em segundo plano (`( ... ) &`), espera 70 s e tira **uma** amostra de CPU e memória dos contêineres. Com os valores padrão (10 s de aquecimento e 120 s de janela), isso cai no meio da janela. Com outros valores, pode cair fora dela; o relatório diz que é uma amostra única.
- Linha 37: `tee -a` mostra a saída e acrescenta ao relatório ao mesmo tempo.
- Linhas 48–52: o aviso de leitura: os números valem para esta máquina e esta configuração.

`scripts/conferir_referencias.py`

Finalidade. Confere que os arquivos e testes citados na documentação existem. É a regra R5 (confirmar antes de citar) transformada em programa.

Dados. Lê `README.md`, `CLAUDE.md` e todo `docs/**/*.md`, menos a especificação original. Imprime os problemas e sai com 1 se houver algum. **Relações.** Passo "Referências da documentação" do CI. **Testes.** Não há teste próprio; o CI o executa sobre a documentação real.

scripts/conferir_referencias.py

```python
"""Confere se os testes e arquivos citados na documentação existem de verdade.

Procura, em docs/*.md e README.md, referências como `tests/x.py::test_y`, as formas
abreviadas `::test_z` e `test_z` (que herdam o último arquivo citado na mesma linha) e caminhos
de arquivos do repositório entre crases. Sai com código 1 se algo não existir.
"""

import re
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
TOKEN = re.compile(r"`([^`\s]+)`")
PATH_PREFIXES = ("tests/", "src/", "scripts/", "docs/", "lab/", "migrations/", ".github/")


def main() -> int:
    documents = [ROOT / "README.md", ROOT / "CLAUDE.md", *sorted((ROOT / "docs").rglob("*.md"))]
    problems: list[str] = []
    checked = 0
    for doc in documents:
        if not doc.exists() or doc.name == "especificacao.md":
            continue
        for number, line in enumerate(doc.read_text(encoding="utf-8").splitlines(), 1):
            current: Path | None = None
            for token in TOKEN.findall(line):
                file_part, _, test = token.partition("::")
                if file_part.startswith(PATH_PREFIXES):
                    path = ROOT / file_part.rstrip("/")
                    if "*" in file_part or "{" in file_part:
                        continue
                    checked += 1
                    if not path.exists():
                        problems.append(f"{doc.name}:{number}: arquivo inexistente {file_part}")
                        continue
                    current = path if path.suffix == ".py" else None
                elif file_part == "" and test:
                    pass  # forma abreviada; usa o arquivo atual
                elif re.fullmatch(r"test_\w+", token) and current is not None:
                    test = token  # nome solto depois de um arquivo citado na mesma linha
                else:
                    continue
                if test:
                    checked += 1
                    if current is None:
                        problems.append(f"{doc.name}:{number}: ::{test} sem arquivo antes")
                    elif not re.search(rf"def {re.escape(test)}\b", current.read_text()):
```

```
                        problems.append(f"{doc.name}:{number}: {test} não existe em {current.name}")
    for problem in problems:
        print(problem)
    print(f"{checked} referências conferidas, {len(problems)} problemas")
    return 1 if problems else 0


if __name__ == "__main__":
    sys.exit(main())
```

- Linha 13: pega todo texto entre crases sem espaços.
- Linha 14: só confere textos que começam com uma pasta do projeto.
- Linhas 27–36: um caminho citado precisa existir. Caminhos com `*` ou `{` são padrões, não arquivos, e são pulados.
- Linhas 37–40: as formas curtas `::test_x` e `test_x` herdam o último arquivo `.py` citado na mesma linha.
- Linhas 43–48: o teste precisa existir no arquivo, procurado por `def nome`.
- **Limites:** não confere números de linha, nem afirmações em texto corrido, nem as fontes deste guia (que ficam em `study/`; o `study/build.py` faz a conferência dos trechos por conta própria).

`scripts/capturar_telas.mjs`

Finalidade. Tira as capturas de tela de `docs/imagens/` com o Playwright, num navegador Chromium sem janela. Opcional; não faz parte dos testes.

scripts/capturar_telas.mjs

```
// Captura as telas usadas no README e em docs/imagens/ (opcional; não faz parte dos testes).
// Requer Node e o pacote "playwright" com Chromium, e a demonstração no ar (scripts/demo.sh).
// Uso: node scripts/capturar_telas.mjs <pasta-de-saida> '[["painel","/"],["monitor","/monitors/1"]]'
import { chromium } from 'playwright';
import fs from 'node:fs';

const [out, pagesJson] = process.argv.slice(2);
const base = process.env.PULSEOPS_URL ?? 'http://127.0.0.1:8000';
const password = fs.readFileSync('.demo/admin-password', 'utf8').trim();
const pages = JSON.parse(pagesJson);
const browser = await chromium.launch();

async function shoot(viewport, list, suffix) {
  const ctx = await browser.newContext({ viewport, locale: 'pt-BR' });
  const page = await ctx.newPage();
  await page.goto(`${base}/login`);
  await page.screenshot({ path: `${out}/login${suffix}.png` });
  await page.fill('input[name=username]', process.env.PULSEOPS_DEMO_USERNAME ?? 'demo');
  await page.fill('input[name=password]', password);
  await page.click('button[type=submit]');
  await page.waitForURL(`${base}/`);
  for (const [name, path] of list) {
    await page.goto(base + path);
    await page.screenshot({ path: `${out}/${name}${suffix}.png`, fullPage: true });
  }
  await ctx.close();
}

await shoot({ width: 1280, height: 800 }, pages, '');
await shoot({ width: 390, height: 844 }, pages.slice(0, 2), '-celular');
await browser.close();
```

- Linhas 7–10: a URL vem da variável `PULSEOPS_URL` ou usa a local; a senha vem de `.demo/admin-password`, nunca do código.

- Linhas 13–27: abre um contexto em português, captura o login, entra e captura cada página da lista com `fullPage: true` (a página inteira, não só a parte visível).
- Linhas 29–30: um tamanho de computador (1280 × 800) e um de celular (390 × 844), este só para as duas primeiras páginas.

D10 — Docker, Compose, CI e configuração do projeto

Estes arquivos não têm lógica de negócio, mas decidem como o projeto é construído, como sobe e o que o CI recusa. Numa entrevista, são os arquivos mais fáceis de perguntar e os mais esquecidos de estudar.

`Dockerfile`

Finalidade. Uma imagem só para API, worker, migrações e laboratório. O que muda é o comando de cada serviço no Compose.

Relações. Construída pelo `compose.yaml` (`build`) e pelo job `docker` do CI. **Testes.** O job `docker` constrói a imagem do zero e roda a demonstração nela.

Dockerfile

```dockerfile
# syntax=docker/dockerfile:1
# Uma imagem para API, worker, migrações e laboratório (comandos diferentes no Compose).

FROM python:3.12-slim AS builder
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=never
WORKDIR /app

# extra_ca (opcional): certificado de uma rede com proxy TLS corporativo. Vazio por padrão.
RUN --mount=type=secret,id=extra_ca,required=false \
    if [ -s /run/secrets/extra_ca ]; then export PIP_CERT=/run/secrets/extra_ca; fi; \
    pip install --no-cache-dir uv==0.11.32

COPY pyproject.toml uv.lock README.md ./
RUN --mount=type=secret,id=extra_ca,required=false \
    if [ -s /run/secrets/extra_ca ]; then export SSL_CERT_FILE=/run/secrets/extra_ca; fi; \
    uv sync --frozen --no-dev --no-install-project

COPY src ./src
RUN --mount=type=secret,id=extra_ca,required=false \
    if [ -s /run/secrets/extra_ca ]; then export SSL_CERT_FILE=/run/secrets/extra_ca; fi; \
    uv sync --frozen --no-dev --no-editable


FROM python:3.12-slim
RUN useradd --system --uid 10001 --home-dir /app --shell /usr/sbin/nologin pulseops
WORKDIR /app
COPY --from=builder /app/.venv /app/.venv
COPY alembic.ini ./
COPY migrations ./migrations
COPY lab ./lab
COPY scripts ./scripts
ENV PATH=/app/.venv/bin:$PATH \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONPATH=/app
USER pulseops
EXPOSE 8000
CMD ["uvicorn", "pulseops.web.app:create_app", "--factory", "--host", "0.0.0.0", "--port", "8000"]
```

- Linha 1: ativa a sintaxe moderna do BuildKit, necessária para `--mount=type=secret`.
- Linhas 4–21: o **estágio de construção** (`AS builder`). Linha 5: compila o *bytecode* na instalação, copia arquivos em vez de criar *links* e proíbe o uv de baixar outro Python. Linha 11: versão exata do uv.

- Linhas 8–10, 14–15, 19–20: o segredo opcional `extra_ca`, para redes com proxy TLS corporativo. Um segredo de build é montado só durante aquele `RUN` e **não** fica gravado em camada nenhuma da imagem. Sem o arquivo, nada muda.
- Linhas 13–16: copia só o `pyproject.toml` e o `uv.lock` e instala as dependências **antes** de copiar o código. Como o Docker guarda cada passo em cache, mudar o código não reinstala as dependências; só mudar o *lock* faz isso. `--frozen` usa o `uv.lock` exatamente como está; `--no-dev` deixa as ferramentas de teste de fora.
- Linhas 18–21: copia o código e instala o projeto (`--no-editable`: uma cópia, e não um *link* para a pasta `src`).
- Linhas 24–38: o **estágio final**, que parte de uma imagem limpa. Os compiladores e caches do estágio anterior não vêm junto; só a `.venv` (linha 27).
- Linha 25: um usuário de sistema sem *shell* de login, com uid fixo. Linha 36: o processo roda como ele, e não como `root`.
- Linhas 28–31: migrações, laboratório e scripts entram na imagem porque os serviços `migrate`, `lab-*` e `demo-runner` usam a mesma imagem.
- Linhas 32–35: `PYTHONUNBUFFERED=1` manda os logs na hora, sem *buffer*; `PYTHONPATH=/app` torna `lab` e `scripts` importáveis.
- Linha 38: o comando padrão é a API. `--factory` diz ao uvicorn que `create_app` é uma função que **devolve** a aplicação.

`.dockerignore`

.dockerignore

```
1 .venv
2 .git
3 **/__pycache__
4 .pytest_cache
5 .mypy_cache
6 .ruff_cache
7 .env
8 docs
9 study
10 tests
11 htmlcov
```

O que **não** vai para o contexto de build: ambientes, caches, `.git`, documentação, testes e, principalmente, o `.env` (linha 7). Sem isso, um `COPY` descuidado poderia colocar a senha do banco dentro da imagem.

`compose.yaml`

Finalidade. O ambiente padrão: banco, migrações, API e worker.

Relações. Usado por `docker compose up`, pelo `scripts/demo.sh` (junto com o `compose.demo.yaml`) e pela instalação limpa descrita no README. **Testes.** O job `docker` do CI e `docs/evidencias/instalacao-limpa.md`.

compose.yaml

```yaml
1 # Ambiente padrão: banco, migrações, API e worker. Sem laboratório (ver compose.demo.yaml).
2 name: pulseops
3
```

```yaml
x-app: &app
  image: pulseops:local
  build:
    context: .
    secrets: [extra_ca]
  env_file:
    - path: .env
      required: false
  environment:
    PULSEOPS_DATABASE_URL: postgresql+psycopg://${POSTGRES_USER:-pulseops}:${POSTGRES_PASSWORD:-pulseops-local}@db:5432/${POSTGRES_DB:-pulseops}
  read_only: true
  tmpfs: [/tmp]
  cap_drop: [ALL]
  security_opt: ["no-new-privileges:true"]

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-pulseops}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-pulseops-local}
      POSTGRES_DB: ${POSTGRES_DB:-pulseops}
    volumes:
      - pgdata:/var/lib/postgresql/data
    # Sem "ports": o banco só é acessível pela rede interna do Compose.
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 2s
      timeout: 3s
      retries: 30
    restart: unless-stopped

  migrate:
    <<: *app
    command: ["alembic", "upgrade", "head"]
    depends_on:
      db: {condition: service_healthy}
    restart: "no"

  api:
    <<: *app
    ports:
      - "127.0.0.1:${PULSEOPS_PORT:-8000}:8000"
    depends_on:
      migrate: {condition: service_completed_successfully}
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request as u; u.urlopen('http://127.0.0.1:8000/health/ready', timeout=3)"]
      interval: 3s
      timeout: 5s
      retries: 20
    restart: unless-stopped

  worker:
    <<: *app
    command: ["python", "-m", "pulseops.worker"]
    depends_on:
      migrate: {condition: service_completed_successfully}
    restart: unless-stopped

volumes:
  pgdata:

secrets:
  extra_ca:
    file: ${PULSEOPS_EXTRA_CA_FILE:-./docker/no-extra-ca.pem}
```

- Linha 2: `name` fixa o prefixo dos contêineres e volumes (`pulseops-...`).

- Linhas 4–17: `x-app` é um **bloco reutilizável** (âncora `&app` do YAML), aplicado aos três serviços da aplicação com `<<: *app`.
  - Linhas 9–11: lê o `.env` se ele existir (`required: false`).
  - Linha 13: monta a URL do banco a partir das mesmas variáveis do serviço `db`. `${VAR:-padrão}` usa o padrão quando a variável não existe.
  - Linhas 14–17: **endurecimento** do contêiner. Sistema de arquivos só de leitura, com `/tmp` em memória; nenhuma *capability* do Linux; e `no-new-privileges`, que impede um processo de ganhar privilégios depois de iniciado.
- Linhas 20–34: o PostgreSQL 16. Linha 28: **sem `ports`**, o banco não é acessível de fora da rede do Compose. Linhas 29–33: o *healthcheck* com `pg_isready`. O `$$` escapa o `$` para o Compose deixar a variável para o *shell* do contêiner.
- Linhas 36–41: `migrate` roda `alembic upgrade head` e termina. Só começa com o banco saudável; `restart: "no"` porque ele **deve** terminar.
- Linhas 43–54: a API. Linha 46: publicada **só em `127.0.0.1`**, ou seja, não fica visível para outras máquinas da rede. Linha 48: só sobe depois de o `migrate` terminar com sucesso. Linhas 49–53: o *healthcheck* chama `/health/ready` com o Python da imagem (não há `curl` na imagem *slim*).
- Linhas 56–61: o worker, com o comando `python -m pulseops.worker`. Também espera o `migrate`, mas não a API.
- Linhas 63–64: o volume nomeado `pgdata` guarda os dados entre reinícios. `docker compose down` mantém o volume; `down -v` o apaga.
- Linhas 66–68: o segredo do build aponta para um arquivo vazio do repositório (`docker/no-extra-ca.pem`), a menos que `PULSEOPS_EXTRA_CA_FILE` diga outro.

`compose.demo.yaml`

Finalidade. O que muda no modo demonstração: variáveis da API e do worker, e os três serviços do laboratório no perfil `demo`.

compose.demo.yaml

```yaml
# Laboratório de falhas. Use sempre junto com compose.yaml e o perfil "demo":
#   docker compose -f compose.yaml -f compose.demo.yaml --profile demo up -d --wait
# (o script scripts/demo.sh faz isso e executa o roteiro).
#
# O que muda em relação ao ambiente padrão:
# - API e worker rodam em modo demonstração: intervalo mínimo de 2 s e uma lista
#   NOMINAL de destinos internos liberados (só lab-target:8081 e lab-receiver:8082).
#   Nenhum outro endereço privado é liberado.
# - Retentativa de webhook mais curta, para a demonstração não esperar minutos.
# - Três serviços extras, todos no perfil "demo" e sem portas publicadas no host.

x-demo-env: &demo-env
  PULSEOPS_DEMO_MODE: "true"
  PULSEOPS_DEMO_ALLOWED_HOSTS: lab-target:8081,lab-receiver:8082
  PULSEOPS_WEBHOOK_BACKOFF_BASE_SECONDS: "1"
  PULSEOPS_WEBHOOK_BACKOFF_MAX_SECONDS: "5"
  PULSEOPS_WEBHOOK_MAX_ATTEMPTS: "20"

x-lab: &lab
  image: pulseops:local
  profiles: [demo]
  read_only: true
  tmpfs: [/tmp]
  cap_drop: [ALL]
  security_opt: ["no-new-privileges:true"]
  restart: unless-stopped

```

```yaml
services:
  api:
    environment: *demo-env

  worker:
    environment: *demo-env

  lab-target:
    <<: *lab
    command: ["uvicorn", "lab.target:app", "--host", "0.0.0.0", "--port", "8081", "--no-access-log"]
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request as u;
  u.urlopen('http://127.0.0.1:8081/control/state', timeout=3)"]
      interval: 2s
      timeout: 5s
      retries: 20

  lab-receiver:
    <<: *lab
    command: ["uvicorn", "lab.receiver:app", "--host", "0.0.0.0", "--port", "8082", "--no-access-log"]
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request as u;
  u.urlopen('http://127.0.0.1:8082/control/received', timeout=3)"]
      interval: 2s
      timeout: 5s
      retries: 20

  # Executa o roteiro (scripts/demo_flow.py) de dentro da rede do Compose, que é o
  # único lugar de onde os controles do laboratório são alcançáveis.
  demo-runner:
    <<: *lab
    command: ["python", "scripts/demo_flow.py"]
    restart: "no"
    environment:
      PULSEOPS_DEMO_API: http://api:8000
      PULSEOPS_DEMO_TARGET: http://lab-target:8081
      PULSEOPS_DEMO_RECEIVER: http://lab-receiver:8082
      PULSEOPS_DEMO_USERNAME: ${PULSEOPS_DEMO_USERNAME:-demo}
      PULSEOPS_DEMO_PASSWORD: ${PULSEOPS_DEMO_PASSWORD:-}
    depends_on:
      api: {condition: service_healthy}
      lab-target: {condition: service_healthy}
      lab-receiver: {condition: service_healthy}
```

- Linhas 1–10: o comentário explica o arquivo e lembra que ele só faz sentido junto com o `compose.yaml`. O Compose **mescla** os dois: `api` e `worker` ganham as variáveis do bloco `x-demo-env`.
- Linhas 12–17: o modo demonstração e a lista **nominal** de destinos liberados (só `lab-target:8081` e `lab-receiver:8082`; nenhum outro endereço privado). As retentativas de webhook ficam curtas (1 s de base, até 5 s) e com até 20 tentativas, para a demonstração não esperar minutos.
- Linhas 19–26: os serviços do laboratório herdam o mesmo endurecimento e só existem com `--profile demo`.
- Linhas 35–51: alvo e receptor, cada um com um *healthcheck* nos próprios controles.
- Linhas 55–68: o `demo-runner` roda o roteiro e termina. Espera a API, o alvo e o receptor estarem saudáveis. A senha chega pela variável que o `demo.sh` exporta.

`.env.example`

Finalidade. O modelo do `.env`: as variáveis que uma pessoa costuma ajustar, com valores que **não** são segredos reais.

.env.example

```
# Copie para .env e ajuste. Nunca coloque o .env no Git.
# Senha do banco usada pelo Compose (troque por uma senha sua).
POSTGRES_PASSWORD=troque-esta-senha

# Porta local da API/dashboard (só em 127.0.0.1).
PULSEOPS_PORT=8000

# Ative quando o PulseOps estiver atrás de HTTPS.
PULSEOPS_COOKIE_SECURE=false
PULSEOPS_SESSION_TTL_MINUTES=480
PULSEOPS_DISPLAY_TIMEZONE=America/Sao_Paulo
PULSEOPS_LOG_LEVEL=INFO

# Portas que os monitores e webhooks podem usar.
PULSEOPS_ALLOWED_PORTS=80,443,8080,8443

# Worker
PULSEOPS_WORKER_CONCURRENCY=20
PULSEOPS_CHECK_RETENTION_DAYS=30
PULSEOPS_WEBHOOK_TIMEOUT_SECONDS=5
PULSEOPS_WEBHOOK_MAX_ATTEMPTS=6

# Apenas para redes com proxy TLS corporativo durante o build da imagem:
# PULSEOPS_EXTRA_CA_FILE=/caminho/para/ca.pem
```

- Linha 3: `troque-esta-senha` é um aviso, não uma senha.
- Linha 9: `PULSEOPS_COOKIE_SECURE=false` porque o ambiente local é HTTP. Atrás de HTTPS, deve ser `true`, para o cookie de sessão nunca trafegar sem criptografia.
- Linhas 17–21: os padrões do worker, iguais aos de `src/pulseops/config.py`.
- A lista completa, com a descrição de cada variável, está em `docs/operacao.md`.

`.gitignore`

.gitignore

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/
.env
build/
dist/
study/build/
docker/local-ca.pem
.demo/
study/node_modules
```

- Linha 9: o `.env`, que guarda a senha real do banco.
- Linha 13: `docker/local-ca.pem`, o certificado de uma rede específica, que não deve ir para o repositório.
- Linha 14: `.demo/`, onde fica a senha da operadora da demonstração.
- Linhas 12 e 15: o resultado do build deste guia e as dependências de Node usadas para gerar o PDF.

`pyproject.toml`

Finalidade. A identidade do projeto e a configuração de todas as ferramentas.

pyproject.toml

```toml
[project]
name = "pulseops"
version = "0.1.0"
description = "Monitoramento explicável de APIs HTTP: verificações, incidentes por limiar e alertas por webhook."
readme = "README.md"
license = "MIT"
requires-python = ">=3.12,<3.13"
dependencies = [
    "alembic>=1.20.0",
    "argon2-cffi>=25.1.0",
    "fastapi>=0.159.1",
    "httpcore>=1.0.9",
    "httpx>=0.28.1",
    "jinja2>=3.1.6",
    "psycopg[binary]>=3.3.6",
    "pydantic-settings>=2.16.1",
    "python-multipart>=0.0.32",
    "sqlalchemy[asyncio]>=2.1.6",
    "uvicorn[standard]>=0.54.0",
]

[project.scripts]
pulseops = "pulseops.cli:main"

[dependency-groups]
dev = [
    "mypy>=2.4.0",
    "pip-audit>=2.10.1",
    "pytest>=9.1.1",
    "pytest-asyncio>=1.4.0",
    "pytest-cov>=7.1.0",
    "ruff>=0.17.1",
]

[build-system]
requires = ["uv_build>=0.11,<0.12"]
build-backend = "uv_build"

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "S", "SIM", "ASYNC", "DTZ", "RUF"]
ignore = ["RUF001", "RUF002", "RUF003"]  # acentos e aspas tipográficas em textos em português

[tool.ruff.lint.per-file-ignores]
"tests/**" = ["S101", "S105", "S106"]
"lab/**" = ["S104"]
"scripts/**" = ["S101"]
"migrations/versions/**" = ["E501"]
# Gerador do material de estudo: chama git, pandoc e node com argumentos fixos.
"study/**" = ["S603", "S607"]

[tool.mypy]
python_version = "3.12"
strict = true
plugins = ["pydantic.mypy"]
files = ["src/pulseops", "lab", "scripts"]

[tool.pytest.ini_options]
testpaths = ["tests"]
asyncio_mode = "auto"
```

```toml
asyncio_default_fixture_loop_scope = "session"
asyncio_default_test_loop_scope = "session"
addopts = "-ra --strict-markers"

[tool.coverage.run]
source = ["pulseops"]
branch = true
# O SQLAlchemy assíncrono executa partes do código dentro de greenlets; sem isto,
# linhas executadas aparecem como não cobertas.
concurrency = ["greenlet", "thread"]

[tool.coverage.report]
show_missing = true
skip_covered = false
```

- Linhas 1–7: nome, versão, licença e a versão de Python aceita: `>=3.12,<3.13`, só a 3.12, que é a testada.
- Linhas 8–20: as dependências de execução, com versão **mínima**. As versões exatas instaladas ficam no `uv.lock`, que é o que o build e o CI usam (`--frozen`).
- Linhas 22–23: cria o comando `pulseops` apontando para `pulseops.cli:main`.
- Linhas 25–33: as ferramentas de desenvolvimento, fora da imagem.
- Linhas 35–37: o *backend* de build do uv, que empacota `src/pulseops`.
- Linhas 39–45: o Ruff. Linha 44: as famílias de regras. `S` são as regras de segurança do Bandit; `ASYNC` pega erros comuns em código assíncrono; `DTZ` recusa datas sem fuso; `B` pega armadilhas como argumentos padrão mutáveis. Linha 45: desliga os avisos sobre acentos e aspas tipográficas, normais em textos em português.
- Linhas 47–53: exceções por pasta. Nos testes, `assert` (S101) e senhas fixas de teste (S105, S106) são esperados. No gerador do guia, chamar programas externos (S603, S607) é o objetivo. **Ponto de atenção:** as exceções `S104` para `lab/` e `S101` para `scripts/` estão sobrando. Rodando o Ruff só com essas regras e sem as exceções (`ruff check --isolated --select S104,S101 lab scripts`), nada é apontado. São inofensivas, mas podem ser removidas.
- Linhas 55–59: o mypy no modo `strict` (todas as funções com tipos, nada de `Any` implícito), com o *plugin* do Pydantic, sobre o código, o laboratório e os scripts. Os testes ficam de fora.
- Linhas 61–66: o pytest. `asyncio_mode = "auto"` trata toda função `async def test_...` como teste assíncrono. As linhas 64–65 fazem todos os testes usarem o **mesmo** *event loop*, necessário porque o engine do banco é criado uma vez por sessão de testes. `--strict-markers` recusa marcadores digitados errado.
- Linhas 68–77: a cobertura mede ramos (`branch = true`, cada `if` nos dois sentidos). O comentário das linhas 71–72 explica `concurrency`: sem ela, linhas executadas dentro de *greenlets* do SQLAlchemy apareceriam como não cobertas.

`.github/workflows/ci.yml`

Finalidade. A integração contínua: em cada *push* na `main` e em cada *pull request*, roda a qualidade e a demonstração. Não implanta nada.

Testes. É ele quem executa os testes. Se o CI está verde, só se pode afirmar com o link de uma execução (volume 1, parte I).

`.github/workflows/ci.yml`

```yaml
# Integração contínua: lint, tipos, testes com PostgreSQL, migrações, auditoria e
# imagem Docker com o roteiro do laboratório. Não implanta nada (não é CD).
name: CI
```

```yaml

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  qualidade:
    name: Lint, tipos, testes e migrações
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: pulseops
          POSTGRES_PASSWORD: pulseops
          POSTGRES_DB: pulseops
        ports: ["5433:5432"]
        options: >-
          --health-cmd "pg_isready -U pulseops"
          --health-interval 2s --health-timeout 3s --health-retries 30
    env:
      PULSEOPS_TEST_DATABASE_URL: postgresql+psycopg://pulseops:pulseops@127.0.0.1:5433/pulseops_test
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v6
        with:
          version: "0.11.32"
      - name: Instalar dependências (uv.lock)
        run: uv sync --frozen
      - name: Ruff (lint)
        run: uv run ruff check .
      - name: Ruff (formatação)
        run: uv run ruff format --check .
      - name: mypy
        run: uv run mypy
      - name: Migrações em banco limpo
        env:
          PULSEOPS_DATABASE_URL: postgresql+psycopg://pulseops:pulseops@127.0.0.1:5433/pulseops
        run: |
          uv run alembic upgrade head
          uv run alembic downgrade base
          uv run alembic upgrade head
      - name: Testes com cobertura
        run: uv run pytest --cov --cov-report=term --cov-fail-under=90
      - name: Metas de cobertura de domínio e segurança
        run: |
          uv run coverage report --include='src/pulseops/domain/*' --fail-under=95
          uv run coverage report --include='src/pulseops/security/*' --fail-under=95
      - name: Referências da documentação
        run: uv run python scripts/conferir_referencias.py
      - name: Auditoria de dependências (pip-audit)
        run: |
          uv export --frozen --no-emit-project --no-hashes -o requirements-audit.txt
          uvx pip-audit==2.9.0 -r requirements-audit.txt --disable-pip --no-deps

  docker:
    name: Imagem Docker e laboratório
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - name: Roteiro do laboratório (sobe tudo e verifica os 11 passos)
```

```
75        run: scripts/demo.sh
76      - name: Logs em caso de falha
77        if: failure()
78        run: docker compose -f compose.yaml -f compose.demo.yaml --profile demo logs --tail 200
79      - name: Derrubar
80        if: always()
81        run: scripts/demo.sh --down
```

- Linhas 10–11: o *token* do workflow só pode **ler** o repositório. Mesmo que um passo fosse comprometido, não poderia publicar nada.
- Linhas 13–15: um *push* novo no mesmo ramo cancela a execução anterior, que já ficou velha.
- Linhas 18–34: o job `qualidade` tem um PostgreSQL como serviço, na porta 5433, a mesma do banco de testes local. O `conftest.py` recria o banco `pulseops_test` a partir dessa conexão.
- Linhas 37–41: o uv numa versão exata e a instalação a partir do `uv.lock`.
- Linhas 42–47: Ruff (regras e formatação) e mypy. Qualquer aviso falha o job.
- Linhas 48–54: as migrações num banco limpo: sobe tudo, **desce até o zero** e sobe de novo. Assim se confirma que o `downgrade` funciona e que a migração pode ser reaplicada.
- Linhas 55–56: os testes com cobertura mínima de 90% no total.
- Linhas 57–60: metas mais altas, 95%, para domínio e segurança.
- Linhas 61–62: o `conferir_referencias.py` (D9).
- Linhas 63–66: o `pip-audit` confere as versões travadas no `uv.lock` contra a base de vulnerabilidades conhecidas. **Ponto de atenção:** o CI fixa a versão 2.9.0 do `pip-audit` (linha 66), enquanto o grupo `dev` do `pyproject.toml` pede `>=2.10.1`. As duas funcionam, mas é uma inconsistência que vale alinhar.
- Linhas 68–81: o job `docker` roda `scripts/demo.sh`, que constrói a imagem e executa os 11 passos. Se falhar, mostra os últimos logs (`if: failure()`); em qualquer caso, derruba o ambiente (`if: always()`).

Arquivos de dados do repositório

- `docker/no-extra-ca.pem`: um arquivo **vazio**, o padrão do segredo de build.
- `uv.lock`: gerado pelo uv (`uv lock`); não se edita à mão. Trava as versões exatas de todas as dependências, inclusive as indiretas, com os *hashes*.
- `alembic.ini` e os arquivos de `migrations/`: explicados em D3.
- `LICENSE`: o texto da licença MIT, sem nada a explicar.

D11 — Os testes

São 3.130 linhas em 31 arquivos (`wc -l tests/*.py tests/*/*.py`), com 173 funções de teste que o pytest expande em 251 casos, por causa de `@pytest.mark.parametrize` (`uv run pytest --collect-only -q`). Os testes se repetem em forma: prepare, execute, confira. Por isso este capítulo explica as **peças compartilhadas** linha a linha, mostra um exemplo de cada padrão e resume os outros arquivos numa tabela.

Os três arquivos `__init__.py` (`tests/`, `tests/unit/`, `tests/integration/`) estão vazios. Só existem para que `from tests.support import ...` funcione.

`tests/conftest.py`

Finalidade. As *fixtures* compartilhadas. Uma *fixture* é uma função que prepara algo para o teste. O teste a pede pelo nome do parâmetro, e o pytest a executa antes.

Relações. O pytest carrega o `conftest.py` sozinho; nenhum teste o importa, a não ser para usar `login` e as constantes.

tests/conftest.py, linhas 1–56

```python
"""Fixtures compartilhadas.

Os testes de integração usam um PostgreSQL real (nunca SQLite). A URL vem de
PULSEOPS_TEST_DATABASE_URL; o banco de teste é recriado do zero a cada execução
e as migrações do Alembic são aplicadas nele.
"""

import os
import re
from collections.abc import AsyncIterator, Iterator

import httpx
import psycopg
import pytest
from alembic import command
from alembic.config import Config
from sqlalchemy import text
from sqlalchemy.engine import make_url
from sqlalchemy.ext.asyncio import AsyncEngine

from pulseops.config import Settings
from pulseops.db.models import Base
from pulseops.db.session import SessionFactory, create_engine, create_session_factory
from pulseops.services.auth import create_operator
from pulseops.web.app import create_app
from tests.support import FakeClock, FakeResolver

TEST_DATABASE_URL = os.environ.get(
    "PULSEOPS_TEST_DATABASE_URL",
    "postgresql+psycopg://pulseops:pulseops@127.0.0.1:5433/pulseops_test",
)
ADMIN_USER, ADMIN_PASSWORD = "operadora", "senha-de-teste-123"


def _recreate_database(url: str) -> None:
    parsed = make_url(url)
    admin_dsn = parsed.set(drivername="postgresql", database="postgres").render_as_string(
        hide_password=False
    )
    with psycopg.connect(admin_dsn, autocommit=True) as conn:
```

```
41          conn.execute(f'DROP DATABASE IF EXISTS "{parsed.database}" WITH (FORCE)')
42          conn.execute(f'CREATE DATABASE "{parsed.database}"')
43
44
45  def alembic_config(url: str) -> Config:
46      config = Config(os.path.join(os.path.dirname(__file__), "..", "alembic.ini"))
47      config.attributes["database_url"] = url
48      return config
49
50
51  @pytest.fixture(scope="session")
52  def database_url() -> Iterator[str]:
53      _recreate_database(TEST_DATABASE_URL)
54      command.upgrade(alembic_config(TEST_DATABASE_URL), "head")
55      yield TEST_DATABASE_URL
```

- Linhas 1–6: a regra: PostgreSQL de verdade, nunca SQLite. Constraints parciais, `FOR UPDATE SKIP LOCKED` e *advisory locks* não existem no SQLite, e um teste nele provaria pouco.
- Linhas 28–31: a URL vem de `PULSEOPS_TEST_DATABASE_URL`, ou usa o banco local da porta 5433.
- Linha 32: o usuário e a senha de teste. Ficam no código porque não protegem nada (S105 e S106 estão liberados para `tests/`).
- Linhas 35–42: apaga e recria o banco de teste. `WITH (FORCE)` derruba conexões abertas. **Ponto de atenção:** a função apaga **o banco que a variável indicar**. Se alguém apontasse `PULSEOPS_TEST_DATABASE_URL` para um banco com dados reais, eles seriam perdidos. A proteção é só o nome padrão (`pulseops_test`); não há uma trava que recuse outros nomes.
- Linhas 45–48: monta a configuração do Alembic com a URL do teste. O `env.py` das migrações lê `config.attributes["database_url"]` (D3).
- Linhas 51–55: `scope="session"`: recria o banco e aplica as migrações **uma vez** por execução da suíte, e não a cada teste. Assim, os testes rodam sobre o esquema que as migrações criam, o mesmo da produção, e não sobre `create_all`.

`tests/conftest.py, linhas 58–98`

```
58  @pytest.fixture(scope="session")
59  async def engine(database_url: str) -> AsyncIterator[AsyncEngine]:
60      engine = create_engine(database_url, pool_size=10)
61      yield engine
62      await engine.dispose()
63
64
65  @pytest.fixture
66  async def sessions(engine: AsyncEngine) -> SessionFactory:
67      """Banco limpo para cada teste (TRUNCATE em todas as tabelas)."""
68      tables = ", ".join(f'"{t.name}"' for t in Base.metadata.sorted_tables)
69      async with engine.begin() as conn:
70          await conn.execute(text(f"TRUNCATE {tables} RESTART IDENTITY CASCADE"))
71      return create_session_factory(engine)
72
73
74  @pytest.fixture
75  def clock() -> FakeClock:
76      return FakeClock()
77
78
79  @pytest.fixture
80  def resolver() -> FakeResolver:
81      return FakeResolver()
82
83
84  @pytest.fixture
85  def settings(database_url: str) -> Settings:
86      return Settings(database_url=database_url, display_timezone="America/Sao_Paulo")
87
```

```
88
89 @pytest.fixture
90 async def app_client(
91     settings: Settings, sessions: SessionFactory, clock: FakeClock, resolver: FakeResolver
92 ) -> AsyncIterator[httpx.AsyncClient]:
93     app = create_app(settings, sessions=sessions, clock=clock, resolver=resolver)
94     async with app.router.lifespan_context(app):
95         transport = httpx.ASGITransport(app=app)
96         async with httpx.AsyncClient(transport=transport, base_url="http://testserver") as client:
97             yield client
```

- Linhas 58–62: um engine para a suíte toda.
- Linhas 65–71: antes de **cada** teste, `TRUNCATE` em todas as tabelas, com `RESTART IDENTITY` (os ids voltam a 1) e `CASCADE`. Cada teste começa com o banco vazio e não depende da ordem em que os outros rodaram.
- Linhas 74–81: um relógio falso e um resolvedor falso novos por teste.
- Linhas 89–97: a aplicação com as peças falsas. `lifespan_context` roda o *lifespan* (D8) como num servidor de verdade. `ASGITransport` entrega as requisições direto à aplicação, **sem rede**: o teste exercita roteamento, dependências, cookies e templates, mas não abre porta nenhuma.

`tests/conftest.py, linhas 100–132`

```
100 CSRF_FIELD = re.compile(r'name="csrf_token" value="([^"]+)"')
101
102
103 async def login(
104     client: httpx.AsyncClient, username: str = ADMIN_USER, password: str = ADMIN_PASSWORD
105 ) -> str:
106     """Faz login pelo formulário e devolve o token CSRF da sessão."""
107     page = await client.get("/login")
108     login_csrf = CSRF_FIELD.search(page.text)
109     assert login_csrf
110     response = await client.post(
111         "/login",
112         data={"username": username, "password": password, "csrf_token": login_csrf.group(1)},
113     )
114     assert response.status_code == 303, response.text
115     dashboard = await client.get("/")
116     token = CSRF_FIELD.search(dashboard.text)
117     assert token
118     return token.group(1)
119
120
121 @pytest.fixture
122 async def operator(sessions: SessionFactory, clock: FakeClock) -> None:
123     async with sessions.begin() as session:
124         await create_operator(session, ADMIN_USER, ADMIN_PASSWORD, now=clock.now())
125
126
127 @pytest.fixture
128 async def api(app_client: httpx.AsyncClient, operator: None) -> httpx.AsyncClient:
129     """Cliente autenticado; o header X-CSRF-Token já vai em todas as requisições."""
130     token = await login(app_client)
131     app_client.headers["X-CSRF-Token"] = token
132     return app_client
```

- Linhas 103–118: login pelo **formulário**, igual a um navegador: lê o token do login, envia, espera 303 e lê o token CSRF da sessão no painel. Os testes de API passam pelo mesmo login da interface.
- Linhas 121–124: cria a operadora.

- Linhas 127–132: `api` é o cliente já logado, com o cabeçalho CSRF em toda requisição. Um teste que pede `api` recebe, encadeadas, as fixtures `app_client`, `operator`, `sessions`, `engine` e `database_url`.

`tests/support.py`

Finalidade. As peças falsas que substituem o tempo, o DNS e a rede.

tests/support.py

```python
"""Peças reutilizadas pelos testes: relógio controlável, resolvedor e alvo HTTP simulados."""

import ipaddress
from collections.abc import Callable
from datetime import UTC, datetime, timedelta

import httpx

from pulseops.checks.checker import HttpChecker
from pulseops.security.url_policy import IPAddress

START = datetime(2026, 1, 15, 12, 0, tzinfo=UTC)


class FakeClock:
    """Relógio parado que só anda quando o teste manda. Sem sleep nos testes."""

    def __init__(self, start: datetime = START) -> None:
        self._now = start
        self._mono = 1000.0

    def now(self) -> datetime:
        return self._now

    def monotonic(self) -> float:
        return self._mono

    def advance(self, seconds: float) -> None:
        self._now += timedelta(seconds=seconds)
        self._mono += seconds


class FakeResolver:
    """Resolve nomes conhecidos para IPs fixos; o resto vai para um IP público fictício."""

    def __init__(self, mapping: dict[str, list[str]] | None = None) -> None:
        self.mapping = mapping or {}
        self.calls: list[str] = []

    async def resolve(self, host: str, port: int) -> list[IPAddress]:
        self.calls.append(host)
        if host in self.mapping:
            return [ipaddress.ip_address(a) for a in self.mapping[host]]
        if host.endswith(".invalid"):
            raise OSError("nome não resolvido")
        return [ipaddress.ip_address("93.184.216.34")]


class ScriptedTarget:
    """Alvo HTTP simulado com httpx.MockTransport.

    `mode` escolhe a resposta: "ok", "slow", "error", "timeout", "refused", "tls".
    A lentidão é simulada avançando o FakeClock, não dormindo.
    """

    def __init__(self, clock: FakeClock, slow_ms: int = 1500) -> None:
        self.clock = clock
```

```
58          self.mode = "ok"
59          self.slow_ms = slow_ms
60          self.requests: list[httpx.Request] = []
61
62      def handler(self, request: httpx.Request) -> httpx.Response:
63          self.requests.append(request)
64          self.clock.advance(0.02)
65          if self.mode == "slow":
66              self.clock.advance(self.slow_ms / 1000)
67              return httpx.Response(200, text="ok")
68          if self.mode == "error":
69              return httpx.Response(500, text="erro")
70          if self.mode == "timeout":
71              raise httpx.ReadTimeout("timeout", request=request)
72          if self.mode == "refused":
73              raise httpx.ConnectError("[Errno 111] Connection refused", request=request)
74          if self.mode == "tls":
75              import ssl
76
77              try:
78                  raise ssl.SSLCertVerificationError(1, "certificate verify failed")
79              except ssl.SSLCertVerificationError as exc:
80                  raise httpx.ConnectError("SSL: CERTIFICATE_VERIFY_FAILED", request=request) from exc
81          return httpx.Response(200, text="ok")
82
83      def checker(self) -> HttpChecker:
84          client = httpx.AsyncClient(transport=httpx.MockTransport(self.handler))
85          return HttpChecker(client, self.clock)
86
87
88  def handler_from(fn: Callable[[httpx.Request], httpx.Response]) -> httpx.MockTransport:
89      return httpx.MockTransport(fn)
```

- Linha 12: uma data fixa para todos os testes. Um teste que usasse a data real poderia mudar de resultado conforme o dia.
- Linhas 15–30: o `FakeClock`. Fica **parado** até o teste chamar `advance`. Com ele, "esperar 60 s" é instantâneo, e o resultado não depende da velocidade da máquina. É por isso que a suíte não tem `sleep` (volume 1, parte G).
- Linhas 33–46: o `FakeResolver`. Nomes conhecidos resolvem para os IPs que o teste escolheu; `.invalid` falha (é um domínio reservado que nunca existe); o resto vai para um IP público. `calls` registra o que foi consultado. É ele que permite simular o DNS *rebinding*: o teste troca o IP no meio do caminho.
- Linhas 49–85: o `ScriptedTarget`, um alvo HTTP falso. O `httpx.MockTransport` chama `handler` em vez de abrir uma conexão. Cada modo produz o que a rede real produziria: 500, `ReadTimeout`, conexão recusada ou erro de certificado. A lentidão avança o relógio falso (linha 66) em vez de dormir.
- Linhas 74–80: para simular o erro TLS, o código levanta um `SSLCertVerificationError` e o encadeia (`from exc`) num `ConnectError`, porque é assim que o httpx entrega esse erro. O verificador (D5) procura a causa encadeada para classificar como `tls_error`.
- Linhas 83–85: um `HttpChecker` de verdade, só com o transporte trocado.

`tests/support_db.py`

Finalidade. Atalhos dos testes de integração: criar monitores, montar o worker, dar um "tique" e ler o banco.

tests/support_db.py, linhas 19–73

```python
BASE = MonitorConfig(
    name="API de pedidos",
    url="https://pedidos.example.com/health",
    interval_seconds=60,
    timeout_seconds=5,
    latency_threshold_ms=1000,
    failure_threshold=3,
    recovery_threshold=2,
)


async def create_monitor(sessions: SessionFactory, clock: FakeClock, **changes: Any) -> Monitor:
    async with sessions.begin() as session:
        return await monitors.create_monitor(
            session,
            replace(BASE, **changes),
            now=clock.now(),
            policy=DestinationPolicy(),
            min_interval_seconds=2,
            resolver=FakeResolver(),
        )


class RecordingNotifier:
    def __init__(self) -> None:
        self.sent: list[tuple[str, str]] = []
        self.result = SendResult(True, False, 200, 5, None)

    async def send(self, url: str, event_id: str, payload: dict[str, Any]) -> SendResult:
        self.sent.append((url, event_id))
        return self.result


def make_worker(
    sessions: SessionFactory,
    clock: FakeClock,
    target: ScriptedTarget,
    settings: Settings,
    notifier: Any = None,
) -> Worker:
    return Worker(
        sessions=sessions,
        checker=target.checker(),
        notifier=notifier or RecordingNotifier(),
        clock=clock,
        settings=settings,
        worker_id="worker-de-teste",
    )


async def tick(worker: Worker, clock: FakeClock, seconds: float = 60) -> int:
    """Executa o que estiver vencido e avança o relógio um intervalo."""
    count = await worker.run_checks_once()
    clock.advance(seconds)
    return count
```

- Linhas 19–27: a configuração base de um monitor. Cada teste muda só o que importa (`create_monitor(..., failure_threshold=1)`), com `dataclasses.replace`.
- Linhas 30–39: cria pelo **serviço**, com as mesmas validações da API.
- Linhas 42–49: um notificador falso que registra os envios e devolve sucesso, a menos que o teste troque `result`.
- Linhas 52–66: um `Worker` real com o alvo falso.
- Linhas 69–73: `tick` executa os checks vencidos e avança o relógio um intervalo. Uma sequência de `tick`s é uma linha do tempo, sem espera real.

tests/support_db.py, linhas 115–142

```python
async def insert_checks(
    sessions: SessionFactory,
    monitor_id: int,
    rows: list[tuple[str, int]],
    *,
    start: datetime,
    step: float = 1,
) -> None:
    """Grava checks concluídos em horários exatos (start, start+step, ...)."""
    async with sessions.begin() as session:
        for i, (outcome, ms) in enumerate(rows):
            at = start + timedelta(seconds=i * step)
            await session.execute(
                insert(Check).values(
                    monitor_id=monitor_id,
                    run_key=f"{monitor_id}:{i}",
                    trigger="scheduled",
                    status="completed",
                    config_version=1,
                    scheduled_at=at,
                    started_at=at,
                    finished_at=at,
                    duration_ms=ms,
                    status_code=200 if outcome in ("healthy", "slow") else None,
                    outcome=outcome,
                    explanation="x",
                )
            )
```

- `insert_checks` grava checks prontos em horários exatos, sem passar pelo worker. Os testes de métricas usam isso para montar janelas com valores conhecidos.
- **Ponto de atenção:** a linha 138 grava `status_code=200` só para `healthy` e `slow`, e `None` para os demais. Um check `unexpected_status` real tem um código (por exemplo, 500). Para as métricas isso não muda nada, porque elas não leem `status_code`, mas é uma simplificação do dado de teste.

As funções das linhas 76–112 (`reload`, `incidents_of`, `count_checks` e `update_monitor`) só abrem uma sessão e leem ou alteram uma coisa.

Padrão 1: regra pura, sem banco

`tests/unit/test_health.py` testa a regra de saúde (D2) sem banco nem rede.

tests/unit/test_health.py, linhas 18–45

```python
INITIAL = TrackerState(Health.NO_DATA, 0, 0, None, incident_open=False)


def run(
    outcomes: list[Outcome], failure: int = 3, recovery: int = 2, state: TrackerState = INITIAL
):
    tracker = HealthTracker(Thresholds(failure, recovery))
    decisions = []
    for i, outcome in enumerate(outcomes):
        obs = Observation(outcome, START + timedelta(minutes=i), f"resultado {i}")
        decision = tracker.apply(state, obs)
        decisions.append(decision)
        state = decision.state
    return state, decisions


F, S = Outcome.TIMEOUT, Outcome.HEALTHY
```

```
35
36
37 def actions(decisions):
38     return [d.incident_action for d in decisions]
39
40
41 def test_mandatory_example_fail_fail_success_opens_nothing() -> None:
42     state, decisions = run([F, F, S])
43     assert IncidentAction.OPEN not in actions(decisions)
44     assert (state.consecutive_failures, state.consecutive_successes) == (0, 1)
45     assert state.incident_open is False
```

- Linha 18: o estado inicial, sem dados.
- Linhas 21–31: `run` aplica uma sequência de resultados e devolve o estado final e as decisões. Cada observação fica um minuto depois da anterior.
- Linha 34: `F` e `S` deixam as sequências legíveis: `[F, F, S]`.
- Linhas 41–45: o primeiro caso do exemplo obrigatório: falha, falha, sucesso **não** abre incidente e deixa os contadores em (0, 1).

Arquivos com o mesmo padrão: `test_monitor_rules.py`, `test_status_codes.py`, `test_metrics_formulas.py`, `test_url_policy.py`, `test_charts.py`, `test_passwords.py`, `test_config.py`.

Padrão 2: um cenário de ponta a ponta com o worker real

`tests/integration/test_incident_lifecycle.py, linhas 21–68`

```
21 async def test_threshold_example_with_database(
22     sessions: SessionFactory, clock: FakeClock, settings: Settings
23 ) -> None:
24     """Exemplo obrigatório do RF03 de ponta a ponta (cenários 5, 6 e 8)."""
25     monitor = await create_monitor(sessions, clock)
26     target = ScriptedTarget(clock)
27     worker = make_worker(sessions, clock, target, settings)
28
29     target.mode = "timeout"
30     await tick(worker, clock)
31     await tick(worker, clock)
32     target.mode = "ok"
33     await tick(worker, clock)
34     assert await incidents_of(sessions, monitor.id) == []  # falha, falha, sucesso
35
36     target.mode = "timeout"
37     for _ in range(3):
38         await tick(worker, clock)
39     opened = await incidents_of(sessions, monitor.id)
40     assert len(opened) == 1 and opened[0].status == "open"
41     assert opened[0].opening_reason == (
42         "Incidente confirmado após 3 falhas consecutivas; último resultado: sem resposta em 5 s."
43     )
44     assert opened[0].started_at < opened[0].confirmed_at  # primeira falha antes da confirmação
45
46     for _ in range(4):  # novas falhas atualizam o mesmo incidente
47         await tick(worker, clock)
48     same = await incidents_of(sessions, monitor.id)
49     assert len(same) == 1 and same[0].failure_count == 7
50
51     target.mode = "ok"
52     await tick(worker, clock)
53     assert (await incidents_of(sessions, monitor.id))[0].status == "open"
54     await tick(worker, clock)
55     resolved = (await incidents_of(sessions, monitor.id))[0]
56     assert resolved.status == "resolved"
57     assert resolved.resolution_explanation is not None
58     assert resolved.resolution_explanation.startswith("Recuperado após 2 sucessos consecutivos")
59
```

```
    async with sessions() as session:
        kinds = list(await session.scalars(select(AlertEvent.kind).order_by(AlertEvent.id)))
        timeline = list(
            await session.scalars(
                select(IncidentEvent.kind).order_by(IncidentEvent.occurred_at, IncidentEvent.id)
            )
        )
    assert kinds == ["incident.opened", "incident.resolved"]  # nada a cada falha repetida
    assert timeline == ["first_failure", "opened", "resolved"]
```

- Linhas 25–27: monitor, alvo falso e worker real.
- Linhas 29–34: falha, falha, sucesso: nenhum incidente.
- Linhas 36–44: três falhas: um incidente aberto, com o motivo **exato** e o início anterior à confirmação.
- Linhas 46–49: mais quatro falhas atualizam o **mesmo** incidente (7 falhas no total).
- Linhas 51–58: um sucesso não encerra; o segundo encerra, com o motivo.
- Linhas 60–68: só **dois** alertas (abertura e recuperação), não um por falha, e a linha do tempo completa.

Mesmo padrão: `test_first_slice.py`, `test_delivery.py`, `test_retention.py` e a maior parte de `test_consistency.py`.

Padrão 3: provar que o banco recusa o impossível

tests/integration/test_consistency.py, linhas 104–141

```
async def test_database_rejects_second_open_incident(
    sessions: SessionFactory, clock: FakeClock, settings: Settings
) -> None:
    monitor = await create_monitor(sessions, clock, failure_threshold=1)
    target = ScriptedTarget(clock)
    target.mode = "error"
    await tick(make_worker(sessions, clock, target, settings), clock)
    [existing] = await incidents_of(sessions, monitor.id)
    with pytest.raises(IntegrityError, match="uq_incidents_one_open_per_monitor"):
        async with sessions.begin() as session:
            session.add(
                Incident(
                    monitor_id=monitor.id,
                    status="open",
                    started_at=existing.started_at,
                    confirmed_at=existing.confirmed_at,
                    failure_threshold=1,
                    recovery_threshold=1,
                    timeout_seconds=5,
                    latency_threshold_ms=1000,
                    opening_reason="duplicado",
                    failure_count=1,
                    last_outcome="timeout",
                    last_explanation="x",
                )
            )


async def test_concurrent_claims_never_take_the_same_monitor(
    sessions: SessionFactory, clock: FakeClock
) -> None:
    for i in range(10):
        await create_monitor(sessions, clock, name=f"monitor {i}")
    results = await asyncio.gather(
        *(processing.claim_due_runs(sessions, now=clock.now(), limit=10) for _ in range(4))
    )
```

```
    claimed = [run.monitor_id for batch in results for run in batch]
    assert len(claimed) == len(set(claimed)) == 10
```

- Linhas 104–129: tenta inserir **direto no banco** um segundo incidente aberto para o mesmo monitor, sem passar pelo código. O teste espera um `IntegrityError` com o nome do índice único parcial. Assim fica provado que a garantia está no banco, e não depende de o código estar certo.
- Linhas 132–141: quatro reservas **ao mesmo tempo** (`asyncio.gather`) sobre dez monitores. A soma dá exatamente dez, sem repetição: é o `FOR UPDATE SKIP LOCKED` funcionando com concorrência de verdade, em conexões separadas.

Padrão 4: segurança com rede simulada no nível do socket

`tests/unit/test_transport.py, linhas 38–82`

```
38 class RecordingBackend(httpcore.AsyncNetworkBackend):
39     def __init__(self) -> None:
40         self.connections: list[tuple[str, int]] = []
41         self.stream = RecordingStream()
42 
43     async def connect_tcp(self, host, port, timeout=None, local_address=None, socket_options=None):  #
   noqa: ASYNC109
44         self.connections.append((host, port))
45         return self.stream
46 
47     async def sleep(self, seconds: float) -> None:
48         pass
49 
50 
51 def guarded_client(resolver: FakeResolver, inner: RecordingBackend):
52     client = build_guarded_client(
53         DestinationPolicy(), max_connections=5, user_agent="teste", resolver=resolver
54     )
55     # Mesmo backend protegido, mas o socket "de verdade" é o de gravação.
56     client._transport._pool._network_backend = GuardedNetworkBackend(
57         DestinationPolicy(), resolver, inner
58     )  # type: ignore[attr-defined]
59     return client
60 
61 
62 async def test_connects_to_validated_ip_and_keeps_host_header() -> None:
63     resolver = FakeResolver({"api.example.com": ["93.184.216.34"]})
64     inner = RecordingBackend()
65     async with guarded_client(resolver, inner) as client:
66         response = await client.get("http://api.example.com/health")
67     assert response.status_code == 204
68     assert inner.connections == [("93.184.216.34", 80)]  # conectou no IP validado
69     assert b"Host: api.example.com\r\n" in inner.stream.written  # Host original preservado
70 
71 
72 async def test_rebinding_is_checked_at_connect_time() -> None:
73     """O nome era público no cadastro e passa a apontar para a rede interna."""
74     resolver = FakeResolver({"rebind.example.com": ["93.184.216.34"]})
75     await validate_destination(DestinationPolicy(), "http://rebind.example.com/", resolver)
76 
77     resolver.mapping["rebind.example.com"] = ["127.0.0.1"]
78     inner = RecordingBackend()
79     async with guarded_client(resolver, inner) as client:
80         with pytest.raises(UnsafeDestinationError, match="loopback"):
81             await client.get("http://rebind.example.com/")
82     assert inner.connections == []  # nenhuma conexão foi aberta
```

- Linhas 38–48: um *backend* de rede falso do `httpcore` (a biblioteca por baixo do httpx). Ele registra para onde tentaram conectar e devolve uma resposta pronta.
- Linhas 51–59: monta o cliente protegido de verdade e troca só o socket final. **Ponto de atenção:** a linha 56 acessa atributos **privados** do httpx (`_transport._pool._network_backend`). Funciona na versão travada no `uv.lock`, mas uma atualização do httpx pode quebrar este teste sem que o PulseOps tenha mudado.
- Linhas 62–69: conecta no IP validado e mantém o cabeçalho `Host` original.
- Linhas 72–82: o DNS *rebinding*. No cadastro, o nome aponta para um IP público e é aceito. Depois, o "DNS" passa a responder `127.0.0.1`. A requisição é recusada **e nenhuma conexão é aberta** (linha 82). Essa última linha é a prova mais importante: recusar depois de conectar já seria tarde.

Padrão 5: migrações num banco vazio

`tests/integration/test_migrations.py, linhas 31–66`

```
31 @pytest.fixture
32 def scratch_url() -> str:
33     """Banco separado, criado vazio, só para este teste."""
34     url = make_url(TEST_DATABASE_URL).set(database="pulseops_migration_test")
35     admin = url.set(drivername="postgresql", database="postgres").render_as_string(
36         hide_password=False
37     )
38     with psycopg.connect(admin, autocommit=True) as conn:
39         conn.execute('DROP DATABASE IF EXISTS "pulseops_migration_test" WITH (FORCE)')
40         conn.execute('CREATE DATABASE "pulseops_migration_test"')
41     return url.render_as_string(hide_password=False)
42
43
44 def test_upgrade_on_empty_database_creates_all_tables(scratch_url: str) -> None:
45     command.upgrade(alembic_config(scratch_url), "head")
46     engine = create_engine(scratch_url)
47     try:
48         tables = set(inspect(engine).get_table_names())
49         assert tables >= EXPECTED_TABLES
50         with engine.connect() as conn:
51             diff = compare_metadata(MigrationContext.configure(conn), Base.metadata)
52         assert diff == [], f"modelos e migração divergem: {diff}"
53     finally:
54         engine.dispose()
55
56
57 def test_downgrade_and_upgrade_again(scratch_url: str) -> None:
58     config = alembic_config(scratch_url)
59     command.upgrade(config, "head")
60     command.downgrade(config, "base")
61     engine = create_engine(scratch_url)
62     try:
63         assert set(inspect(engine).get_table_names()) == {"alembic_version"}
64     finally:
65         engine.dispose()
66     command.upgrade(config, "head")
```

- Linhas 31–41: um banco à parte, criado vazio só para este teste.
- Linhas 44–54: aplica as migrações e compara o banco resultante com os modelos (`compare_metadata`, a mesma comparação do `alembic revision --autogenerate`). Uma lista vazia de diferenças prova que a migração e os modelos dizem a mesma coisa.
- Linhas 57–66: desce até o zero (só sobra a tabela `alembic_version`) e sobe de novo.

Os demais arquivos

Arquivo	Casos	O que cobre	Padrão
`unit/test_charts.py`	4	gráfico de latência, B9 e B10	1
`unit/test_checker.py`	15	classificação de cada resultado de check	alvo falso (`ScriptedTarget`)
`unit/test_config.py`	3	padrões e validação das variáveis	1
`unit/test_health.py`	17	regra de saúde e incidentes	1
`unit/test_lab_receiver.py`	3	deduplicação do receptor do laboratório	1
`unit/test_logging.py`	4	logs JSON, contexto e segredos mascarados	1
`unit/test_metrics_formulas.py`	8	percentil e razões	1
`unit/test_monitor_rules.py`	16	regras combinadas do cadastro	1
`unit/test_passwords.py`	3	Argon2 e hash corrompido (B4)	1
`unit/test_status_codes.py`	10	leitura de `200-299,304`	1
`unit/test_transport.py`	12	SSRF no momento da conexão	4
`unit/test_url_policy.py`	38	URLs aceitas e recusadas	1
`integration/test_api_history.py`	8	paginação, filtros e janelas de tempo	API
`integration/test_api_monitors.py`	17	cadastro, ações e conflitos pela API	API
`integration/test_auth.py`	26	sessão, CSRF, cookies e redirecionamento	API e páginas
`integration/test_cli.py`	2	`pulseops create-admin`	2
`integration/test_consistency.py`	14	duplicidade, concorrência, reinício e B8	2 e 3
`integration/test_dashboard.py`	12	páginas, escape e cabeçalhos	páginas
`integration/test_delivery.py`	17	outbox, retentativas e webhook	2
`integration/test_first_slice.py`	1	o primeiro fluxo completo	2
`integration/test_health_endpoints.py`	2	`/health/live` e `/health/ready`	API
`integration/test_incident_lifecycle.py`	9	ciclo de vida dos incidentes	2
`integration/test_metrics.py`	6	métricas sobre dados gravados	`insert_checks`
`integration/test_migrations.py`	2	migração do zero e volta	5
`integration/test_retention.py`	2	limpeza de checks antigos, preservando incidentes	2

“API” e “páginas” querem dizer: o teste usa as fixtures `api` ou `app_client` e confere o status, o JSON ou o HTML devolvido.

Como estudar este capítulo. Escolha um teste da tabela, leia só o nome e escreva o que você acha que ele faz. Depois abra o arquivo e compare. Em seguida, quebre de propósito a linha do código que ele protege (numa cópia, como na parte J do volume 1) e confira que o teste falha. Um teste que não falha quando o código está errado não protege nada.